СОДЕРЖАНИЕ

Введение 7
1 Информационные системы открытых электронных архивов 9
1.1 Принципы построения открытых электронных архивов 9
1.2 Обзор референсных веб-приложений для разработки открытого электронного архива 11
1.3 Выводы по главе и постановка задач на дипломное проектирование .. 18
2 Эргономическое преоктирвароние веб-приложения «Открытый электронный архив истории г. Витебска» 22
2.1 Разработка алгоритмов работы пользователя 22
2.2 Разработка эргономических требований и сценария информационного взаимодействия 33
2.3 Эргономическая оценка проектируемого веб-приложения и выводы .. 48
3 Разработка веб-приложения «Открытый электронный архив истории г. Витебска» 53
3.1 Архитектура веб-приложения 53
3.2 Алгоритм работы веб-приложения 57
3.3 Структура базы данных веб-приложения 63
3.4 Выводы и оценка результатов разработки 71
4 Тестирование веб-приложения открытого электронного архива истории г. Витебска 73
5 Технико-экономческое обоснование себестоимочсти цены веб-приложения «Открытый электронный архив истории г. Витебска» 87
5.1 Характеристика веб-приложения 87
5.2 Определение объема и трудоемкости веб-приложения 87
5.3 Расчет себестоимости и цены веб-приложения 90
6 Охрана труда. Расчет естественного освещения в производственном помещении 97
Заключение 102
Список использованных источников 104
Приложение А (обязательное) Отчет о проверке на заимствование 107
Приложение Б (обязательное) Листинг программы 108

ВВЕДЕНИЕ

В последние годы в стране наблюдается устойчивая тенденция к цифровизации архивов и популяризации локальной истории, что проявляется в росте интереса у граждан к изучению электронных баз данных, краевых порталов и онлайн-сообществ, посвященных прошлому своих регионов. Современные пользователи стремятся не только получать информацию, но и активно делиться ею, внося свой вклад в сохранение исторического наследия. Однако существование способы распространения краеведческих материалов, исторических заметок, вспоминании и архивных документов нередко ограничены: информация о прошлом рассредоточена по разным источникам – печатным изданиям, социальным сетям, специализированным ресурсам, что затрудняет её поиск и систематизацию [1]. Обычные жители, желающие рассказать о событиях, людях или местах своего города, нередко не имеют инструмента для публикации собственных материалов, а исследователям и историкам сложно находить и подтверждать достоверность данных.

Целевая аудитория разработки – жители города Витебска, краеведы, историки, энтузиасты локальной истории, а также муниципальные учреждения культуры.

Создание централизованной платформы позволит систематизировать исторические данные города, повысить доступность культурного наследия для широких слоёв населения, снизить временные затраты на поиск архивных материалов и сформировать сообщество граждан, участвующих в сохранении исторической памяти.

В настоящее время существует ряд информационных систем в сфере архивного дела и краеведения. К ним относятся проекты цифровизации («Беларусь помнит»), региональные краеведческие порталы, а также специализированные платформы для совместной работы с историческими данными (FamilySearch) [2] [3]. Однако данные решения либо не предполагают активного участия пользователей в наполнении контента, либо ориентированы на узкие задачи (генеалогия, история Великой Отечественной Войны). Таким образом, проблема создания открытой краеведческой платформы с элементами социальной сети и современными механизмами поиска остаётся актуальной.

Цель дипломного проектирования — разработка информационной системы «Открытый электронный архив города Витебска», обеспечивающей централизованное хранение, интеллектуальный поиск и коллективное наполнение краеведческими материалами.

Объект исследования — процессы сбора, хранения, обработки и предоставления краеведческой информации в цифровой среде.

Предмет исследования — принципы проектирования веб-ориентированных информационных систем с применением технологий полнотекстового поиска (Elasticsearch), систем управления контентом,

механизмов пользовательской модерации и современными техническими решениями для веб-разработки.

Для достижения поставленной цели необходимо решить следующие задачи:

1 Провести анализ предметной области открытых электронных архивов и обзор существующих веб-приложений, сформулировать требования к разработке.

2 Разработать алгоритмы работы пользователя, эргономические требования и сценарий информационного взаимодействия, провести эргономическую оценку проектируемого веб-приложения

3 Разработать архитектуру веб-приложения, алгоритм работы системы, спроектировать структуру базы данных.

4 Провести функциональное тестирование веб-приложения.

5 Выполнить технико-экономическое обоснование разработки, рассчитать себестоимость и цену веб-приложения.

6 Разработать мероприятия по охране труда, выполнить расчёт естественного освещения в производственном помещении

Разрабатываемая система представляет собой открытый электронный архив города Витебска, отличающийся следующими особенностями: возможностью самостоятельной публикации материалов пользователями с предварительной модерацией для обеспечения достоверности; интеллектуальным поиском с учётом опечаток, вариантов написания и контекстных предложений; фасетной фильтрацией для навигации по архиву. В отличие от существующих аналогов, система объединяет функции архивного хранилища и социальной платформы, что способствует активному вовлечению граждан в процесс сохранения культурной памяти.

Данный дипломный проект выполнен мной лично, проверен на заимствования, процент оригинальности составляет 97% (отчет о проверке на заимствования прилагается).

1 ИНФОРМАЦИОННЫЕ СИСТЕМЫ ОТКРЫТЫХ ЭЛЕКТРОННЫХ АРХИВОВ

Традиционные модели архивного дела, построенные на принципах ограниченного доступа, перестают отвечать потребностям информационного общества, где пользователи привыкли к мгновенному получению информации и активному участию в её создании. Открытые электронные архивы призваны заполнить этот пробел, предложив вариант совместного формирования и использования исторических данных.

Настоящий раздел посвящён анализу предметной области открытых электронных архивов и обзору существующих веб-приложений, на основе которых формируются требования к разрабатываемой информационной системе. В разделе 1.1 рассматривается шаги для перехода от классичеких архивных систем к открытым краудсорсинговым платформам, выявляются концептуальные требования и необходимые модули системы. Раздел 1.2 содержит детальный анализ трёх референсных систем, выбранных в качестве источников для заимствования и адаптации функциональных решений: Kufar.by как образца поиска, Instagram как платформы социального взаимодействия и Library.by как системы организации процессов публикации. В разделе 1.3 формулируются выводы по результатам анализа и конкретные задачи дипломного проектирования, определяющие направления дальнейшей разработки.

1.1 Принципы построения открытых электронных архивов

В современном информационном обществе наблюдаются тенденции к свободности доступа к историческим и культурным данным. Традиционные архивные системы, функционирующие по модели государственных учреждений, предполагают строго регламентированный порядок доступа: физическое присутствие исследователя, предварительное составление заявок, согласование сроков ознакомления с материалами, а нередко и оплата услуг по копированию документов [5]. Этот подход исключает широкие слои населения, ограничивая круг пользователей профессиональными историками и архивистами, обладающими соответствующей подготовкой и ресурсами.

Открытые электронные архивы представляют собой совершенно иной принцип организации доступа к исторической информации. Ключевым отличием выступает отказ от закрытой модели в пользу открытой платформы, где любой пользователь может не только получить доступ к материалам, но и внести собственный вклад в сохранение культурного наследия. Данный подход получил название краудсорсинга, но его реализация требует соблюдения определённых принципов, обеспечивающих баланс между открытостью и достоверностью накапливаемых данных [6].

Концептуальные требования к разрабатываемой информационной системе определяются спецификой краеведческой деятельности и

необходимостью объединения функций архивного хранения и социальной платформы. Ключевыми концептуальными требованиями являются: обеспечение открытого доступа к историческим материалам; поддержка коллективного наполнения архива с сохранением контроля качества через модерацию; реализация интеллектуального поиска, адаптированного к неструктурированным историческим данным и типичным ошибкам пользователей; создание условий для формирования сообщества через механизмы подписок, рекомендаций и обратной связи; обеспечение прозрачности процессов публикации для всех участников. Реализация указанных требований обеспечивает трансформацию архива из пассивного репозитория в динамичное пространство, где пользователи выступают одновременно и потребителями, и создателями контента.

Проводя анализ информационных потребностей целевой аудитории можно выделить несколько категорий пользователей с различными сценариями взаимодействия:

7 Обычные жители города нуждаются в простом инструменте для публикации семейных фотографий, воспоминаний и личных историй, связанных с городом, с возможностью получить обратную связь от сообщества.

8 Краеведы и местные историки требуют эффективных средств поиска и систематизации разрозненных материалов.

9 Исследователи и представители муниципальных учреждений культуры заинтересованы в привлечении внимания жителей к истории их города.

Все категории пользователей объединяет потребность в интуитивном интерфейсе, минимизирующем необходимость обучения, и в оперативном получении уведомлений о появлении релевантных материалов.

На основании проведённого анализа в состав информационной системы выделяются следующие основные модули:

– модуль управления пользователями;
– модуль создания и управления материалами;
– модуль интеллектуального поиска;
– модуль модерации;
– модуль социального взаимодействия;
– модуль хранения данных.

Модуль управления пользователями обеспечивает аутентификацию и авторизацию, формируя базу данных профилей, которые используются модулем социального взаимодействия для организации подписок и рекомендаций. Модуль создания материалов взаимодействует с модулем хранения для размещения файлов и с модулем модерации для инициации процесса проверки. После одобрения материала модуль модерации передаёт данные в модуль интеллектуального поиска для индексации, обеспечивая доступность материала через поисковый интерфейс. Модуль социального взаимодействия получает события об активности пользователей (новые

публикации, лайки) и формирует ленты рекомендаций. Такая модульная структура обеспечивает независимое развитие компонентов и масштабируемость системы при росте объёма данных и числа пользователей.

1.2 Обзор референсных веб-приложений для разработки открытого электронного архива

Проектирование открытых электронных архивов требует нестандартного и комплексного подхода, объединяющего достижения из различных сфер. Успешные решения в смежных областях – от коммерческих площадок до социальных сетей демонстрируют эффективные паттерны взаимодействия с пользовательским контентом, которые могут быть адаптированы для архивного контекста.

Как следствие разработка эффективной информационной системы предполагает анализ и адаптацию успешных решений из смежных предметных областей. В качестве референсных источников для проектирования отдельных подсистем выбраны три веб-приложения, демонстрирующие успешные варианты реализации в соответствующих направлениях: Kufar.by как образец реализации поиска и фильтрации, Instagram как платформа социального взаимодействия и управления контентом, а также Library.by как система организации процессов публикации и модерации документов.

Анализ системы поиска на примере Kufar.by

Платформа Kufar.by представляет собой крупнейший белорусский сервис объявлений, где критически важную роль играет эффективность поисковой подсистемы [7].

Поиск работает на основе индексирования объявлений, что позволяет быстро обрабатывать запросы к большому массиву данных. При вводе запроса система выдаёт результаты, сгруппированные по степени соответствия: в первую очередь показываются объявления с точным совпадением слов в заголовке, затем – с частичным совпадением заголовка, далее – с совпадением только в тексте описания. Результаты поиска представлены на рисунке 1.1

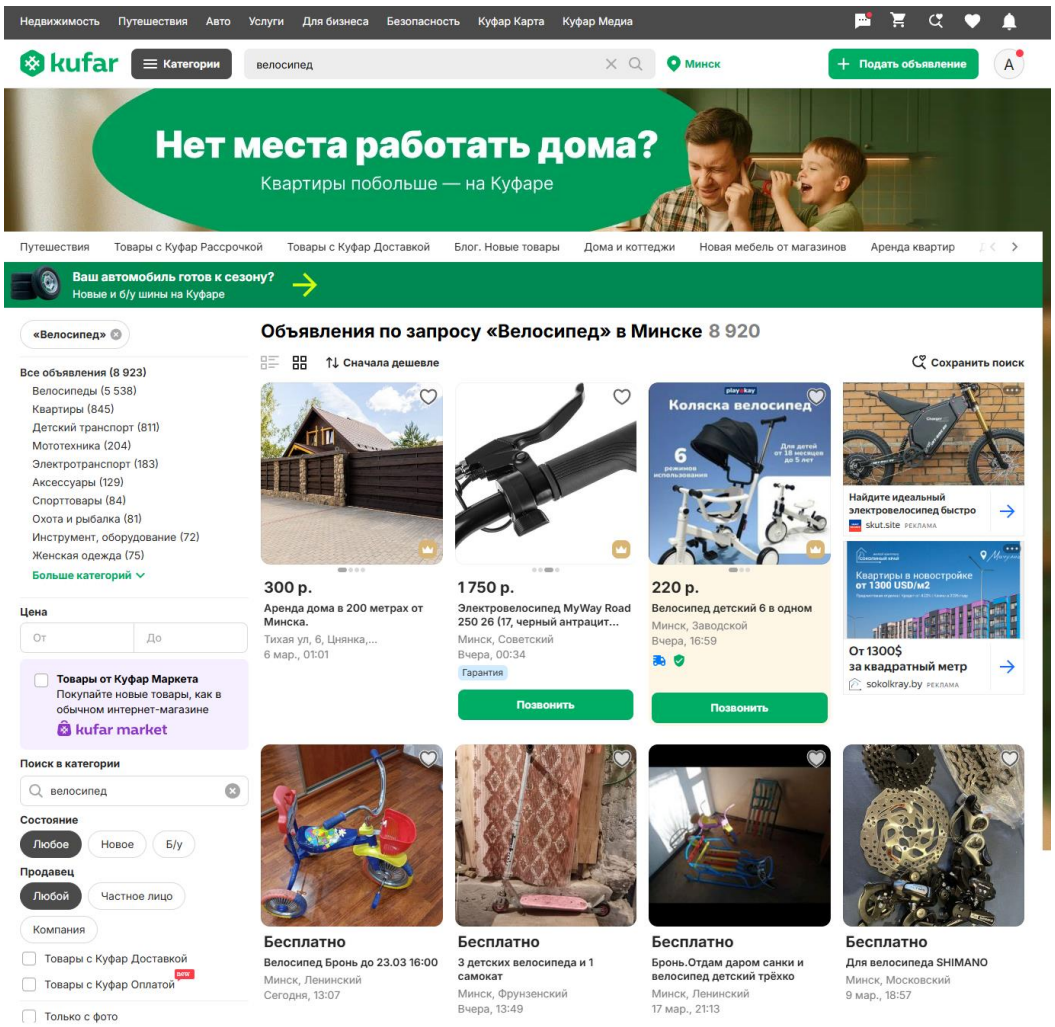

Рисунок 1.1 – Результаты поиска Kufar.by.

Внутри каждой группы применяется выбранная пользователем сортировка, например по новизне или цене. Кроме того, система предлагает контекстные подсказки при вводе – релевантные варианты из существующего контента и подходящие категории для уточнения поиска. Варианты подсказок приведены на рисунке 1.2

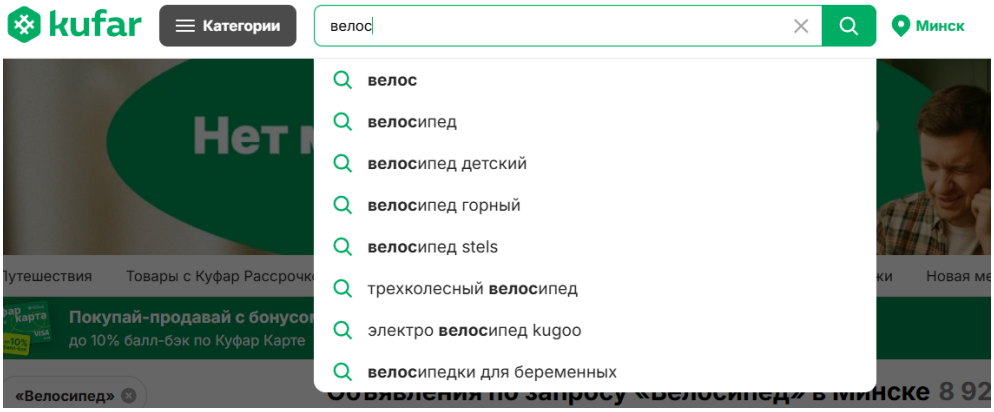

Рисунок 1.2 – Варианты подсказок Kufar.by

Однако стоит отметить, что при перестановке букв или явных опечатках качество поиска заметно снижается: например, запрос «клавиатюра» или «велосепед» уже не даёт тех же результатов, что и правильное написание. Отсутствие результатов при неправильно поисковом запросе приведено на рисунке 1.3

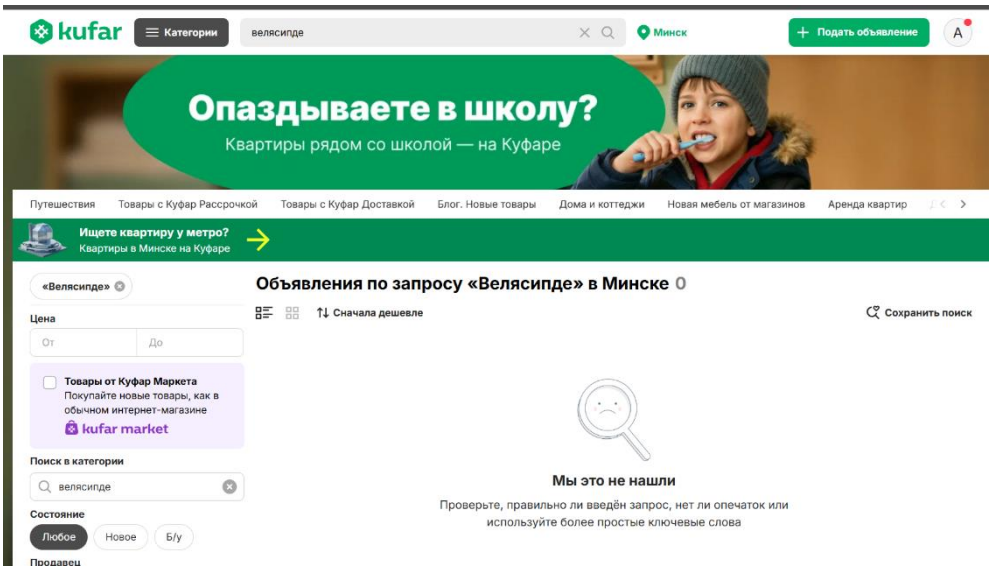

Рисунок 1.3 – Отсутствие результатов поиска Kufar.by

Фасетная навигация Kufar реализована на высоком уровне и представляет собой систему динамически пересчитываемых фильтров. При выборе одного параметра автоматически обновляются доступные значения всех остальных фильтров, исключая комбинации с нулевой выдачей – так называемый механизм «живых» или «зависимых» фасетов (dependent facets). . Каждый фильтр отображает числовой индикатор – количество объявлений, соответствующих данному значению, что позволяет пользователю оценить информативность применения фильтра до его активации. Система поддерживает множественный выбор в пределах одного фасета и иерархическую вложенность. Реализация фасетов приведена на рисунке 1.4

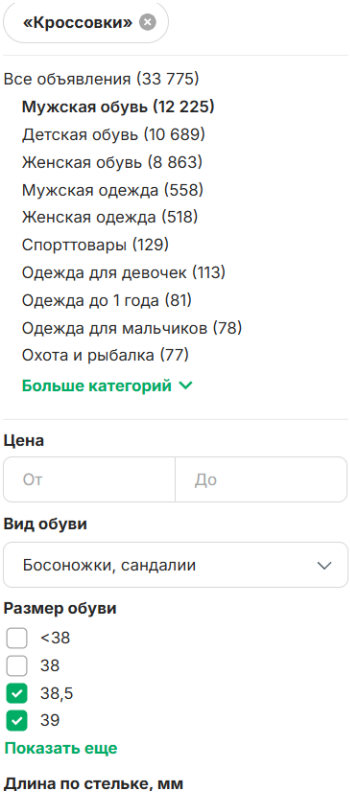

Рисунок 1.4 – Реализация фасетов Kufar.by

Для переноса в архивную систему следует перенять механизм ранжирования по степени соответствия: приоритет заголовков и метаданных над полнотекстовым содержимым обеспечивает предсказуемость выдачи, что критично для работы с историческими источниками. Фасетная навигация с «живыми» фильтрами и числовыми индикаторами позволит исследователям оценивать объём выборки до её применения. Контекстные подсказки при вводе ускорят работу с редкими словами. Но также необходимо уделить внимание улучшению механизма обработки ошибок: для архивной системы недопустима потеря результатов из-за опечатки в устаревшем названии. Кроме того, следует расширить семантические связи: архив должен связывать материалы по контексту.

Таким образом, Kufar демонстрирует эффективную базовую модель поиска и навигации, но для исторических материалов требуется её существенное усиление механизмами лингвистической нечёткости и историко-семантических ассоциаций.

Социальная платформа на примере Instagram

Приложение Instagram демонстрирует эффективную модель построения социальных взаимодействий вокруг пользовательского контента [8].

Алгоритмическая лента рекомендаций, формируемая на основе поведенческих сигналов (просмотры, лайки, подписки), обеспечивает персонализированную подачу материалов даже без активного поискового запроса. Примеры результатов алгоритма рекомендаций представлены на рисунках 1.5 и 1.6

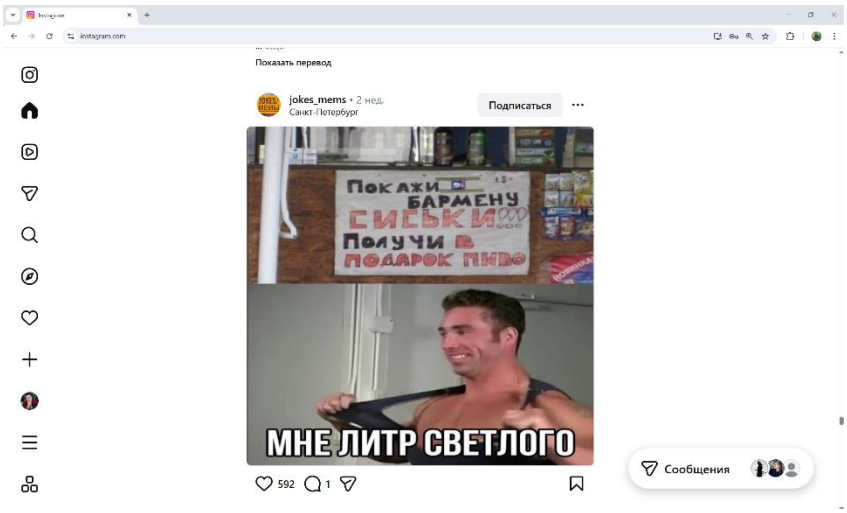

Рисунок 1.5 – Пост из ленты рекомендаций Instagram

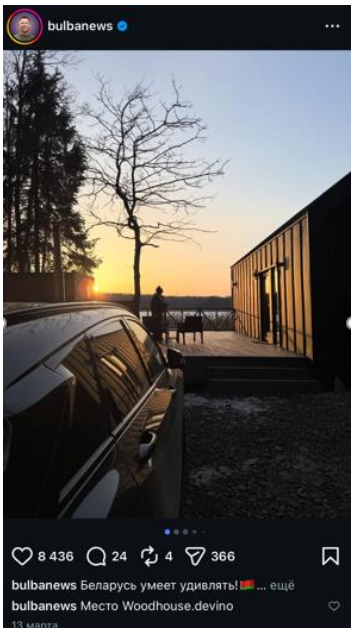

Рисунок 1.6 – Пост из ленты рекомендаций Instagram

Система лайков и подписок создаёт механизм обратной связи. Данные показатели стимулируют авторов к публикации качественного контента через механизм социального признания.

Личный кабинет Instagram объединяет управление собственными публикациями, показатели вовлечённости по каждому материалу, сохранённые материалы с возможностью тематической сортировки, настройки приватности и модерации комментариев в едином интерфейсе. Особенностью является разграничение контента по статусам: опубликованные материалы, черновики, архивированные публикации, избранное – каждый со своим набором действий. Главная страница личного кабинета представлена на рисунке 1.7.

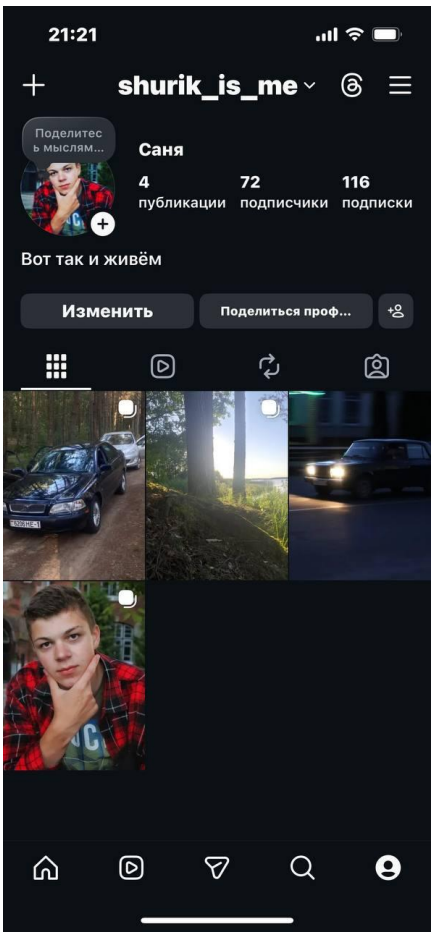

Рисунок 1.7 – Личный кабинет Instagram

Анализ функциональности Instagram выявляет значительный потенциал для адаптации в социально направленной архивной системе. Перенять рекомендуется механизм алгоритмической ленты: для архива это означает автоматическую подборку исторических материалов по темам, ранее вызывавшим интерес пользователя. Это решает проблему «холодного старта» исследователя, который ещё не знает, что именно искать в архиве, и стимулирует пассивное знакомство с культурным наследием. Система лайков и подписок позволит выстраивать репутацию авторов-краеведов и формировать тематические сообщества вокруг узких направлений. Личный кабинет с разграничением статусов публикаций (черновик, на модерации, опубликовано, архив) критически важен для прозрачности workflow. Улучшить необходимо механизм формирования рекомендаций. Для архивной системы приоритетом становится не удержание внимания, а образовательная ценность и достоверность – алгоритм должен приоритизировать материалы с высокой степенью доверия и преднамеренно включать контент за пределами узкой зоны интересов пользователя для расширения кругозора. Кроме того, требуется усиление механизмов сохранения: в Instagram материалы хранятся на платформе без возможности экспорта, тогда как архив должен предоставлять возможность скачать открытый документ.

Как итог, Instagram демонстрирует эффективные паттерны социального взаимодействия и персонализации, но их перенос в архивную среду требует переориентации с развлечения на образование и с вирусности на достоверность.

Система публикации и модерации на примере Library.by

Библиотечная онлайн платформа Library.by иллюстрирует организацию полного жизненного цикла документа от загрузки до публикации [9].

Многоэтапная форма создания записи предусматривает последовательное заполнение библиографических данных, загрузку файлов в различных форматах (PDF, docx, изображения), назначение рубрикаторов и ключевых слов. Интеграция OCR-модулей позволяет автоматически распознавать текст отсканированных документов, делая их содержимое доступным для полнотекстового поиска.

Система модерации Library.by построена на принципе статусных переходов: материал последовательно проходит статусы «на рассмотрении», «на доработке», «опубликован» или «отклонён». Каждый переход сопровождается уведомлением автора с комментарием модератора, обеспечивая прозрачность процесса и возможность постепенного улучшения публикации. Для архивного приложения данный подход критически важен, поскольку обеспечивает баланс между открытостью платформы и достоверностью размещаемой исторической информации. Пример формы создания статьи приведен на рисунке 1.8.

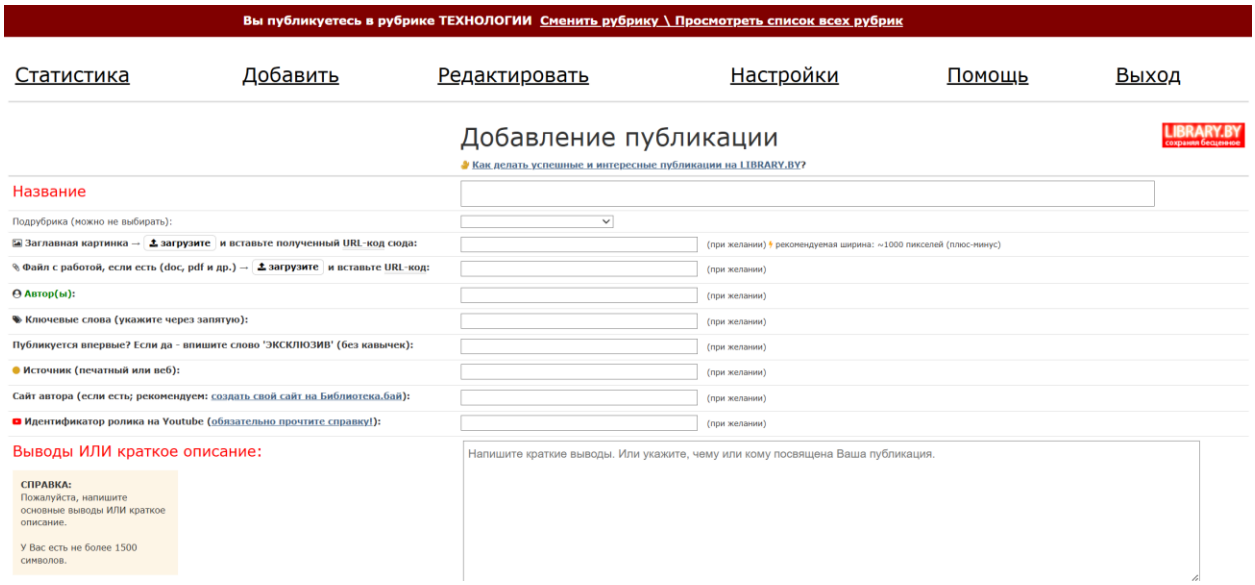

Рисунок 1.8 – Форма создания статьи Library.by

На примере Library.by можно выявить ключевые элементы для архивной системы: многоэтапная форма создания записи, обеспечивающая структурированность метаданных, и интеграция OCR для оцифровки документов, система статусных переходов с уведомлениями авторов

критически важна для баланса открытости и достоверности: она исключает некачественные материалы, давая возможность доработки.

Комплексный анализ трёх референсных систем позволяет сформулировать принципы проектирования архивной платформы. От Kufar.by следует перенести фасетную навигацию и ранжирование по релевантности, усилив механизм нечёткого поиска для компенсации ошибок ввода исторических терминов. Instagram демонстрирует эффективность алгоритмических рекомендаций и социальных механизмов, однако для архивного контекста требуется переориентация ценностей. Library.by задаёт образец структурированного ввода метаданных и прозрачной модерации, обеспечивающей баланс открытости и качества. Таким образом, проектируемая система должна объединить точность поиска, социальную вовлеченность и контроль достоверности – три компонента, критических для успеха открытого электронного архива.

1.3 Выводы по главе и постановка задач на дипломное проектирование

Проведённый анализ предметной области открытых электронных архивов и обзор существующих референсных веб-приложений позволяет сформулировать комплекс требований к разрабатываемой системе и определить направления дипломного проектирования.

Цель разработки информационной системы «Открытый электронный архив города Витебска» – создание единой цифровой среды для сохранения историко-культурного наследия города Витебска путем объединения краеведческих материалов в одном месте и вовлечения граждан в процесс развития архива.

Практическое назначение системы заключается в предоставлении инструментов для эффективной работы с историческими данными и накопления достоверной информации в базе знаний.

В течение разработки системы необходимо реализовать следующие подсистемы:

– Подсистема управления пользователями – регистрация, авторизация, управление ролями, профили пользователей.

– Подсистема создания и управления материалами – загрузка файлов, заполнение метаданных, редактирование, удаление, отслеживание статуса модерации.

– Подсистема поиска – полнотекстовый поиск, фасетная фильтрация, автокоррекция запросов.

– Подсистема модерации – проверка материалов, присвоение статусов, обратная связь с авторами.

– Подсистема социального взаимодействия – система лайков, подписки на авторов и категории, уведомления, лента рекомендаций.

– Подсистема хранения данных – управление медиафайлами в объектном хранилище, OCR-обработка документов.

Для информационного обмена между системами следует использовать брокер сообщений (например, Kafka), что способствует большей независимости подсистем.

Архив должен предоставлять круглосуточный доступ к опубликованным материалам, с использованием фоновых процессов для оцифровки документов и системы нотификаций.

Взаимодействие со смежными системами не планируется, но может быть добавлено в ходе разработке или последующем развитии системы. Также приоритетными направлениями развития системы могут стать расширение форматов исторических записей, разработка мобильного приложения и внедрение машинного обучения для более глубокого анализа поисковых запросов.

Основой успеха любой системы является её привлекательность с точки зрения удобства, поэтому система должна обеспечивать высокий уровень управляемости и освояемости. Как следствие предъявляются следующие эргономические требования:

– единообразие поведения элементов интерфейса;

– визуальное акцентирование основных управляющих элементов;

– необходимость подтверждения необратимых действий;

– поддержку единого стиля во всех частях системы;

– предсказуемость результатов действий.

Психофизиологические требования предполагают достаточный размер текста (не менее 12–14 пунктов) и интерактивных элементов (минимум 40×40 пикселей), оптимальный контраст между фоном и символами (соотношение не менее 4.5:1), минимизацию зрительных переходов при работе, поддержку клавиатурной навигации. Целевой интегральный показатель эргономичности должен быть не ниже 0,8.

По сравнению с существующими решениями разрабатываемая система обладает следующими конкурентными преимуществами:

1 Комбинирование функций архива и социальной платформы – система обеспечивает активное взаимодействие пользователей и формирование сообщества единомышленников.

2 Интеллектуальный поиск с учётом специфики исторических материалов.

3 Прозрачная система модерации с обратной связью – многостадийный процесс проверки с комментариями модераторов и возможностью доработки материалов, обеспечивающий баланс открытости и достоверности.

4 Встроенная OCR-обработка – автоматическое распознавание текста на загружаемых сканах документов делает их содержимое доступным для полнотекстового поиска без дополнительных действий пользователя.

Пользователи – это основа веб-приложения архива, поэтому основные усилия в первую очередь должны быть направлены на реализацию их функционала, включающего в себя:

– регистрация и управление личным профилем

– загрузка материалов с заполнением метаданных

– отслеживание статуса модерации созданных материалов и получение обратной связи

– поиск материалов с использованием интеллектуальных подсказок и фильтров

– добавление материалов в избранное

– подписка на интересующие категории, теги и авторов

– получение персонализированных рекомендаций и уведомлений о новых публикациях.

Возможности администратора отходят на второй план, так как их разработка является экономически менее выгодной. Поскольку способны реализовываться при помощи внешних инструментов, хотя и более трудных в освоении, но предоставляющих необходимый функционал:

– уравление пользователями: назначение ролей, блокировка учётных записей;

– управление структурой архива: создание и редактирование категорий, тегов, справочников;

– мониторинг системы;

– резервное копирование и восстановление данных.

Выбор технологического стека обусловлен требованиями к производительности, масштабируемости и специфике обрабатываемых данных.

Серверная часть: Spring Framework. Выбор Java и экосистемы Spring обусловлен состоятельностью платформы, развитой поддержкой микросервисной архитектуры (Spring Boot, Spring Cloud), встроенными механизмами безопасности и транзакционности. Spring Data обеспечивает удобную интеграцию с Elasticsearch и реляционными базами данных [10].

Клиентская часть: JavaScript обеспечивает создание отзывчивого пользовательского интерфейса с динамическим обновлением контента без перезагрузки страницы, что критично для поисковой системы и ленты рекомендаций [11].

Поисковый движок: Elasticsearch. Специализированное решение для полнотекстового поиска обеспечивает морфологический анализ русского и белорусского языков, нечёткий поиск с редакционным расстоянием, агрегации для фасетной навигации и горизонтальное масштабирование при росте индекса [12].

Брокер сообщений: Apache Kafka. Асинхронная коммуникация между микросервисами через Kafka обеспечивает отказоустойчивость при пиковых нагрузках (массовая загрузка материалов), гарантирует доставку событий

модерации и уведомлений, позволяет масштабировать обработчиков независимо от генераторов событий [13].

Кэширование: Redis. Использование Redis для кэширования сессий пользователей, популярных запросов и результатов поиска снижает нагрузку на основную базу данных и ускоряет отклик системы [14].

OCR: Tesseract. Открытая библиотека оптического распознавания символов с поддержкой более 100 языков, включая русский и белорусский, обеспечивает извлечение текстового слоя из отсканированных документов без лицензионных затрат [15].

Хранилище файлов: S3-совместимое объектное хранилище. Облачное хранилище с API S3 обеспечивает неограниченную масштабируемость для медиафайлов [16].

Таким образом, результаты анализа, проведенного в первом разделе, позволили сформировать требования и задачи дипломного проектирования, которые необходимо реализовать в последующих разделах дипломного проекта. Ожидаемым результатом дипломного проектирования является функционирующий прототип веб-приложения, демонстрирующий возможность эффективного объединения функций архивного хранилища и социальной платформы.

Критериями оценки успешности проекта служат: реализация полного жизненного цикла пользовательского, работоспособность интеллектуальных функций поиска, система избранного и подписок, релевантное использование системы рекомендаций и уведомлений, а также положительная оценка эргономичности интерфейса (не ниже 0,8) по результатам экспертного анализа.

2 ЭРГОНОМИЧЕСКОЕ ПРЕОКТИРВАРОНИЕ ВЕБ-ПРИЛОЖЕНИЯ «ОТКРЫТЫЙ ЭЛЕКТРОННЫЙ АРХИВ ИСТОРИИ Г. ВИТЕБСКА»

Эргономическое проектирование веб-приложения открытого электронного архива истории г. Витебска осуществляется в соответствии с антропоцентрическим подходом, при котором главной задачей является обеспечение безопасных и комфортных условий труда пользователя в системе «человек–машина–среда». Данный подход предполагает детальный анализ деятельности пользователей, распределение функций между человеком и компьютером, а также разработку требований к интерфейсу, обеспечивающих высокую эффективность взаимодействия. При проектировании учитываются особенности целевой аудитории, включающей краеведов, историков, обычных горожан и администраторов архива, которые различаются уровнем компьютерной грамотности и целями использования системы.

Структура эргономического проектирования включает последовательное выполнение следующих этапов:

– разработку алгоритмов работы пользователей для формализации сценариев взаимодействия с системой;

– формулирование эргономических требований и создание сценариев информационного взаимодействия;

– проведение эргономической оценки проектируемого веб-приложения с количественным определением степени соответствия разработанным требованиям.

Реализация данных этапов позволяет создать интуитивно понятный и эффективный пользовательский интерфейс, минимизирующий когнитивную нагрузку и снижающий вероятность ошибок при работе с архивом.

2.1 Разработка алгоритмов работы пользователя

Назначение системы – веб-приложение открытого электронного архива обеспечивает загрузку пользователями материалов (фото, статьи, документы, заметки), которые после модерации оказываются в общем доступе, а также интеллектуальный поиск по этим материалам. Таким образом, система способствует упорядочению и сохранению цифрового культурного наследия, а также повышает доступность информации о прошлом города для широкой аудитории.

Задачи проектируемого веб-приложения:

1. Обеспечить возможность загрузки материалов пользователями с описанием через метаданные и публикацией в открытом доступе.

2. Реализовать поиск материалов по метаданным и содержимому с поддержкой фасетной навигации и исправления запросов.

3. Обеспечить администраторам инструменты модерации и управления архивом для контроля качества и достоверности публикуемых материалов.

4. Обеспечить надежное хранение и сохранность материалов с круглосуточным доступом к архиву.

Для эффективного функционирования открытого электронного архива города Витебска система должна обеспечивать широкий спектр возможностей, охватывающих как взаимодействие пользователей с платформой, так и администрирование и поддержку целостности данных. Анализ содержания основных функций системы является важным этапом проектирования, позволяющим определить структуру, возможности и логику взаимодействия пользователей с системой [17]. Ниже представлено подробное описание функций компьютерной системы, обеспечивающих полный цикл работы с архивом — от регистрации и загрузки материалов до их модерации, публикации и дальнейшего использования.

Функции компьютерной системы и их описание:

1 Регистрация пользователя.

Позволяет новым пользователям создать учетную запись, введя имя, email и пароль. Функция обеспечивает формирование базы зарегистрированных участников и защищает систему от несанкционированного доступа.

2 Авторизация пользователя.

Обеспечивает вход в систему с использованием email и пароля, с проверкой корректности введенных данных. Она гарантирует безопасное использование личных данных и индивидуальный доступ к функциям архива.

3 Сброс пароля.

Позволяет пользователю восстановить доступ к учетной записи через электронную почту при утрате пароля. Данная функция повышает удобство работы и снижает риск потери данных при технических ошибках пользователя.

4 Просмотр личного кабинета.

Отображает все загруженные пользователем материалы, их статус модерации, статистику просмотров и лайков. Это помогает пользователю контролировать активность и оценивать интерес аудитории к своим публикациям.

5 Редактирование профиля.

Позволяет пользователю изменять персональные данные, аватар и контактную информацию. Функция способствует поддержанию актуальности информации о пользователе и улучшает коммуникацию внутри платформы.

6 Загрузка материала.

Обеспечивает добавление пользователем файлов (тексты, фото, документы) с заполнением метаданных. Это основная функция, обеспечивающая наполнение архива и участие пользователей в развитии проекта.

7 Редактирование материала.

Позволяет пользователю изменять описание, метаданные или загруженный файл. Она даёт возможность оперативно вносить корректировки и повышает качество публикуемых материалов.

8 Удаление материала.

Позволяет пользователю удалять собственные материалы из системы. Функция помогает поддерживать актуальность архива и управлять личным контентом.

9 Публикация материала.

Обеспечивает размещение проверенного материала в открытом доступе после успешной модерации. Это завершает процесс взаимодействия пользователя с системой и делает информацию доступной широкой аудитории.

10 Модерация материалов.

Предоставляет администраторам инструменты для проверки загруженных материалов на соответствие правилам и качеству. Функция обеспечивает достоверность данных и поддерживает высокий уровень доверия к архиву.

11 Присвоение статуса материала.

Позволяет модераторам помечать материал как «опубликован», «удален» или «требует доработки» с комментариями. Это упрощает обратную связь с пользователями и повышает прозрачность процесса проверки.

12 Поиск по содержимому.

Обеспечивает возможность поиска материалов по тексту, метаданным и другим атрибутам с учетом исправления опечаток. Функция помогает пользователям быстро находить нужную информацию даже при неточном вводе запроса.

13 Фасетный поиск.

Позволяет пользователю фильтровать результаты поиска по метаданным. Данная функция делает навигацию по большому количеству данных интуитивной и удобной.

14 Просмотр материала.

Обеспечивает отображение содержимого материала, его метаданных и статистики просмотров. Это позволяет пользователям полноценно знакомиться с контентом и оценивать его значимость.

15 Оценка материала.

Обеспечивает возможность оценивать материалы, формируя рейтинговую систему популярности. Функция стимулирует пользователей создавать качественные публикации и поддерживает активность сообщества.

16 Отслеживание статуса модерации.

Позволяет пользователям видеть текущий статус проверки своих материалов и получать уведомления о результатах модерации. Это делает процесс взаимодействия с администрацией прозрачным и понятным.

17 Управление ролями пользователей.

Обеспечивает администраторам назначение или изменение ролей (обычный пользователь, эксперт, модератор) для контроля прав доступа. Функция обеспечивает гибкость управления системой и распределение ответственности между участниками.

18 Хранение и резервирование данных.

Обеспечивает сохранность всех материалов и метаданных с возможностью восстановления после сбоев или обновлений. Это гарантирует надежность работы архива и предотвращает потерю важной информации.

19 Создание категорий и тегов.

Позволяет администраторам или экспертам формировать структуру архива для удобной навигации и классификации материалов. Данная функция обеспечивает логическую организацию контента и улучшает поиск.

20 Уведомления пользователей.

Позволяет информировать пользователей о статусе модерации, комментариях к материалам или изменениях в архиве. Это способствует оперативной коммуникации между участниками платформы.

21 Экспорт материала.

Обеспечивает возможность скачивания материалов для личного использования или оффлайн-работы с ними, с учетом прав доступа. Функция расширяет возможности использования архива вне онлайн-среды.

22 Получение уведомлений.

Позволяет пользователю получать уведомления о новых материалах и изменениях в статусе модерации. Это обеспечивает постоянное информирование и вовлеченность пользователей.

23 Подписка на материалы.

Обеспечивает пользователю возможность подписываться на интересующие материалы (категории, авторы, тип материала). Функция способствует персонализации контента и повышает интерес к платформе.

24 Рекомендации похожих материалов.

Предоставляет пользователю рекомендации материалов, связанных по тематике или автору, на основе метаданных и содержимого. Это улучшает пользовательский опыт и способствует более глубокому изучению контента архива.

Проведённый анализ основных функций системы показывает, что проектируемая платформа обладает продуманной и комплексной функциональной структурой. Все ключевые этапы взаимодействия пользователя с системой логически связаны между собой. Особое внимание уделено механизмам поиска и рекомендаций, что повышает эффективность работы с большим объёмом данных. Реализация данных функций позволяет достичь основной цели системы – упрощения процесса публикации, хранения и поиска исторических материалов, а также способствует сохранению и популяризации культурного наследия города Витебска.

Но для обеспечения эффективного функционирования веб-приложения необходимо не только определить функции но и чётко разграничить действия, выполняемые пользователем, и процессы, автоматически реализуемые системой. Разграничение функций позволяет исключить дублирование операций, снизить вероятность ошибок, что способствует более гармоничному взаимодействию человека и системы, обеспечивая согласованность их действий и повышение общей производительности работы. Пользователю предоставляются более простые и понятные задачи: ввод данных, принятие

решений, публикация материалов, или же функции, которые требует человеческого участия, чтобы принять во внимание все факторы, например, модерация материалов. А система берёт на себя автоматическую обработку, хранение, проверку и отображение информации, достигается логичная и упорядоченная работа платформы. Распределение функций между человеком и компьютером представлено в таблице 2.1 и указано, какая из сторон – человек или система – отвечает за их выполнение.

Таблица 2.1 – Распределение функций в системе между человеком и компьютером.

Функция	Человек	Компьютер
Регистрация пользователя.	Вводит имя, email и пароль, подтверждает регистрацию.	Сохраняет данные, проверяет уникальность email и создаёт учетную запись.
Авторизация пользователя.	Вводит email и пароль для входа.	Проверяет корректность данных и предоставляет доступ к аккаунту.
Сброс пароля.	Запрашивает восстановление доступа через форму.	Отправляет письмо с ссылкой для восстановления и обновляет пароль.
Просмотр личного кабинета.	Открывает раздел со своими материалами и статистикой.	Отображает данные профиля, статусы модерации и показатели активности.
Редактирование профиля.	Изменяет личные данные, фото и контакты.	Сохраняет изменения и обновляет профиль в базе данных.
Загрузка материала.	Выбирает файлы, вводит описание и метаданные.	Загружает данные на сервер и сохраняет их в базе.
Редактирование материала.	Изменяет текст, файл или метаданные.	Обновляет материал и сохраняет новую версию.

Продолжение таблицы 2.1.

Удаление материала.	Выбирает материал для удаления.	Удаляет данные из архива и освобождает место в хранилище.
Публикация материала.	Отправляет материал на публикацию после проверки.	Делает материал доступным всем пользователям после одобрения.
Модерация материалов.	Проверяет контент на соответствие правилам.	Предоставляет интерфейс для проверки и позволяет удалять, отправлять на доработку или публиковать материал
Присвоение статуса материала.	-	Присвоение материалу статуса в зависимости от результатов модерации
Поиск по содержимому	Вводит поисковый запрос.	Анализирует запрос, корректирует опечатки и выводит результаты.
Фасетный поиск.	Выбирает фильтры и категории для уточнения поиска.	Обрабатывает фильтры и отображает отфильтрованные результаты.
Просмотр материала.	Открывает страницу с публикацией.	Отображает контент, метаданные и статистику просмотров.
Оценка материала.	Ставит «лайк».	Фиксирует оценку и обновляет рейтинг материала.
Отслеживание статуса модерации.	Просматривает текущий статус своих публикаций.	Обновляет статусы и отображает уведомления об изменениях.
Управление ролями пользователей.	Назначает роли и права доступа.	Применяет изменения и контролирует доступ к функциям.
Хранение и резервирование данных.	-	Выполняет автоматическое сохранение и резервное копирование данных.
Создание категорий и тегов.	Добавляет и редактирует категории.	Сохраняет структуру категорий и связывает их с материалами.

Продолжение таблицы 2.1.

Уведомления пользователей.	Получает уведомления о действиях и изменениях.	Формирует и отправляет уведомления по заданным событиям.
Экспорт материала.	Выбирает материал для скачивания.	Формирует файл и предоставляет ссылку для загрузки.
Получение уведомлений.	Просматривает уведомления в интерфейсе или на почте.	Обновляет ленту уведомлений и управляет их отображением / отправляет сообщения на почту
Подписка на материалы.	Оформляет подписку на категории или авторов.	Сохраняет подписку и уведомляет о новых публикациях.
Рекомендации похожих материалов.	-	Анализирует понравившиеся материалы и предлагает публикации со схожей тематикой.

Таким образом, распределение функций между пользователями и системой играет ключевую роль в построении надёжной и удобной платформы. Пользователи взаимодействуют с интерфейсом, вводят данные, создают и оценивают контент, принимают решения в процессе публикации и модерации. В свою очередь система обрабатывает информацию, хранит её, автоматизирует рутинные действия, обеспечивает безопасность, предоставляет аналитические инструменты и помогает ориентироваться в большом массиве данных. Такое соотношение ролей не только улучшает качество обслуживания и точность выполнения операций, но и способствует расширяемости и развитию платформы: система поддерживает масштабирование и интеллектуальную обработку информации, а пользователи – наполняют её содержанием и формируют ценность сервиса.

Для обеспечения корректного функционирования системы и эффективного взаимодействия ее участников необходимо определить основные роли пользователей [18]. Каждая роль отражает уровень ответственности и доступных действий, что позволяет структурировать права доступа и делегировать задачи между участниками. Такое распределение помогает поддерживать порядок, безопасность и качество публикуемого контента.

На основании анализа, проведенного в разделе выше, в системе выделены три основные роли:

– Пользователь: участник приложения, который может всесторонне взаимодействовать с контентом (добавлять, редактировать, удалять, совершать поиск).

– Модератор: основная задача модератора освободить администратора от проверки контента от пользователей перед его публикацией.

– Администратор: управляет системой, обеспечивает ее функционирование, а также разбирается с происшествиями.

Таким образом, четкое определение ролей пользователей способствует повышению управляемости системы и минимизации перегрузки отдельных участников. Разделение обязанностей между пользователем, модератором и администратором обеспечивает плавную обработку контента и поддержание порядка в системе. Ролевая модель служит фундаментом для дальнейшего развития функциональности и расширения возможностей приложения.

Работа пользователя включает множество разнообразных сценариев – от простых: просмотра и поиска до более сложных и комплексных: сложной загрузки, редактирования и управления подписками – поэтому для полного охвата всех требований системы желательно выделить как можно больше отдельных алгоритмов. Чем детальнее разбивается поведение на отдельные последовательности действий, тем точнее можно описать обращения к системе, предвидеть ошибки и узкие места. Набор детальных алгоритмов позволит проработать все варианты взаимодействия разных категорий пользователей, ускорить тестирование и облегчить дальнейшую автоматизацию процессов. В результате система станет более надёжной, удобной и адаптируемой к реальным потребностям пользователей.

Для всех пользователей доступна функция поиска и просмотра материалов в системе. Любой посетитель может искать публикации по содержимому архива, просматривать документы, фотографии и другие материалы, а также ознакомиться с основной информацией о них.

После прохождения регистрации и авторизации пользователь получает доступ к личному кабинету, где становится возможным полноценное взаимодействие с контентом. Через личный кабинет пользователь может добавлять, редактировать и удалять собственные материалы, связываться с модераторами для уточнения статуса публикаций, а также просматривать понравившиеся категории и материалы. Кроме того, пользователю предоставляется возможность настраивать параметры своего аккаунта, управлять уведомлениями и подписками.

Система поиска дополнительно предоставляет раздел с персонализированными рекомендациями, где отображаются материалы и публикации, похожие по тематике или интересам пользователя, что позволяет более глубоко изучать контент архива и повышает удобство работы с платформой. В таблицах 2.2 – 2.5 приведены шаги алгоритмов работы пользователя в разных режимах.

Таблица 2.2 – Алгоритм работы человека в подсистеме «Пользователь–ПК–среда» в режиме загрузки материала

№	Содержание операции	Обращение к СОИ	Обращение к ОУ
1	Включение ПК	Индикатор на системном блоке	Кнопка включения на системном блоке
2	Включение дисплея	Индикатор включения дисплея	Кнопка включения на дисплее
3	Открытие сети интернет	Ярлык браузера на дисплее	Щелчок мышью
4	Переход на сайт платформы	Ссылка в поисковой строке браузера	Клавиатура, мышь
5	Переход на страницу авторизации	Кнопка на главной странице сайта	Щелчок мышью
6	Ввод логина	Поле ввода логина на странице авторизации	Клавиатура ПК, манипуляции мышью
7	Ввод пароля	Поле ввода пароля на странице авторизации	Клавиатура ПК, манипуляции мышью
8	Авторизация	Кнопка на странице авторизации	Щелчок мышью
9	Переход в личный кабинет	Кнопка в шапке сайта	Щелчок мышью
10	Переход в раздел «Мои материалы»	Кнопка в личном кабинете	Щелчок мышью
11	Нажатие на кнопку «Загрузить материал»	Кнопка на странице	Щелчок мышью
12	Ввод текстовой информации материала	Поле ввода текстовой информации	Клавиатура
13	Загрузка фото-материала	Поле ввода фото-материала	Манипуляции мышью
14	Загрузка файлов	Поле ввода файлов-материала	Манипуляции мышью
15	Заполнение метаинформации материала	Поля выбора метаинформации	Манипуляции мышью
16	Нажатие кнопки «Отправить на модерацию»	Кнопка публикации	Щелчок мышью
17	Закрытие сайта	Значок на экране дисплея	Щелчок мышью

Таблица 2.3 – Алгоритм работы человека в подсистеме «Пользователь–ПК–среда» в режиме поиска содержимого

№	Содержание операции	Обращение к СОИ	Обращение к ОУ
1	Включение ПК	Индикатор на системном блоке	Кнопка включения на системном блоке
2	Включение дисплея	Индикатор включения дисплея	Кнопка включения на дисплее
3	Открытие сети интернет	Ярлык браузера на дисплее	Щелчок мышью
4	Переход на сайт платформы	Ссылка в поисковой строке браузера	Клавиатура, мышь
5	Ввод поискового запроса	Поле ввода поискового запроса	Клавиатура, мышь
6	Нажатие кнопки «Найти»	Кнопка поиска	Щелчок мышью
7	Просмотр материала	Форма отображения результатов поиска	Скролл мышью
8	Закрытие сайта	Значок на экране дисплея	Щелчок мышью

В последующих алгоритмах стартовой точкой будет считаться момент, когда пользователь уже открыл сайт и находится на соответствующей странице интерфейса. Таким образом, однотипные шаги по включению устройства и переходу на платформу, а также закрытие сайта опускаются.

Таблица 2.4 – Алгоритм работы человека в подсистеме «Модератор–ПК–среда» в режиме модерации материала

№	Содержание операции	Обращение к СОИ	Обращение к ОУ
1	Ввод поискового запроса	Поле ввода поискового запроса	Клавиатура, мышь
2	Нажатие кнопки «Найти»	Кнопка поиска	Щелчок мышью
3	Выбор материала для редактирования	Карточка материала	Щелчок мышью
4	Просмотр материала	Страница отображения отдельного материала	Скролл мышью
5	Оставления замечаний и предложений по улучшению материала	Чат с пользователем	Клавиатура, мышь

Продолжение таблицы 2.4.

6	Вернуться к остальным материалам	Кнопка «Назад»	Щелчок мышью
7	Выбор следующего материала для редактирования	Карточка материала	Щелчок мышью
8	Просмотр материала	Страница отображения отдельного материала	Скролл мышью
9	Одобрение и публикация материала	Кнопка «Опубликовать»	Щелчок мышью

Таблица 2.5 – Алгоритм работы человека в подсистеме «Пользователь–ПК–среда» в режиме взаимодействия с модератором

№	Содержание операции	Обращение к СОИ	Обращение к ОУ
1	Переход на страницу авторизации	Кнопка на главной странице сайта	Щелчок мышью
2	Ввод логина	Поле ввода логина на странице авторизации	Клавиатура, мышь
3	Ввод пароля	Поле ввода пароля на странице авторизации	Клавиатура, мышь
4	Авторизация	Кнопка на странице авторизации	Щелчок мышью
5	Переход в личный кабинет	Кнопка в шапке сайта	Щелчок мышью
6	Переход в раздел «Мои материалы»	Кнопка в личном кабинете	Щелчок мышью
7	Выбор неопубликованного материала	Карточка материала	Щелчок мышью
8	Переход к чату с модератором	Кнопка «Чат»	Щелчок мышью
9	Просмотр сообщений модеатора	Модальное окно чата	Скролл мышью
10	Отправить сообщение модератору	Поле ввода сообщения	Клавиатура

Представленные алгоритмы демонстрируют последовательность взаимодействия пользователей и модераторов с системой в различных режимах. Анализ этих последовательностей дает возможность выявить этапы, требующие оптимизации интерфейса, автоматизации или улучшения логики системы. Такие описания служат основой для проектирования удобных пользовательских сценариев, разработки функциональных требований и тестирования. В результате можно повысить эффективность работы платформы, уменьшить нагрузку на участников и улучшить качество взаимодействия между пользователями и модераторами.

2.2 Разработка эргономических требований и сценария информационного взаимодействия

Для проекта «Открытый электронный архив города Витебска» особое внимание уделяется свойствам управляемости и осваиваемости, поскольку система предоставляет большой набор функций, некоторые из которых требуют особой внимательности и осознанности, а также система является непривычной для пользователей [19].

– Управляемость обеспечивает чувство контроля над системой, что снижает уровень неудовлетворенности системой и количество ошибок при выполнении более сложных задач, как добавление материала, взаимодействие с модератором или расширенный поиск.

– Освояемость важна для быстрого старта новых пользователей и осваивания ими всех функций системы. Также хорошая освояемость должна уменьшить работу администратора и модераторов по работе с обычными пользователями.

Эргономические требования, применяемые к системе сведены в таблицу 2.6.

Таблица 2.6 – Эргономические требования.

Группа ЭТ	Номенклатура ЭТ
Эргономическое свойство «Управляемость»	
Психологические	– однотипность реакции системы – акцентирование основных управляющих элементов – необходимость подтверждения необратимых действий – поддержка единого стиля во всех частях системы – предсказуемость результатов действий пользователя
Психофизические	– достаточный размер текста и элементов управления – оптимальный контраст между фоном и символами – расположение кнопок и элементов с учётом привычек и общепринятых решений

Продолжение таблицы 2.6.

Эргономическое свойство «Освояемость»	
Психологические	– наличие встроенных подсказок и инструкций по работе с системой – использование понятных пользователю терминов и пиктограмм – примеры заполнения форм – простая структура интерфейса, которая не требует запоминания – возможность отмены последнего действия без потери данных и прощение ошибок – отсутствие скрытых или неочевидных функци
Психофизические	– использование нейтральной цветовой схемы – минимизация зрительных переходов при работе – поддержка клавиатурной навигации и сочетаний клавиш

Однотипность реакции системы. Все элементы интерфейса должны реагировать одинаковым образом: например, кнопки – визуально выделяться при наведении, формы – выдавать сообщения об ошибке в одном стиле. Это снижает когнитивную нагрузку и уменьшает количество ошибок, так как пользователь переносит уже сформированное ожидание на аналогичные элементы.

Акцентирование основных управляющих элементов. Ключевые кнопки (например, «Сохранить», «Добавить материал») должны визуально выделяться – более контрастным цветом, увеличенным размером или размещением в зоне основного внимания. Это ускоряет поиск нужных действий и снижает вероятность ошибочного выбора.

Необходимость подтверждения необратимых действий. Функции удаления данных или отмены процесса должны сопровождаться запросом подтверждения. Это предотвращает необратимые ошибки и повышает чувство контроля пользователя над системой.

Поддержка единого стиля во всех частях системы. Единая типографика, расположение меню и элементы управления создают узнаваемость интерфейса. Пользователь, освоив логику в одном модуле, автоматически переносит знание на другие.

Предсказуемость результатов действий пользователя. Любое действие должно иметь очевидный результат – например, нажатие кнопки «Поиск» приводит к отображению результатов, а «Сохранить» – к обновлению данных. Это снижает уровень психологического напряжения и повышает доверие к системе.

Достаточный размер текста и элементов управления. Согласно принципам эргономики угловой размер символов, для комфортного чтения должен быть не менее 30–40′. Это соответствует минимальной высоте букв около 3–4 мм при стандартной дистанции просмотра, что эквивалентно

примерно 12–14 пунктам цифрового шрифта. Такой размер снижает утомляемость зрения и обеспечивает уверенное распознавание текста большинством пользователей. Кроме того, ISO 9241–9 рекомендует минимальные размеры интерактивных элементов на экране порядка 16–22 мм для уверенного попадания мышью или пальцем, что соответствует примерно 40×40 пикселям на типичных дисплеях. Эти значения позволяют сократить количество промахов при взаимодействии с элементами интерфейса и повышают точность ввода [20].

Оптимальный контраст между фоном и символами. Рекомендуемый коэффициент контрастности – от 0,6 до 0,95, что соответствует стандартам WCAG для комфортного восприятия текста. Это снижает утомляемость зрения и повышает доступность интерфейса.

Расположение кнопок и элементов с учётом привычек и стандартов. Например, кнопка «Назад» – слева вверху, подтверждающие действия – справа. Привычная логика расположения элементов ускоряет понимание интерфейса и снижает количество неверных действий [21].

Наличие встроенных подсказок и инструкций. Всплывающие подсказки, краткие пояснения рядом с полями и обучающие сообщения помогают пользователю быстро начать работу без обращения к документации.

Использование понятных терминов и пиктограмм. Названия функций должны быть интуитивно понятными (например, «Добавить материал», а не «Инициализировать объект»). Пиктограммы – стандартные (лупа для поиска, корзина для удаления), что уменьшает необходимость обучения.

Простая структура интерфейса, не требующая запоминания. Пользователь должен видеть все основные функции в 1–2 клика, без необходимости помнить последовательность переходов. Это уменьшает когнитивную нагрузку и облегчает освоение.

Возможность отмены последнего действия и прощение ошибок. Кнопка «Отменить» или «Вернуть» снижает страх ошибок и способствует экспериментированию пользователя с системой.

Отсутствие скрытых или неочевидных функций. Все важные функции должны быть доступны в видимой области интерфейса – это ускоряет освоение и снижает фрустрацию.

Использование нейтральной цветовой схемы. Рекомендуется избегать чрезмерно насыщенных или агрессивных цветовых решений и применять мягкие, умеренные тона, поскольку они снижают зрительное утомление при длительной работе. Нейтральная палитра также способствует лучшему восприятию контента и формирует комфортную визуальную среду.

Минимизация зрительных переходов. Важные элементы должны находиться в пределах одной области экрана – например, заголовок, форма и кнопка подтверждения расположены вертикально. Это сокращает движения глаз и повышает скорость выполнения задачи.

Поддержка клавиатурной навигации. Возможность работать с системой через клавиши Tab, Enter, Ctrl+S ускоряет освоение у опытных пользователей.

Таким образом, уделение особого внимания свойствам **управляемости** и **осваиваемости** позволяет создать веб-приложение, которое будет доступным для широкой аудитории пользователей. Продуманная структура интерфейса, логическая предсказуемость действий и наличие подсказок обеспечивают комфортное взаимодействие с системой, снижают вероятность ошибок и ускоряют освоение функций. В результате пользователи смогут эффективно работать с контентом, а нагрузка на администраторов и модераторов снижается, что повышает общую эффективность платформы.

Для информационной системы «Открытый электронный архив города Витебска» были разработаны макеты интерфейса, отражающие основные пользовательские сценарии работы с архивом. Они охватывают процесс навигации по коллекциям, выполнения поиска, подачи материалов на модерацию и взаимодействия с администрацией, обеспечивая целостное представление о логике системы и особенностях пользовательского опыта. Эти сценарии служат основой для последующей реализации интерфейса и позволяют оценить удобство системы на этапе проектирования.

Поскольку именно пользователи являются самой многочисленной группой из выделенных в разделе 4.1, создание макетов интерфейса в первую очередь ориентировалось на удовлетворение их потребностей. При разработке учитывались простота навигации, интуитивность действий и минимизация лишних шагов, чтобы обеспечить комфортное использование системы даже без специальных знаний.

При запуске приложения пользователь попадает на экран авторизации, где ему предлагается ввести адрес электронной почты и пароль для доступа к системе, приведен на рисунке 2.1. Интерфейс выполнен в минималистичном стиле, ключевые элементы расположены по центру экрана, а визуальные акценты помогают быстро сориентироваться. В верхней части предусмотрена кнопка перехода к регистрации, что облегчает вход новым пользователям и формирует однотипную навигацию уже на стартовом этапе взаимодействия.

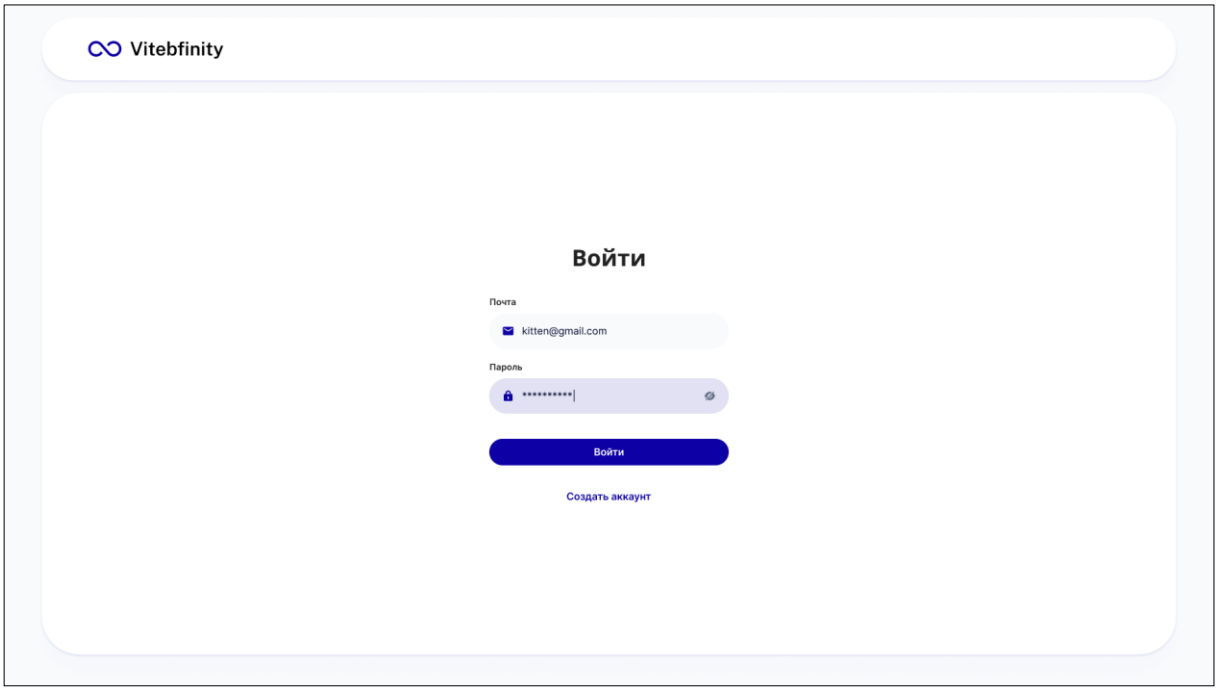

Рисунок 2.1 – Макет экрана «Вход»

Если пользователь не имеет учетной записи, он может воспользоваться ссылкой для создания аккаунта и перейти к форме регистрации, приведена на рисунке 2.2. В данном разделе предусмотрено заполнение обязательных полей – имени, адреса электронной почты и пароля с подтверждением, что позволяет предотвратить ошибочный ввод и обеспечивает базовую защиту данных. Кнопка завершения регистрации находится под формой, визуально отделена и акцентирована, что делает процесс понятным и структурированным.

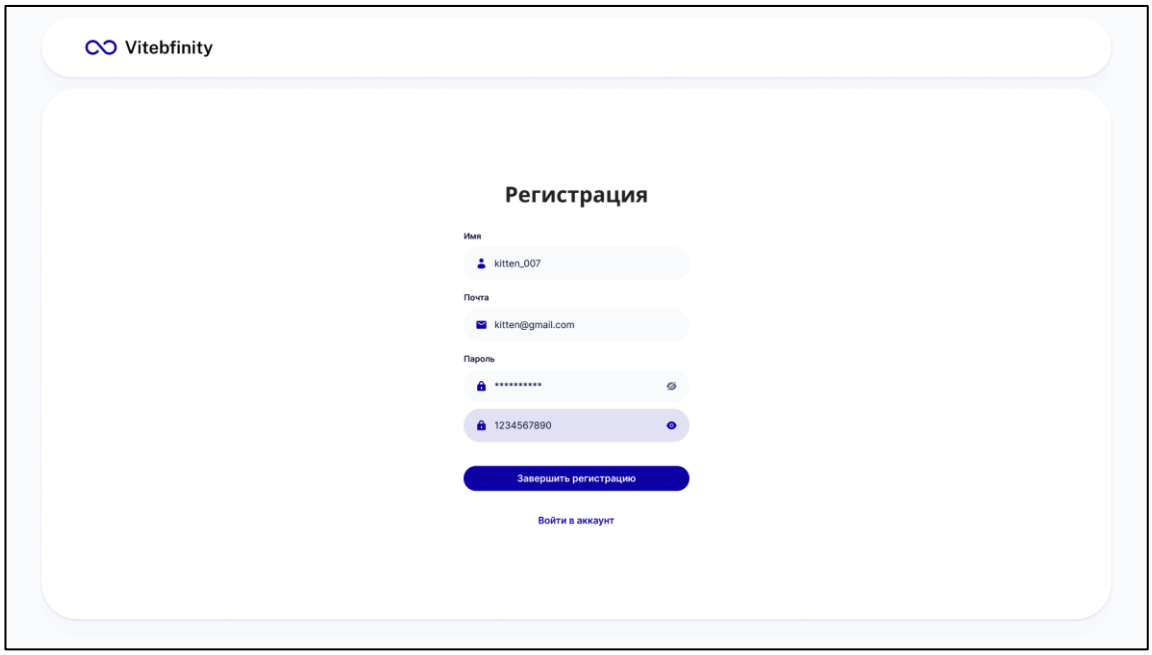

Рисунок 2.2 – Макет экрана «Регистрация»

После успешной авторизации пользователь попадает на главную страницу системы, где представлена центральная поисковая строка, позволяющая начать взаимодействие с архивом через ввод интересующих запросов. Данная страница приведена на рисунке 2.3. Интерфейс организован таким образом, чтобы основной акцент был направлен на поиск – поле ввода размещено по центру, что побуждает пользователя сразу начать исследование содержимого. Справа располагается панель фильтрации, включающая параметры дат, типа документа, геолокации и достоверности материалов, что помогает сузить область поиска и получать более релевантные результаты. Также предусмотрено боковое меню для перехода в профиль, на главную страницу или к функции добавления материалов, обеспечивая быстрый доступ к ключевым разделам системы.

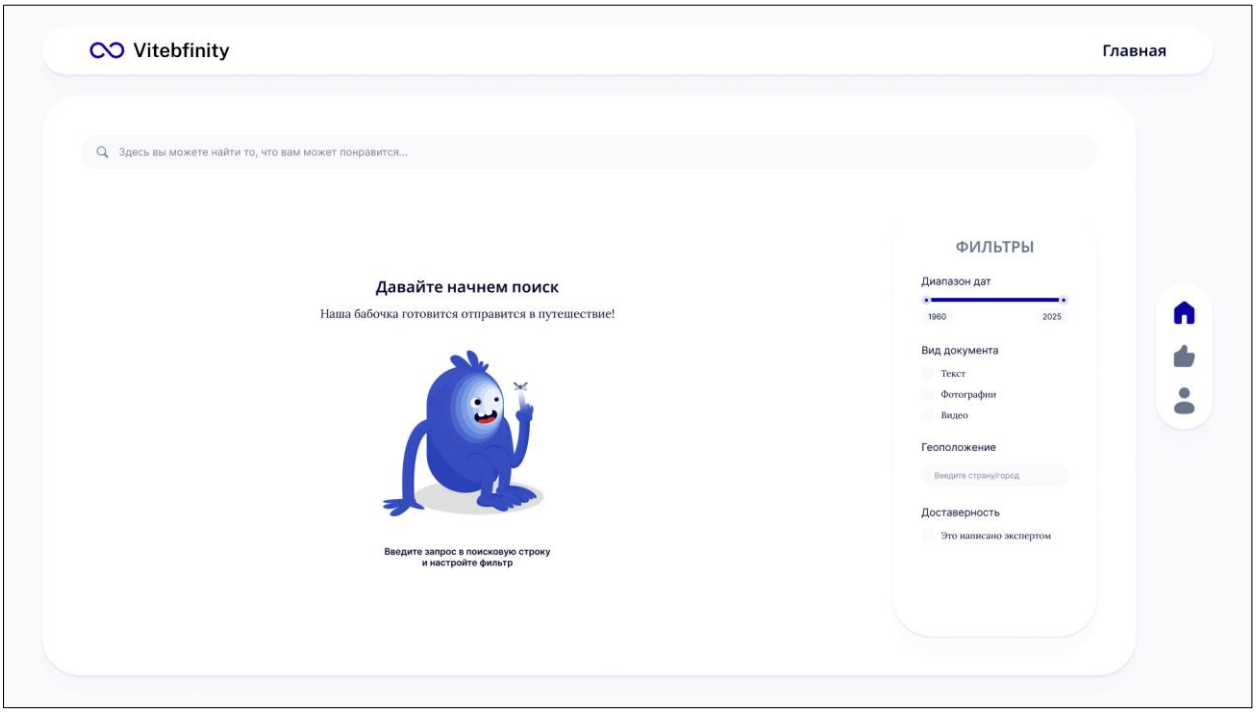

Рисунок 2.3 – Макет экрана «Главная страница»

После ввода поискового запроса на экране отображаются результаты в виде карточек найденных материалов, приведены на рисунке 2.4. В каждой карточке присутствует изображение, краткое описание, указание даты добавления и источника, что позволяет пользователю быстро оценить полезность записи и перейти к ее подробному просмотру. Отфильтрованная выдача визуально структурирована в сетку, что повышает читаемость и облегчает сравнение материалов. Панель фильтров и боковое меню сохраняются на экране, обеспечивая возможность оперативного уточнения запроса или переключения к другим функциям. Такое решение обеспечивает пользователю плавный, наглядный процесс поиска информации – от

начального ввода запроса до анализа найденного контента, формируя логичный и удобный сценарий работы с системой.

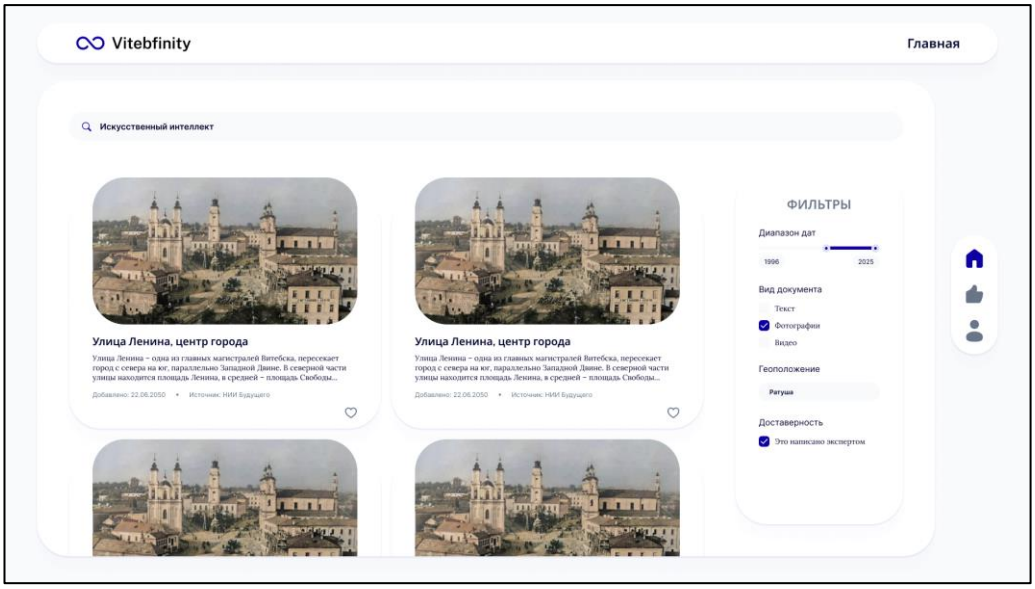

Рисунок 2.4 – Макет экрана «Главная страница» с результатами поиска

При клике на карточку материала открывается отдельная страница с полной информацией о нём, приведена на рисунке 2.5. В верхней части расположено изображение материала или его превью, под ним – заголовок, дата добавления и указание источника. Ниже размещён основной текст документа, структурированный по абзацам с возможностью прокрутки. Справа от контента находится боковая панель с метаданными: тип документа, геолокация, теги, уровень достоверности и дата создания оригинала. В нижней части экрана предусмотрены кнопки действий: «Скачать», «Добавить в избранное», «Поделиться», а также ссылка «Вернуться к результатам поиска». Навигация внутри страницы поддерживается боковым меню, которое остаётся доступным для перехода в профиль, на главную или к добавлению материалов.

Рисунок 2.5 – Макет экрана «Страница материала»

На странице рекомендаций пользователю представлена подборка материалов, подобранных системой на основе его ранее просмотренных или понравившихся записей. В верхней части экрана размещён заголовок раздела «Здесь вы можете найти то, что вам может понравиться...». Основное содержимое страницы организовано в виде вертикального списка карточек рекомендаций. Страница рекомендаций приведена на рисунке 2.6.

В правой части экрана находится отдельный блок «Факт дня», выделенный жирным шрифтом для заголовка и содержащий интересную или познавательную информацию, не обязательно связанную с предыдущими действиями пользователя. Сохраняется боковое меню для навигации по основным разделам приложения: главная, личный кабинет, добавление материала. Данный экран позволяет пользователю пассивно знакомиться с новым контентом, релевантным его интересам, без необходимости самостоятельного поиска.

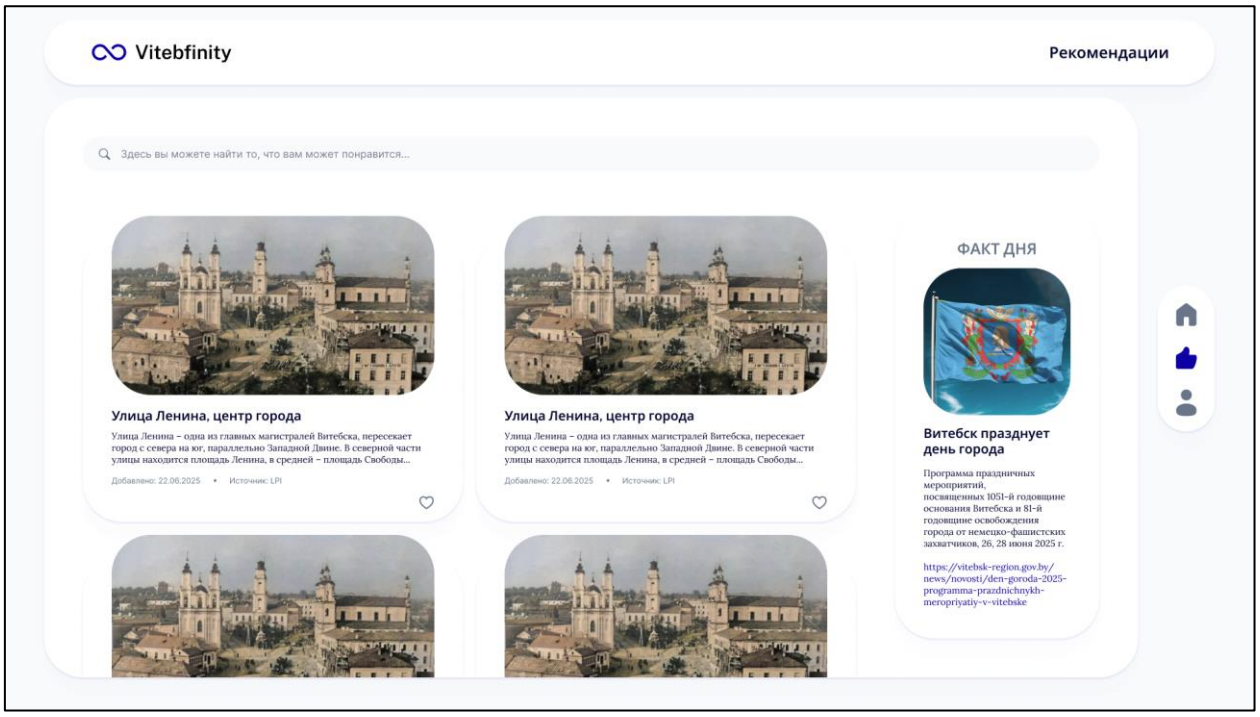

Рисунок 2.6 – Макет экрана «Страница рекомендаций»

Перейдя через правое боковое меню, пользователь попадает в раздел личного кабинета, отображающий основную информацию о его, приведена на рисунке 2.7. Центральную часть экрана занимает форма с данными пользователя: логин, адрес электронной почты и скрытый пароль. Под формой расположена кнопка «Изменить фото» для обновления аватара.

В левой части экрана расположена дополнительная вертикальная панель навигации внутри личного кабинета с разделами: «Информация обо мне», «Мои публикации», «Понравившееся» и «Выйти». Текущий раздел («Информация обо мне») визуально выделен. Правый боковой блок основного меню приложения (для перехода на главную, в поиск и т.д.) остаётся доступным. Данный экран позволяет пользователю просматривать и редактировать свои учетные данные, а также переключаться между другими функциями аккаунта.

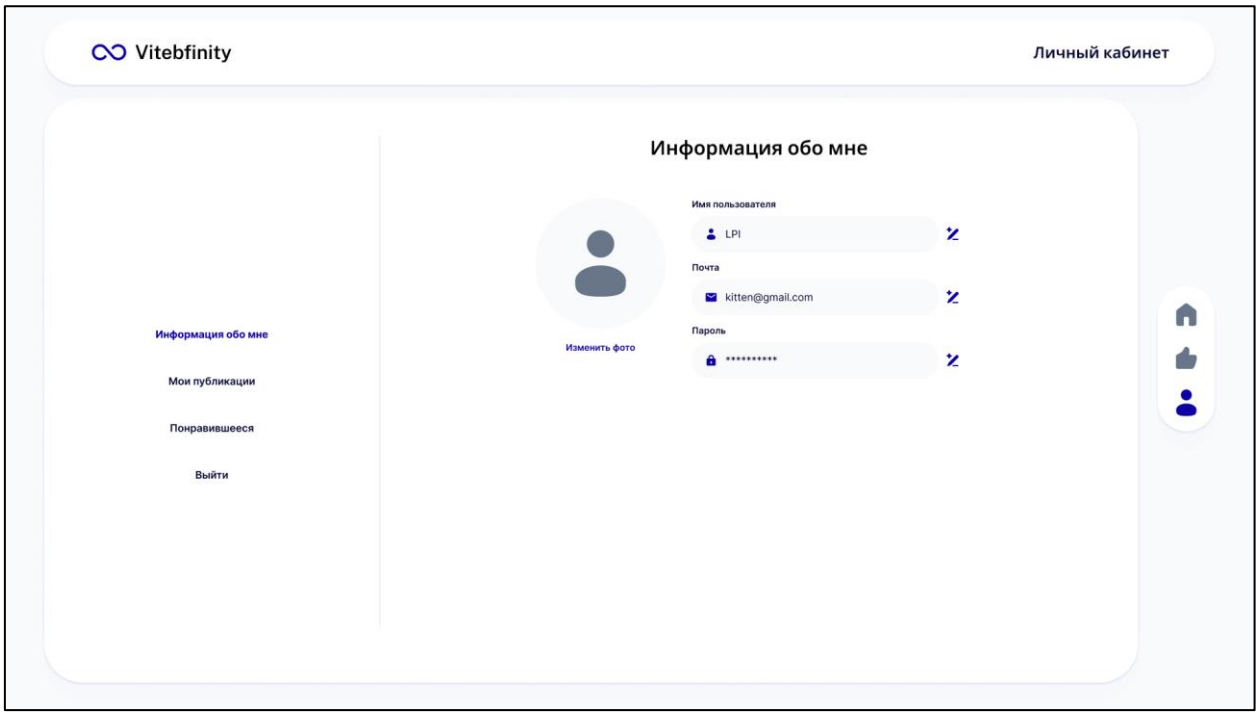

Рисунок 2.7 – Макет экрана «Личный кабинет. Информация обо мне»

Персональные данные расположены в колонку по полям. Каждый тип персональных данных имеет возможность редактирования по нажатию соответствующей иконки, которая расположена справа от поля.

По нажатию на иконку редактирования пароля открывается модальное окно, в котором находятся поля ввода пароля и его подтверждения, приведена на рисунке 2.8. Они расположены вертикально, каждое сопровождается соответствующей иконкой слева, позволяющей посмотреть введенный текст. Кнопка «Сохранить» находится внизу модального окна и выделена тёмно-синим цветом. В правом верхнем углу модального окна – крестик для закрытия без сохранения изменений.

После успешного нажатия «Сохранить» модальное окно закрывается, а обновлённые данные сразу отображаются в разделе «Информация обо мне» личного кабинета.

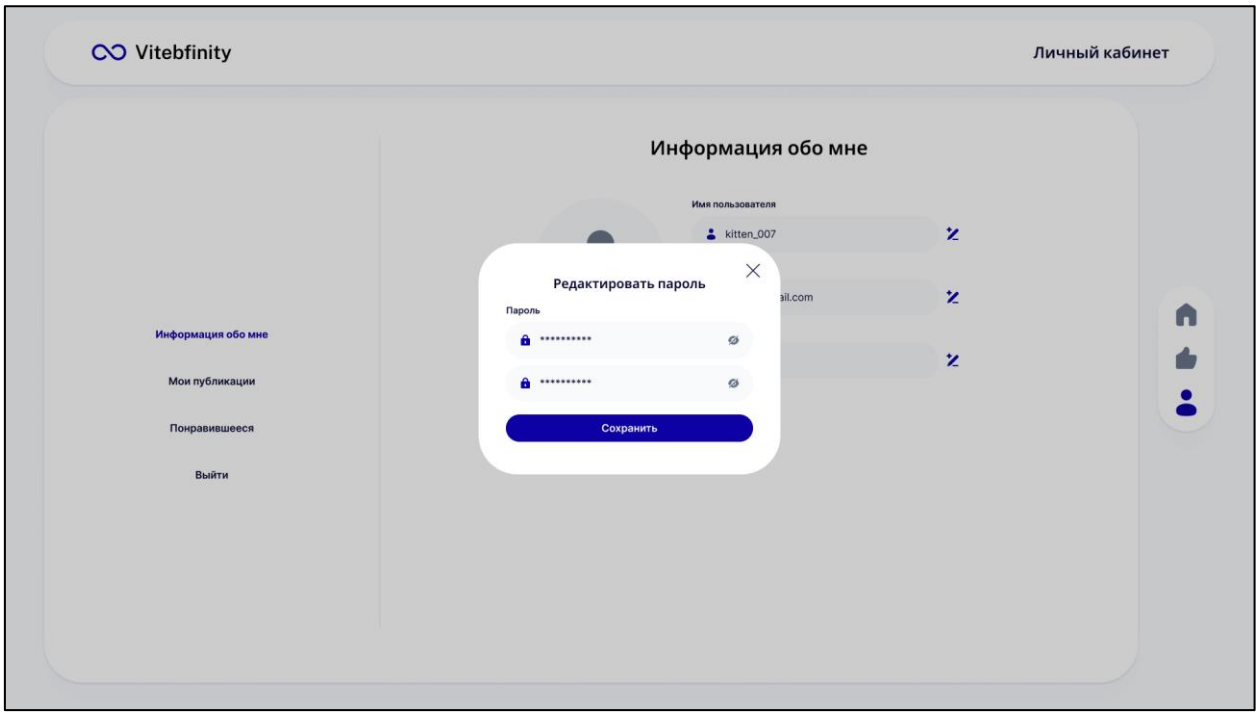

Рисунок 2.8 – Модальное окно «Редактировать профиль» в личном кабинете

Чтобы попасть на следующий экран, пользователь в личном кабинете выбирает в левой вертикальной панели пункт «Мои публикации» (становится подсвеченным синим цветом). Страница «Мои публикации» приведена на рисунке 2.9.

На экране отображаются все материалы, добавленные текущим пользователем, в виде горизонтальных карточек, расположенных в две колонки. Каждая карточка содержит в дополнение: в правом нижнем углу карточки – синюю иконку карандаша для перехода к редактированию этого материала, а в левом нижнем углу иконку, соответствующую статусу материала (глобус – материал опубликован и доступен пользователям, лупа – материал изучается модератором).

Если пользователь ещё ничего не публиковал, вместо карточек отображается пустое состояние с соответствующим текстом и кнопкой добавления.

В правой части экрана остаётся основное боковое меню приложения с активной кнопкой «+» для создания новой записи. Нажатие на эту кнопку открывает форму добавления нового материала.

Рисунок 2.9 – Макет экрана «Личный кабинет. Мои публикаии»

Форма создания и редактирования материалов одинаковая и чтобы открыть ее, пользователь либо нажимает синюю иконку карандаша в правом нижнем углу любой карточки на экране «Мои публикации» (для редактирования уже существующего материала), либо нажимает большую синюю кнопку «+» в правом боковом меню приложения (для создания новой публикации). При создании заголовок модального окна автоматически меняется на «Создать публикацию». Пример формы редактирования материала приведен на рисунке 2.10.

Модальное окно появляется по центру экрана со скруглёнными углами и лёгкой полупрозрачной подложкой. Сразу под заголовком размещено крупное превью текущего изображения материала; если изображение ещё не загружено, показывается пустой блок с приглашением загрузить фото. Под превью находится синяя ссылка «Загрузить новое фото», открывающая стандартный выбор изображения с устройства.

Далее следуют поля ввода: строка «Название публикации» (при редактировании уже содержит текущее название), поле «Почта» (отображается только для чтения и показывает email автора), а под ними большое текстовое поле для основного содержимого материала. Текстовое поле поддерживает базовое форматирование – над ним появляется стандартная панель инструментов с кнопками жирного, курсива, списков и вставки ссылок.

В нижней части окна расположена основная тёмно-синяя кнопка «Сохранить». При редактировании уже опубликованного материала рядом с ней появляется дополнительная серая кнопка «Снять с публикации». В правом верхнем углу окна – крестик для закрытия без сохранения изменений.

После нажатия «Сохранить» модальное окно закрывается, изменения сразу отражаются на соответствующей карточке в разделе «Мои публикации», а публикация появляется в списке со статусом «На модерации».

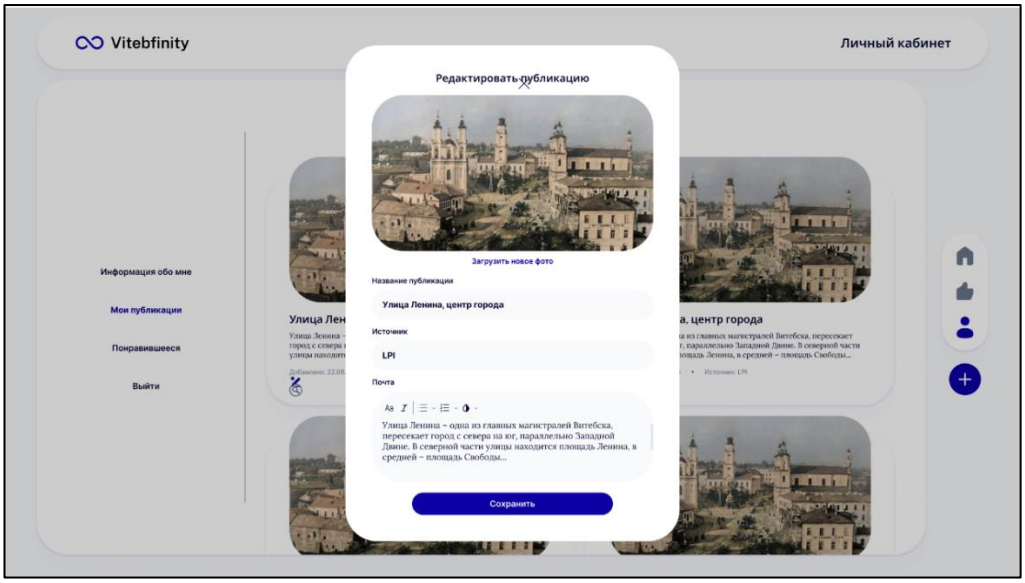

Рисунок 2.10 – Модальное окно «Редактировать материал» в личном кабинете

Если запись в данный момент находится в процессе модерации. То по нажатию на иконку в левом нижнем углу карточки, откроется модальное окно с чатом между пользователем и модератором, посвященному данному материалу. Модальное окно с чатом приведено на рисунке 2.12.

Модальное окно выполнено в едином стиле и соответствует ранее описанным. В середине отображается текущая переписка между модератором и пользователем, а внизу поле ввода сообщений, которое постоянно доступно независимо от перемещения по чату.

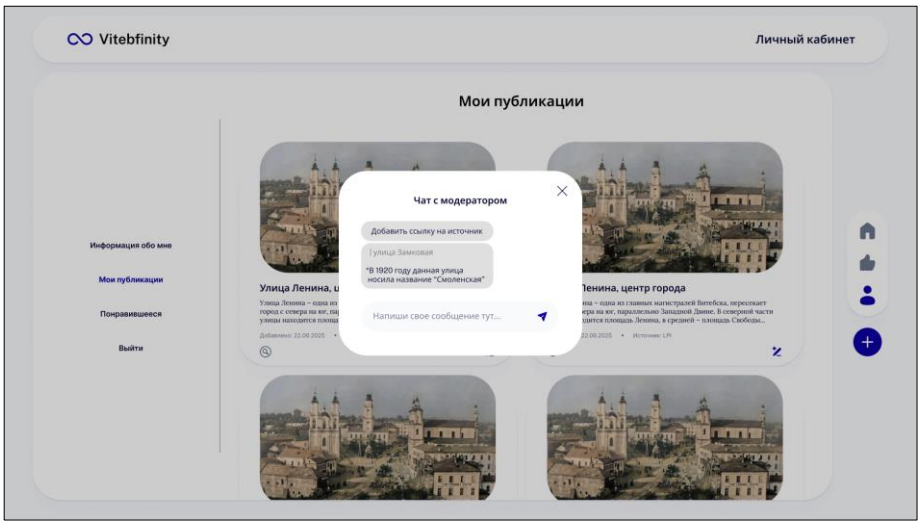

Рисунок 2.11 – Модальное окно «Чат с модератором» в личном кабинете

Последний экран личного кабинета – понравившиеся материалы, приведен на рисунке 2.12. Пользователь в левой вертикальной панели личного кабинета выбирает пункт «Понравившееся» (он подсвечивается синим цветом). Экран отображает все материалы, которые пользователь когда-либо отметил сердечком (добавил в избранное) в любом месте приложения – на главной странице, в результатах поиска, на странице отдельного материала или в рекомендациях.

В правом нижнем углу каждой карточки находится красное сердечко – оно уже заполненное, показывая, что материал находится в избранном. При нажатии на сердечко оно становится пустым, и карточка исчезает из списка «Понравившееся».

Если пользователь ещё ничего не добавил в избранное, экран остаётся пустым и показывает соответствующее сообщение с подсказкой, что материалы можно сохранять нажатием сердечка на любой карточке или странице материала.

Правое боковое меню приложения остаётся на месте, позволяя быстро вернуться на главную страницу, в поиск или добавить новую публикацию. Таким образом, раздел «Понравившееся» служит личной подборкой сохранённых материалов, к которой пользователь может вернуться в любой момент.

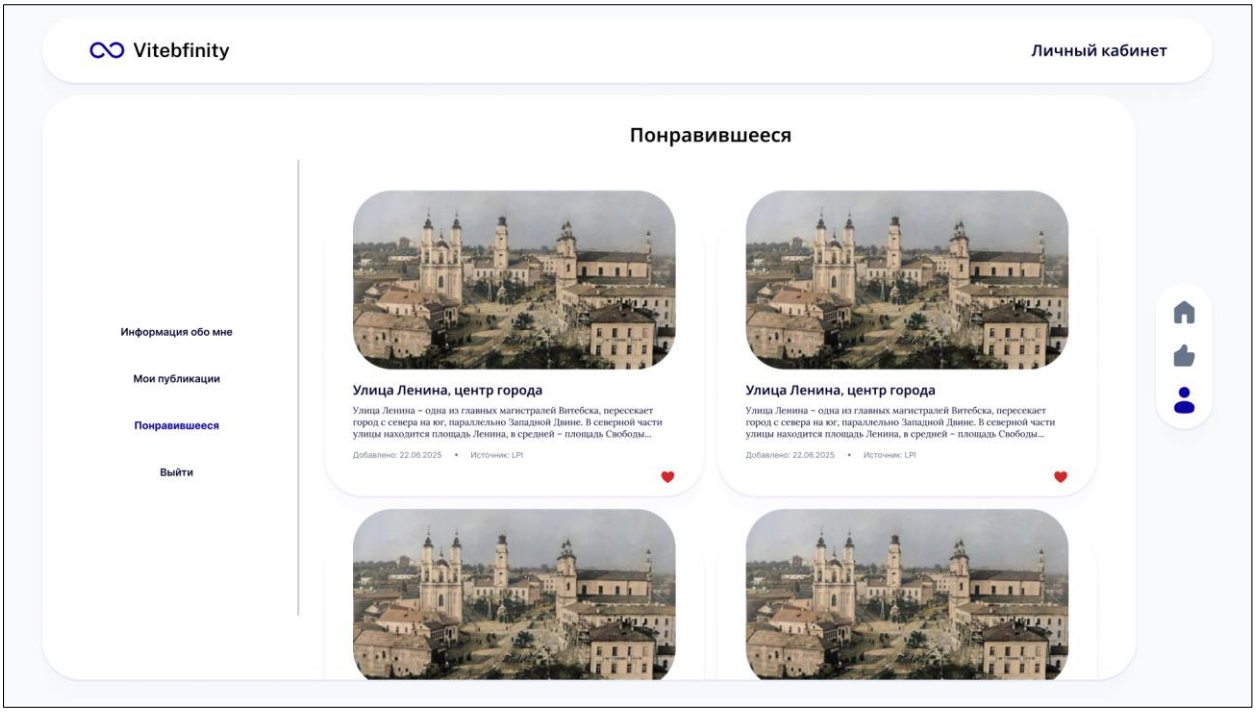

Рисунок 2.12 – Макет экрана «Личный кабинет. Понравившееся»

Иногда пользователь переходит по несуществующей или временно недоступной ссылке, пытается открыть удалённый материал или заходит в раздел, который в данный момент находится на техническом обслуживании,

что может вызвать сбой в работе. В таких случаях нам стоит показать ему что мы знаем проблеме и самое главное, что это не он стал её источником. Для этого можно использовать собственную страницу 404. Пример страницы 404 приведен на рисунке 2.13.

Вся площадь экрана занята тёмно-синим космическо-подводным фоном с мягкими лучами света и пузырьками. По центру крупно выведена надпись «404» и сразу под ней заголовок «Наш путешественник свернул не туда». Справа от текста находится фирменный персонаж приложения – милый аксолотль-исследователь в скафандре, который растерянно разводит лапками и говорит в облачке: «И да: ни одна бабочка в процессе не пострадала!»

В нижней части экрана размещена единственная белая кнопка со скруглёнными углами «Вернуться на главную страницу». Нажатие на неё мгновенно переносит пользователя на главную страницу приложения.

Такой экран полностью заменяет стандартные системные сообщения об ошибках и показывает пользователю, что проблема известна разработчикам, носит временный и не критичный характер, а приложение продолжает работать в штатном режиме. Юмор и дружелюбный персонаж снимают напряжение и страх, что «что-то сломалось навсегда».

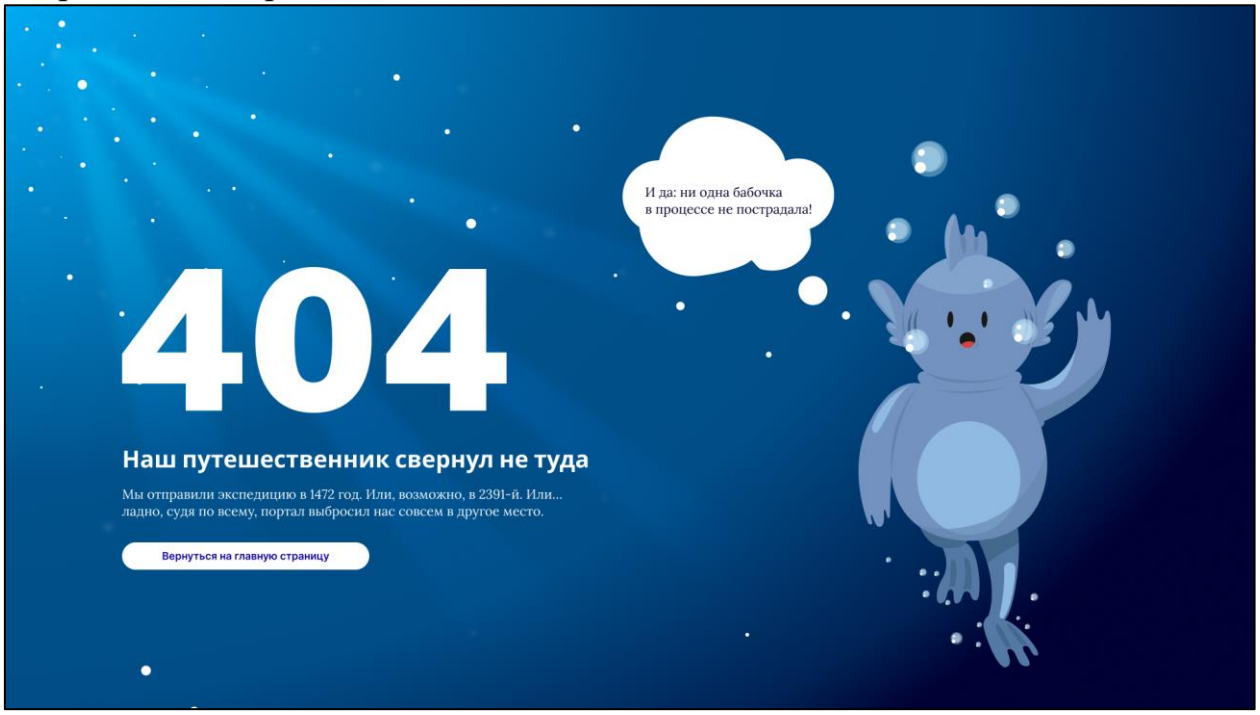

Рисунок 2.13 – Макет экрана «Страница 404»

Разработанные пользовательские сценарии и макеты интерфейса демонстрируют целостную архитектуру взаимодействия пользователя с системой и позволяют оценить удобство работы ещё на этапе проектирования. Анализ экранов показал, что навигация логична, поддерживается повторяющимися паттернами и сохраняется доступной вне зависимости от контекста, что снижает когнитивную нагрузку пользователя. При этом выявлены зоны для доработки – например, расширение визуальной обратной

связи в отдельных местах, оптимизация состояний пустых экранов и дополнение подсказок при создании публикаций. Представленные решения создают фундамент для полноценной реализации интерфейса, пилотного тестирования и дальнейшего улучшения по результатам поведения реальных пользователей.

2.3 Эргономическая оценка проектируемого веб-приложения и выводы

Для проверки того, насколько проект отвечает этим требованиям, применяется эргономический анализ, позволяющий выявить сильные стороны интерфейса и определить области, требующие доработки [22]. Такая оценка обеспечивает объективное измерение качества пользовательского опыта и помогает убедиться, что система соответствует потребностям реальных пользователей, снижает нагрузку на их внимание и способствует эффективному взаимодействию, что отражено в структуре требований и критериях оценки, представленных в таблице 2.7.

Таблица 2.7 – Эргономические требования к проектируемой системе и соответствующие им единичные эргономические показатели

Группа ЭТ	Эргономические требования	Единичные эргономические показатели
Эргономическое свойство «Управляемость»		
Психологические	П-1. однотипность реакции системы	Единообразие поведения интерфейсных элементов (анимация кнопок, стиль уведомлений, формат ошибок).
	П-2. акцентирование основных управляющих элементов.	Выделенность ключевых кнопок контрастом, размером и расположением в зоне визуального внимания.
	П-3. необходимость подтверждения необратимых действий	Наличие модальных окон подтверждения при удалении, публикации, сбросе данных.
	П-4. поддержка единого стиля во всех частях системы	Согласованность типографики, сетки расположения элементов и визуальных паттернов во всех интерфейсах.

Продолжение таблицы 2.7.

	П-5. предсказуемость результатов действий пользователя	Логичность отклика интерфейса – соответствие результата стандартным ожиданиям (клик → переход, сохранение → обновление данных).
Психофизиологические	ПФ-1. достаточный размер текста и элементов управления	Минимальный размер шрифта 30–40′ (12–14 pt), минимальный размер интерактивной зоны 40×40 px.
	ПФ-2. оптимальный контраст между фоном и символами	Соответствие коэффициента контрастности 4.5:1 для текста и 3:1 для крупных элементов.
	ПФ-3. положение кнопок и элементов с учётом привычек и общепринятых решений	Стандартное размещение действий: пункт «Назад» – слева, подтверждение – справа, элементы основной навигации – в фиксированных зонах.
Эргономическое свойство «Освояемость»		
Психологические	П-6. наличие встроенных подсказок и инструкций по работе с системой	Наличие всплывающих подсказок, текстовых инструкций и контекстных помощников.
	П-7. использование понятных пользователю терминов и пиктограмм	Семантическая однозначность подписей, применение стандартных пиктограмм (лупа – поиск, корзина – удаление).
	П-8. примеры заполнения форм	Использование placeholder-примеров внутри полей
	П-9. простая структура интерфейса, которая не требует запоминания	Глубина навигации не более 2–3 уровней, наличие элементов ориентации («хлебных крошек», подсветки активного раздела).

Продолжение таблицы 2.7.

	П-10. возможность отмены последнего действия без потери данных и прощение ошибок	Наличие undo-действий, кнопок «Отменить», отменяемых модальных окон вместо необратимых операций.
	П-11. отсутствие скрытых или неочевидных функций	Доступность всех ключевых функций через видимые элементы – отсутствие функциональности, скрытой за нестандартными жестами.
Психофизические	ПФ-4. использование нейтральной цветовой схемы	Применение мягкой палитры оттенков (белый, серый, спокойные синие), избегание агрессивных цветов.
	ПФ-5. минимизация зрительных переходов при работе	Группировка взаимосвязанных элементов вертикально, отсутствие разброса элементов по противоположным зонам экрана.
	ПФ-6. поддержка клавиатурной навигации и сочетаний клавиш	Наличие tab-фокуса, горячих клавиш (например, Enter = подтвердить, Ctrl+S = сохранить).

Рассмотрим соответствие указанным требованиям в разрабатываемом веб-приложении. Составим таблицу 2.8 для последующего подсчета эргономической оценки.

Таблица 2.8 – Значения единичных и групповых эргономических показателей проектируемой системы

Группа ЭТ	Значения единичных ЭП	Значения групповых ЭП
Эргономическое свойство «Управляемость»		
Психологические	П-1, П-2, П-4, П-5 = 1; П-3 = 0	∑1 = 4, ∑0 = 1 4 /5 = **0.8**
Психофизиологические	ПФ-1, ПФ-2 = 1; ПФ-3 = 0	∑1 = 2, ∑0 = 1 2 / 3 = **0.67**

Продолжение таблицы 2.8.

Эргономическое свойство «Освояемость»		
Психологические	П-8, П-9, П-10, П-11, П-7 = 1; П-6 = 0	∑1 = 5, ∑0 = 1 5/6 = **0.87**
Психофизиологические	ПФ-4, ПФ-5, ПФ-6 = 1	∑1 = 3, ∑0 = 0 3 / 3 = **1**

Оценим эргономические свойства СЧМ в соответствии с аддитивной функцией при помощи таблицы 2.9:

Таблица 2.9 – Значения весовых коэффициентов для оценки эргономических свойства «управляемость» и «освояемость»

Групповой ЭП	Значение весового коэффициента
Эргономическое свойство «Управляемость»	
Психофизиологический	0.4
Психологический	0.6
Эргономическое свойство «Освояемость»	
Психологический	0.7
Психофизиологический	0.3

ЭСв управляемость = (0.6*0.8) + (0.4*0.67) = 0.48 + 0.27 = 0.75

ЭСв освояемость = (0.7*0.87) + (0.3*1) = 0.61 + 0.3 = 0.91

Присвоим весовые коэффициенты эргономическим свойствам «управляемость» и «освояемость» в соответствии с экспертной оценкой их вклада в эргономичность проектируемой системы: управляемости – 0,65, освояемости – 0,35. Вычислим эргономичность системы в целом:

Эргономичность = 0.65 * 0.75 + 0.35 * 0.91 = 0.488 + 0.319 = 0.807

Полученное значение соответствует оценке «отлично».

Полученный показатель эргономичности свидетельствует о том, что проект соответствует ранее разработанным эргономическим требованиям. Интерфейс реализован последовательно, понятен большинству пользователей и обеспечивает уверенное выполнение ключевых.

Тем не менее, при достаточном наличии временных и человеческих ресурсов возможно дальнейшее повышение эргономичности по следующим пунктам, представлены в таблице 2.10. Такие доработки позволят улучшить опыт пользовательского взаимодействия с системой.

Таблица 2.10 – Рекомендации по улучшению эргономичности системы

Невыполненное эргономическое требование	Предложение по улучшению эргономичности
П-3. необходимость подтверждения необратимых действий	Для всех необратимых действий внедрить вспывающие диалоги подтверждения.
ПФ-3. положение кнопок и элементов с учётом привычек и общепринятых решений	При проектировании интерфейса следует в основном придерживаться общепринятых решений и пользовательских привычек, допуская осознанные отклонения от них для формирования уникального и узнаваемого стиля системы.
П-6. наличие встроенных подсказок и инструкций по работе с системой	Реализовать встроенные подсказки для авторизованных пользователей при первом открытии новой страницы

Проведённый эргономический анализ показал, что проектируемая система обладает высоким уровнем эргономичности и в целом соответствует установленным требованиям к управляемости и освояемости. В то же время эргономический уровень не является абсолютным и может быть дополнительно улучшен. Выявлены невыполненные требования и предложены рекомендации, которые указывают на конкретные направления доработки.

3 РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ «ОТКРЫТЫЙ ЭЛЕКТРОННЫЙ АРХИВ ИСТОРИИ Г. ВИТЕБСКА»

Проектирование информационной системы, объединяющей функции архивного хранилища и социальной платформы, требует комплексного подхода к выбору архитектурных решений, технологического стека и организации данных. Особенности предметной области – необходимость работы с неструктурированными историческими материалами, обеспечение краудсорсингового наполнения контента при сохранении контроля достоверности, а также реализация интеллектуального поиска по слабоструктурированным данным – обусловливают применение современных инструментов разработки и специализированных программных компонентов.

Настоящий раздел посвящён детальному описанию программной реализации веб-приложения «Открытый электронный архив города Витебска». В разделе 3.1 представлена архитектура системы с обоснованием выбора микросервисного подхода и применяемых технологий. Раздел 3.2 содержит описание ключевых алгоритмов функционирования приложения, включая жизненный цикл материала и механизмы интеллектуального поиска. В разделе 3.3 приведена структура баз данных и индекса Elasticsearch. Раздел 3.4 завершает главу оценкой результатов разработки и соответствием проекта целевым требованиям.

3.1 Архитектура веб-приложения

В качестве архитектурного решения для реализации системы архива выбрано веб-приложение – клиент-серверное ПО, доступное пользователям через веб-браузер без необходимости установки специализированных утилит. Данный выбор обусловлен рядом ключевых преимуществ, критичных для достижения целей проекта. Во-первых, веб-приложение обеспечивает кроссплатформенную доступность: пользователи могут работать с архивом с любых устройств, что расширяет потенциальную аудиторию системы. Во-вторых, централизованное размещение приложения на сервере упрощает процессы развёртывания обновлений и исправлений – изменения становятся доступны всем пользователям одновременно без необходимости переустановки клиентских компонентов. В-третьих, веб-архитектура обеспечивает естественную интеграцию с современными протоколами передачи данных и API, что критично для реализации краудсорсингового сбора материалов.

Архитектурное решение предполагает чёткое разделение клиентской и серверной частей с применением паттерна «тонкий клиент». В данной модели клиентское приложение, ограничивается функциями отображения интерфейса, обработки пользовательских действий и передачи запросов на сервер, в то время как вся бизнес-логика обрабатывается на серверной стороне. Такой подход минимизирует требования к вычислительным ресурсам

клиентских устройств и позволяет пользоваться унифицированным шаблоном общения между клиентом и сервером.

Для оптимизации взаимодействия между клиентом и серверной архитектурой применён паттерн BFF (Backend for Frontend). Суть паттерна заключается в создании специализированного промежуточного слоя (api gateway) – отдельного сервиса, который принимает все запросы от клиентского приложения и перенаправляет их к внутренним микросервисам. Главной функцией BFF-сервиса является централизованная обработка аутентификации и авторизации. Применение BFF позволяет изолировать клиентское приложение от сложности внутренней микросервисной структуры, обеспечивая простой и стабильный API, при этом сохраняя гибкость и независимость развития отдельных бизнес-модулей.

Архитектура серверной часть системы реализована в виде микросервисов, предполагающему декомпозицию приложения на набор независимо развёртываемых подсистем, каждый из которых отвечает за ограниченный контекст бизнес-логики. Данный выбор обусловлен необходимостью обеспечения отказоустойчивости, масштабируемости и гибкости развития системы.

Ключевые преимущества применённого подхода заключаются в следующем:

5 Логически слабосвязанные механизмы системы разделены по независимым сервисам, что предотвращает взаимное влияние компонентов при изменениях: модификация алгоритма рекомендаций не требует пересборки модуля поиска или модерации.

6 Разделение по сервисам обеспечивает изоляцию ресурсов – вычислительно интенсивные операции OCR-обработки или полнотекстового индексирования не отбирают ресурсы у критичных компонентов аутентификации и авторизации, поскольку каждый сервис функционирует в собственном контейнере с выделенными лимитами CPU и памяти.

7 Отказоустойчивость достигается через изоляцию отказов: ошибка или перегрузка одного сервиса (например, временная недоступность модуля рекомендаций) не распространяется на остальные компоненты системы, пользователи продолжают пользоваться поиском и просмотром материалов.

При декомпозиции системы выделили шесть основных бизнес-модулей, каждый из которых реализует автономную область функциональности:

8 Модуль управления пользователями – аутентификация, авторизация, управление профилями и ролями;

9 Модуль управления материалами – жизненный цикл архивных материалов, метаданные, статусы публикации;

10 Модуль поиска – полнотекстовый поиск, фасетная фильтрация, автокоррекция запросов;

11 Модуль модерации – проверка качества, присвоение статусов, обратная связь с авторами;

12 Модуль социального взаимодействия – лайки, подписки, уведомления, рекомендации;

13 Модуль хранения данных – управление файлами, OCR-обработка, генерация превью.

Каждый из этих модулей реализован отдельным микросервисом. Помимо этого, система включает служебные компоненты, обеспечивающие инфраструктурные функции. Модуль сбора даанных (harvesting) отвечает за сбор и агрегацию данных для модуля поиска и обслуживания индекса Elasticsearch (например удаление устаревших данных). Модуль нотификаций (notificator) централизует отправку уведомлений пользователям, обеспечивая единый интерфейс для остальных сервисов при необходимости уведомить пользователя о событии. Gateway выполняет маршрутизацию внешних запросов и rate limiting.

Организация кодовой базы серверной части структурирована по принципу разделения на core-модули и common-библиотеки, что обеспечивает управление сложностью и переиспользование кода при сохранении независимости бизнес-сервисов. Core-модули представляют собой самостоятельные – микросервисы, содержащие полный контур бизнес-логики конкретной предметной области: собственные контроллеры, сервисы, репозитории, конфигурацию и точку входа. Каждый core-модуль может быть независимо собран, протестирован и развёрнут. Common-библиотеки содержат переиспользуемую инфраструктурную функциональность, не привязанную к конкретной бизнес-области, и подключаются к core-модулям как зависимости. Выделение common-компонентов позволяет стандартизировать сквозную функциональность: обработку исключений и формирование единого формата ошибок, конфигурацию логирования, базовые классы для работы с Kafka, утилиты безопасности, общие DTO для межсервисного взаимодействия, базовые классы тестирования. Данный подход гарантирует консистентность реализации сквозных аспектов (security, observability).

Коммуникация между микросервисами реализована по гибридной схеме с применением двух механизмов в зависимости от характера взаимодействия.

Асинхронное взаимодействие через Apache Kafka применяется для событийно-ориентированной коммуникации, когда инициатору не требуется немедленный ответ от получателя, а критична гарантированная доставка сообщения. Потребители подписываются на релевантные топики и обрабатывают события асинхронно. Данный механизм используется для: инициации OCR-обработки при загрузке файла, индексации материала в Elasticsearch, отправки уведомлений подписчикам о новых публикациях. Преимущества Kafka включают гарантированную доставку (at-least-once), возможность replay сообщений при сбоях, буферизацию при временной недоступности потребителей, естественное разделение потоков по тематическим топикам.

Синхронное взаимодействие через WebClient (реактивный HTTP-клиент Spring) применяется для операций, требующих немедленного ответа в рамках одного пользовательского запроса. Например, для загрузки материала в индекс моуль harvesting запрашивает у модуля управления пользователями

информацию для обогащения информации о записи. Взаимодействие осуществляется внутри изолированной Docker-сети, что обеспечивает низкую латентность и безопасность (трафик не выходит за пределы хоста). Используемые технологии сведены в таблицу 3.1.

Таблица 3.1 – Технологический стек веб-приложения

Компонент/функция	Технология	Версия
Клиентское ПО	HTML, CSS, JavaScript	HTML5, CSS3, ES16
Серверная платформа	Java	21
Сборка сервера	Apache Maven	4.0.0
Фреймворк микросервисов	Spring Boot	4.x
Реляционная база данных	PostgreSQL [23]	18.2
Поисковый движок	Elasticsearch	9.3.2
Брокер сообщений	Apache Kafka	4.20
Кэширование и сессии	Redis	Alpine 3.23
OCR-обработка	Tesseract OCR	4.0.0
Объектное хранилище	MinIO [24]	RELEASE.2025-09-07
Контейнеризация	Docker [25]	24.0.5

Выбор Java 21 в качестве языка серверной разработки обусловлен статусом Long-Term Support (LTS), обеспечивающим стабильность и длительную поддержку, а также производительностью JVM для IO-интенсивных нагрузок, характерных для веб-приложений с интенсивной работой с файлами и сетевыми запросами. Spring Boot 4.x предоставляет автоконфигурацию, встроенные метрики (Micrometer), observability (distributed tracing) и нативную поддержку GraalVM для потенциальной компиляции в нативный код.

Применение WebClient обеспечивает эффективную обработку HTTP-запросов, критичную для BFF-слоя, где требуется выполнение множественных параллельных вызовов к микросервисам без блокировки потоков исполнения. Данный подход позволяет обслуживать большее количество конкурентных соединений при меньшем потреблении памяти по сравнению с традиционной servlet-based архитектурой.

Клиентская часть реализована на HTML5, CSS3 и JavaScript (ES16) – стандартный стек для простой разработки современных веб-приложений, обеспечивающий кроссплатформенную доступность через веб-браузер без необходимости установки специализированного ПО.

PostgreSQL выбран как зрелая реляционная СУБД с поддержкой JSONB для гибких метаданных материалов и расширением PostGIS для потенциальной геопространственной индексации. Flyway обеспечивает контроль версий схемы базы данных, критичный для командной разработки и CI/CD.

Elasticsearch 9.3.2 предоставляет специализированные возможности полнотекстового поиска, превосходящие реляционные СУБД:

морфологический анализ через плагины analysis-morphology, нечёткий поиск с автоматическим определением редакционного расстояния, агрегации для фасетной навигации с динамическими счётчиками. Spring Data Elasticsearch обеспечивает типобезопасное взаимодействие с индексом на уровне репозиториев.

Apache Kafka 4.20 выбран как промышленный стандарт для event-driven архитектуры, обеспечивающий необходимую надёжность доставки событий модерации и уведомлений. Redis (Alpine 3.23) применяется для распределённого хранения сессий пользователей (обеспечение stateless-архитектуры микросервисов), кэширования популярных запросов к модулю поиска и реализации rate limiting на уровне API Gateway.

Tesseract OCR 4.0.0 интегрирован как оптимальное соотношение функциональности и стоимости (открытая лицензия Apache 2.0) при поддержке требуемых языков (русский, белорусский). MinIO (RELEASE.2025-09-07) обеспечивает S3-совместимое объектное хранилище для хранения файлов архивных материалов.

Apache Maven 4.0.0 используется для сборки серверной части приложения, обеспечивая управление зависимостями и стандартизированный процесс сборки. Docker 24.0.5 обеспечивает инфраструктуру контейнеризации для непрерывного развёртывания, автомасштабирования и самовосстановления сервисов при сбоях.

Таким образом, предложенная архитектура представляет собой современное масштабируемое решение, сочетающее микросервисный подход с оптимально технологическим стеком. Применение паттерна BFF обеспечивает чёткое разделение ответственности между клиентом и сервером, а гибридная схема межсервисной коммуникации позволяет эффективно балансировать между требованиями к производительности и надёжности системы. Выбранные технологии обеспечивают полный функционал, необходимый для реализации краудсорсингового архива: от обработки пользовательских сессий и полнотекстового поиска до OCR-распознавания и управления цифровыми объектами, при этом сохраняя возможность независимого развития отдельных компонентов системы.

3.2 Алгоритм работы веб-приложения

Понимание того, как работает веб-приложение критично при дальнейшем тестировании, отладке и развитии платформы, поскольку позволяет выявлять узкие места, проектировать сценарии нагрузочного тестирования и обеспечивать согласованность данных при взаимодействии распределенных компонентов. Приведенные далее алгоритмы работы веб-приложения отражают основы взаимодействия компонентов системы. Рассматриваются ключевые процессы: управление жизненным циклом пользователей и аутентификация через интеграцию с Keycloak, цикл обработки архивных материалов, механизмы интеллектуального поиска и

персонализированных рекомендаций, а также распознавание документов и работа api gateway.

Алгоритм управления Пользователями.

Алгоритм управления пользователями реализует централизованную идентификацию через выделенный микросервис Identity, который интегрируется с Keycloak посредством Java-библиотеки keycloak-admin-client [26]. Данный подход инкапсулирует сложность протоколов OAuth 2.0 и Admin API внутри серверного компонента, обеспечивая простой интерфейс для взаимодействия [27].

Регистрация нового пользователя инициируется через Gateway, который перенаправляет REST-запрос к Identity. Там происходит валидация введенных данных и в случае успеха сервис сохраняет данную запись в PostgreSQД. После сохранения записи в локальную БД при помощи keycloak-admin-client, создается учетная запись в Keycloak: формируется объект UserRepresentation, устанавливаются учетные данные, генерирует уникальный идентификатор пользователя, устанавливаются базовые роли. Идемпотентность операции обеспечивается ограничением уникальности идентификатора пользователя в базе данных при повторной обработке.

Обновление профиля выполняется через тотже скервис identity. Клиент отправляет PUT-запрос с изменяемыми полями, которые заносятся в базу данных. И в случае изменения критичных атрибутов – email или пароля – сервис выполняет вызов Keycloak Admin API через keycloak-admin-client.

Сброс пароля инициируется через Identity по запросу пользователя, указавшего email. Сервис выполняет поиск пользователя в Keycloak по email, генерирует action token через Admin API и инициирует отправку письма со ссылкой на сброс. При переходе пользователя по ссылке происходит прямое взаимодействие с Keycloak для установки нового пароля.

Деактивация аккаунта выполняется по запросу авторизованного пользователя или администратора. Identity устанавливает флаг enabled: false в Keycloak через Admin API, что блокирует возможность аутентификации. Параллельно обновляется статус в PostgreSQL на INACTIVE. Все токены добавляются в blacklist с TTL до истечения срока действия. Материалы пользователя сохраняются в системе, но скрываются из публичного доступа до повторной активации или переноса прав собственности.

Унифицированная схема алгоритма управления пользователями приведена на рисунке 3.1

Рисунок 3.1 – Алгоритм управления пользователем

Алгоритм управления материалами

Алгоритм управления материалами описывает полный жизненный цикл архивной записи от момента создания пользователем до становления доступной для интеллектуального поиска, с акцентом на асинхронную обработку данных через брокер сообщений и коннекторы индексации.

Создание и первичное сохранение инициируется в личном кабинете пользователя. Клиент отправляет POST-запрос к Gateway с метаданными материала: названием, описанием, категориями и прочей информацией. Gateway выполняет переадресацию в сервис Catalog. Сервис создает запись в PostgreSQL со статусом DRAFT, генерирует id материала, устанавливает связь с автором через user_id. Для загрузки файлов Catalog запрашивает у S3-хранилища presigned URL для прямой записи в MinIO, исключая сервер из последующего запроса. По завершении загрузки клиент уведомляет Catalog о переходе статуса в MODERATION и публикует событие в Kafka-топик.

Асинхронная обработка через Kafka обеспечивает слабую связанность компонентов и масштабируемость обработки. Событие содержит material_id, user_id, список file_urls, и прочие метаданные. Сообщение маршрутизируется в партицию на основе material_id для сохранения порядка обработки записей одного материала. Несколько независимых consumer groups подписаны на топик:

– Harvester обогащает материла инофрмацией из Identity и Social и публикует обогащенный материал в индекс эластика.

– OCR Service при наличии документов для сканирования инициирует распознавание текста.

– Moderation создает задачу проверки в очереди модераторов.

Доступ из поиска становится возможным сразу после подтверждения индексации.

Аналогичные события публикуются в Kafka-топике и для обновления/удаления материала.

Алгоритм управления материалом приведен на рисунке 3.2.

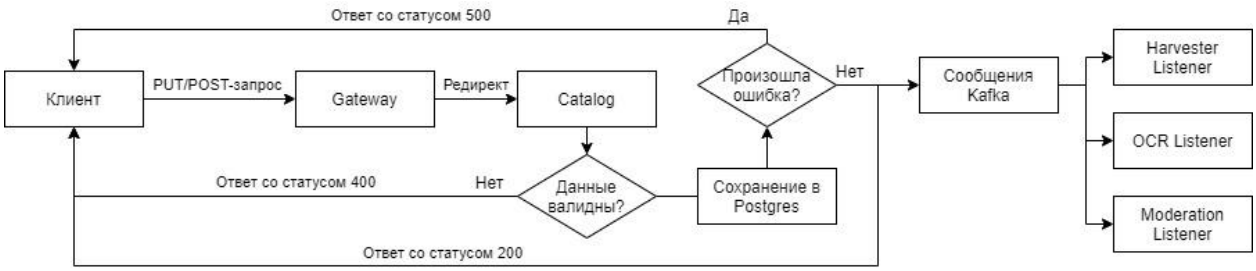

Рисунок 3.2 – Алгоритм управления материалом

Алгоритм поиска.

Алгоритм поисковой подсистемы описывает преобразование пользовательского запроса в структурированный поисковый язык Elasticsearch, выполнение поиска и формирование релевантной выдачи.

Пользователь вводит поисковую фразу в интерфейсе, выбирает опциональные фильтры (категория, временной диапазон и так далее). Discovery, нормализует поисковой запрос: убирает спец символы символы, приводит к нижнему регистру. Выполняется замена исторических вариантов написания через встроенный словарь синонимов: «Витебскъ» → «Витебск», «Віцебск» → «Витебск». Для запросов с опечатками система вычисляет кандидаты исправлений через алгоритм расстояния Левенштейна с порогом 1, подставляя и возможные варианты в поисковой запрос без участия пользователя.

На основе предобработанной фразы формируется сложный bool-запрос, комбинирующий несколько типов соответствия:

– Must (обязательные условия): multi_match по полям с взвешенной значимостью – название (boost 3.0), теги (boost 2.0), описание (boost 1.0), OCR-текст (boost 0.8).

– Filter (жесткие ограничения): точное совпадение статуса PUBLISHED, термы по выбранным категориям, диапазон по дате создания оригинала.

– Should (улучшение релевантности): точное совпадение фразы в названии (match_phrase с boost 5.0), функциональный скоринг по популярности материала (logarithmic boost от количества лайков).

Выполнение поиска и обработка результатов происходит в Elasticsearch. Поиск выполняется по индексу с пагинацией. Параллельно с поиском документов выполняются агрегации для фасетной навигации: terms по категориям с подсчетом документов, date_histogram по декадам. Результаты сортируются по вычисленному score с учетом функционального скоринга, при равенстве – по дате публикации (created_at).

Основная идея состоит в постоянном поддержании согласованности индекса с реляционными базами данных, чтобы в результате поиска пользователь получал релевантную информацию.

Алгоритм поиска приведен на рисунке 3.3.

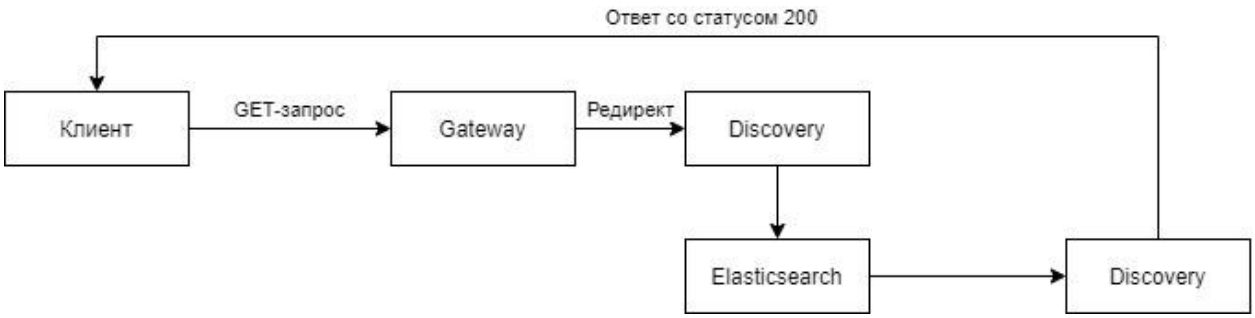

Рисунок 3.3 – Алгоритм поиска

Алгоритм OCR-обработки документов.

Алгоритм OCR-обработки обеспечивает извлечение текстового слоя из отсканированных документов, делая их содержимое доступным для полнотекстового поиска без ручного ввода, через асинхронный конвейер обработки с отслеживанием статуса.

Триггер и постановка в очередь активируется при загрузке файла в Catalog. Сервис определяет необходимость OCR по MIME-типу (image/jpeg, image/png, application/pdf). При положительном решении запись создается в PostgreSQL в таблице ocr_jobs: material_id, file_url в MinIO, detected_language, priority (normal для приватных загрузок и high для публичных загрузок), статус QUEUED, временная метка.

Оцифровка документа выполняется OCR Service – выделенным микросервисом с пулом воркеров. Сервис потребляет записи из таблицы ocr_jobs, обновляет статус в PostgreSQL на PROCESSING. Конвейер обработки включает: декодирование PDF в растровые изображения через Poppler (300 DPI), предобработку изображения (адаптивная бинаризация, медианный фильтр для удаления шума, коррекция наклона через Hough Transform), распознавание через Tesseract 5.x с языковым пакетом rus+bel и режимом LSTM, постобработку через словарь исторических терминов Витебска для коррекции распространенных ошибок распознавания. Результат сохраняется в PostgreSQL: полный текст, структура с координатами блоков (hOCR), средняя уверенность распознавания (0–100) и сам распознанный текст.

Завершение обработки и обновление материала инициируется публикацией события в Kafka-топик material.updates. Событие содержит material_id, распознанный текст, confidence score, путь к hOCR-файлу в MinIO. Catalog потребляет событие, обновляет запись в PostgreSQL: заполняет поле ocr_text, устанавливает ocr_confidence, при значении ниже 60% добавляет флаг ocr_review_required для ручной проверки модератором.

Дублирование в поисковый индекс выполняется через Kafka-Connect. Коннектор извлекает обновленное поле ocr_text, мерджит его в существующий документ Elasticsearch через операцию update с doc_as_upsert, применяя к новому тексту те же анализаторы (морфология, синонимы

исторических названий). Таким образом, распознанный контент становится доступным для полнотекстового поиска без участия пользователя.

Алгоритм OCR-обработки приведен на рисунке 3.4.

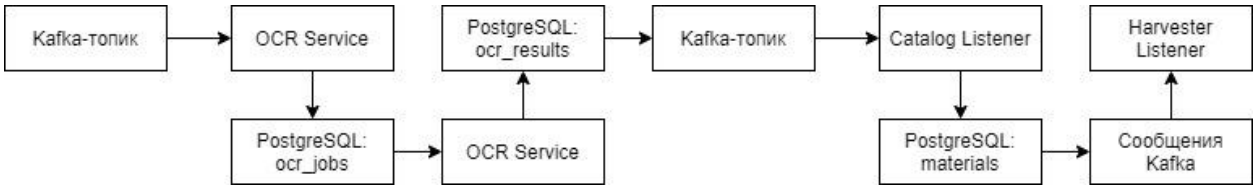

Рисунок 3.4 – Алгоритм OCR-обработки

API Gateway.

Алгоритм работы API Gateway обеспечивает единую точку входа для всех клиентских запросов, централизуя сквозные аспекты безопасности, маршрутизации и наблюдаемости, при этом делегируя бизнес-логику специализированным микросервисам.

Gateway извлекает хост из заголовка Host и определяет целевой маршрут по конфигурации Spring Cloud Gateway [28]. Для всех эндпоинтов, кроме публичных (/api/v1/search, /api/v1/materials/{id}, /api/v1/registration и рядя прочих), требуется наличие заголовка Authorization: Bearer {token}.

Валидация JWT-токена выполняется в два уровня [29]. Первый уровень – offline-валидация Gateway без обращения к внешним сервисам: парсинг структуры JWT (header.payload.signature), проверка подписи через кэшированный публичный ключ из JWKS endpoint Keycloak (обновление кэша каждые 24 часа), проверка стандартных claims – exp (истечение срока), nbf (не ранее), iss (эмитент), aud (аудитория). При невалидной подписи или истекшем сроке – немедленный ответ 401 Unauthorized. Второй уровень –online-валидация через introspection endpoint Keycloak, проверяющая отзыв токена.

При успешной валидации Gateway извлекает парсит JWT-токен на X-заголовки для последующего редиректа к микросервисам. Gateway проверяет соответствие запрошенного ресурса и HTTP-метода ролям пользователя по конфигурации маршрутов: например, POST /api/v1/materials требует USER, PUT /api/v1/materials/{id}/moderate требует MODERATOR. При недостаточных правах – ответ 403 Forbidden без проксирования запроса.

Rate limiting и защита от злоупотреблений применяется до маршрутизации. Для анонимных запросов ключ лимита – хеш IP + User-Agent, лимит 100 запросов в минуту. Для авторизованных – ключ по X-User-Id, лимит 1000 запросов в минуту с возможностью burst 10 запросов. Реализация через Redis с алгоритмом token bucket, ключ ratelimit:{type}:{key} с TTL 60 секунд. Превышение лимита – ответ 429 Too Many Requests с заголовком Retry-After.

Обработка ответа: Gateway перехватывает ответ целевого сервиса, добавляет CORS-заголовки для разрешенных origin.

Алгоритм работы API Gateway приведен на рисунке 3.5.

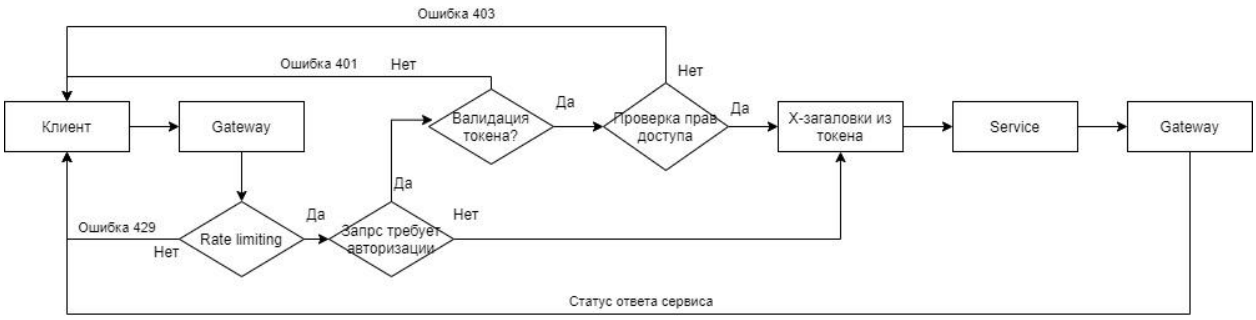

Рисунок 3.4 – Алгоритм API Gateway

Приведенные алгоритмы демонстрируют архитектурные решения, обеспечивающие функционирование открытого электронного архива как распределенной системы с четким разделением ответственности между компонентами. Интеграция с Keycloak через выделенный микросервис Identity централизует управление идентификацией, сохраняя при этом гибкость для расширения бизнес-логики профилей. Конвейер обработки материалов от загрузки до поисковой доступности, построенный на Kafka и Elasticsearch, обеспечивает масштабируемость и устойчивость к пиковым нагрузкам при краудсорсинговом наполнении архива. Асинхронная OCR-обработка с автоматической индексацией результатов исключает ручной труд при оцифровке документов. API Gateway, выполняющий валидацию токенов и контроль доступа, создает единую точку входа с единообразной политикой безопасности для всех клиентов. В совокупности данные алгоритмы обеспечивают баланс между открытостью платформы для пользователей и контролем качества через модерацию, что критично для сохранения достоверности исторических материалов.

3.3 Структура базы данных веб-приложения

Данные представляют собой критически важный актив веб-приложения «Открытый электронный архив города Витебска», являясь, по существу, сердцевиной всей системы. Сохранность, целостность и доступность исторических материалов, метаданных и пользовательской информации определяют успешность функционирования архива как долгосрочного хранилища культурного наследия. Для выполнения разнородных требований к хранению и обработке данных применяется распределенная многоуровневая архитектура хранилищ, где каждый компонент специализируется на своей задаче. Реляционная база данных PostgreSQL обеспечивает транзакционную целостность и сложные связи между сущностями. Поисковый движок Elasticsearch хранит оптимизированные для полнотекстового поиска индексы. Объектное хранилище S3-совместимого MinIO содержит бинарные файлы неструктурированных материалов. Брокер сообщений Apache Kafka с различными топиками (для чатов между пользователями и модераторами, нотификаций, событий модерации и поисковой индексации) обеспечивает

потоковую обработку и временное хранение событий. Хранилище ключ-значение Redis используется для кэширования часто запрашиваемых данных, управления сессиями пользователей и rate limiting. Такое разделение позволяет масштабировать каждый уровень независимо и обеспечивает устойчивость системы к отказам отдельных компонентов.

Реляционные базы данных

Архитектура микросервисов предполагает изоляцию доменных данных: каждый бизнес-модуль взаимодействует с выделенным логическим экземпляром PostgreSQL, исключая непосредственное обращение одного сервиса к таблицам другого. Данный паттерн Database per Service декомпозирует монолитную схему на шесть независимых контекстов: Identity (управление пользователями), Catalog (материалы и файлы), Social (социальное взаимодействие), OCR (задачи распознавания) и служебную базу Keycloak. Ниже приведено детальное описание структуры каждой базы данных с перечнем таблиц, атрибутов и индексов.

Identity отвечает за управление профилями пользователей и интеграцию с Keycloak. Собственная база данных сервиса хранит расширенную информацию о пользователях, дополняющую учётные данные Keycloak. Схема состоит из одной основной таблицы 3.2

Таблица 3.2 – Таблица users

Атрибут	Тип	Описание	Ограничения
id	BIGINT	Первичный ключ	PRIMARY KEY
username	VARCHAR(255)	Имя пользователя	UNIQUE, NOT NULL
email	VARCHAR(255)	Электронная почта	UNIQUE, NOT NULL
avatar_url	VARCHAR(255)	Путь к аватару в s3	
status	ENUM	Статус аккаунта	DEFAULT 'ACTIVE', NOT NULL
permitions	JSONB	Набор пермишенов пользователя	DEFAULT '[]', NOT NULL
created_at	TIMESTAMP	Дата создания записи	DEFAULT now()
updated_at	TIMESTAMP	Дата последнего обновления записи	

Для оптимизации типовых операций на таблицу наложены три индекса. Уникальные индексы на username и email поддерживают аутентификацию и поиск пользователей по этим атрибутам, а также обеспечивают уникальность в рамках системы. Индекс на поле status ускоряет фильтрацию пользователей при выборке активных аккаунтов или обработке деактиваций. Триггер updated_at автоматически модифицирует временную метку при каждом обновлении записи.

Поле status принимает значения: ACTIVE – аккаунт функционирует нормально, INACTIVE – деактивирован пользователем или администратором, SUSPENDED – временная блокировка.

Keycloak функционирует с собственной служебной базой данных (таблицы realm, user_entity, credential, user_role_mapping, user_session и др.), доступ к которой осуществляется исключительно через Admin API и keycloak-admin-client. Identity не модифицирует таблицы Keycloak напрямую, а выполняет операции создания, обновления и деактивации учётных записей через программный интерфейс. Локальная таблица users поддерживает кэширование часто запрашиваемых атрибутов и хранение дополнительных бизнес-полей, отсутствующих в стандартной схеме Keycloak.

Catalog управляет жизненным циклом архивных материалов и их метаданными. База данных содержит центральную таблицу 3.3, агрегирующую всю информацию о материале, включая ссылку на файл в объектном хранилище.

Таблица 3.3 – Таблица materials

Атрибут	Тип	Описание	Ограничения
id	BIGINT	Первичный ключ материала	PRIMARY KEY
user_id	BIGINT	Идентификатор автора	NOT NULL
title	VARCHAR(500)	Название материала	NOT NULL
description	TEXT	Описание содержания	
status	ENUM	Статус жизненного цикла	DEFAULT DRAFT, NOT NULL
file_key	VARCHAR(100)	Путь к файлу в MinIO	
file_type	VARCHAR(100)	MIME-тип файла	
file_size	BIGINT	Размер файла в байтах	
metadata	JSONB	Гибкие метаданные	
ocr_text	TEXT	Распознанный текст	
ocr_confidence	DECIMAL(5,2)	Уверенность OCR (0-100)	

Продолжение таблица 3.3.

ocr_review_required	BOOLEAN	Флаг ручной проверки OCR	DEFAULT FALSE
view_count	BIGINT	Счётчик просмотров	DEFAULT 0
created_at	TIMESTAMP	Дата создания записи	DEFAULT now(), NOT NULL
updated_at	TIMESTAMP	Дата последнего обновления	DEFAULT now()
published_at	TIMESTAMP	Дата публикации	

Индексы на таблице оптимизируют ключевые паттерны доступа. GIN-индекс на поле metadata обеспечивает эффективный поиск по вложенным JSON-структурам, включая фильтрацию по категориям, тегам и произвольным атрибутам материалов. Составной индекс на поля status и created_at ускоряет выборку списков материалов с сортировкой по дате – наиболее частая операция в личном кабинете пользователя и при публичном просмотре. Индекс на user_id оптимизирует получение всех материалов конкретного автора. Отдельный индекс на status поддерживает административные выборки по статусам модерации. Индекс на file_key обеспечивает целостность ссылок на объектное хранилище и быстрый поиск материала по пути файла при обработке событий от MinIO.

Поле status реализует конечный автомат состояний: DRAFT – черновик, виден только автору; MODERATION – на проверке модератором; PUBLISHED – опубликован и доступен для поиска; REJECTED – отклонён модератором; ARCHIVED – снят с публикации.

Поля file_key, file_type и file_size заполняются при загрузке файла и содержат информацию, необходимую для генерации presigned URL и отображения метаданных без обращения к MinIO. Поле metadata агрегирует в себе всю дополнительную классификацию: категории, теги, исторические даты и прочие атрибуты, специфичные для предметной области архива.

Social управляет социальным взаимодействием пользователей: лайками материалов и подписками на авторов. База данных содержит две таблицы: реализующие связи многие-ко-многим. Таблица лайков 3.4 и таблица подписок 3.5.

Таблица 3.4 – Таблица likes

Атрибут	Тип	Описание	Ограничения
user_id	UUID	Идентификатор пользователя, поставившего лайк	NOT NULL
material_id	UUID	Идентификатор материала	NOT NULL
created_at	TIMESTAMP	Дата постановки лайка	DEFAULT now(), NOT NULL

Первичный ключ составной – пара полей user_id и material_id, что гарантирует уникальность лайка пользователя на конкретный материал и обеспечивает быстрый поиск по обеим сторонам связи. Индекс на поле material_id оптимизирует подсчёт количества лайков материала и выборку списка пользователей, поставивших лайк. Индекс на поле user_id ускоряет получение истории лайков пользователя для персонализированных рекомендаций. Индекс на created_at поддерживает сортировку по времени при выборке последних активностей.

Таблица 3.5 – Таблица subscriptions

Атрибут	Тип	Описание	Ограничения
followers	ARRAY	Идентификатор подписчика	NOT NULL
following_id	BIGINT	Идентификатор объекта подписки	NOT NULL
following_type	ENUM	Тип объекта подписки	NOT NULL
created_at	TIMESTAMP	Дата создания подписки	NOT NULL

Первичный ключ составной – два поля following_id и following_type. Данная структура обеспечивает – уникальность подписки пользователя на конкретный объект определённого типа и позволяет подписываться на разные сущности с пересекающимися идентификаторами. Индекс на поля following_id и following_type оптимизирует выборку подписчиков конкретного объекта – необходима для рассылки уведомлений о новых материалах автора, публикациях в категории или появлении материалов с отслеживаемым тегом.

Поле following_type определяет семантику подписки. Значение USER соответствует подписке на автора материалов – при публикации нового материала подписчик получает уведомление. Значение CATEGORY означает подписку на рубрику архива – уведомление приходит при появлении

материала в указанной категории. Значение TAG реализует подписку на тег – уведомление при публикации материала с отслеживаемым ключевым словом.

Таблица не содержит внешних ключей на уровне базы данных – целостность обеспечивается на уровне приложения. При удалении пользователя, категории или тега в соответствующем сервисе генерируется событие в Kafka, которое обрабатывается Social для каскадного удаления связанных подписок.

OCR Service управляет асинхронным конвейером распознавания текста из отсканированных документов. База данных содержит две таблицы: очередь задач на обработку 3.6 и результаты распознавания 3.7.

Таблица 3.6 – Таблица ocr_jobs

Атрибут	Тип	Описание	Ограничения
id	BIGINT	Первичный ключ задачи	PRIMARY KEY
material_id	BIGINT	Идентификатор материала	NOT NULL
file_key	VARCHAR(100)	Путь к файлу в MinIO	NOT NULL
status	ENUM	Статус обработки	DEFAULT QUEUED, NOT NULL
priority	ENUM	Приоритет выполнения	DEFAULT NORMAL
detected_language	VARCHAR(10)	Определённый язык документа	
error_message	TEXT	Сообщение об ошибке при сбое	
created_at	TIMESTAMP	Дата постановки в очередь	NOT NULL
started_at	TIMESTAMP	Дата начала обработки	
completed_at	TIMESTAMP	Дата завершения обработки	

Индексы оптимизируют выборку задач и мониторинг очереди. Составной индекс на поля status, priority и created_at обеспечивает эффективную выборку следующей задачи из очереди: сначала по приоритету, затем по времени ожидания. Индекс на material_id ускоряет проверку существования задачи для конкретного материала и предотвращает дублирование. Индекс на file_key связывает задачу с объектом в хранилище. Частичный индекс на поле status со значением FAILED поддерживает быструю выборку неудачных задач для повторной обработки.

Поле status отслеживает жизненный цикл задачи. Значение QUEUED назначается при создании задачи и постановке в очередь. Статус PROCESSING устанавливается при захвате задачи воркером. Значение COMPLETED присваивается при успешном завершении распознавания. Статус FAILED фиксирует ошибку обработки с записью причины в поле error_message.

Таблица 3.7 – Таблица ocr_results

Атрибут	Тип	Описание	Ограничения
id	BIGINT	Первичный ключ результата	PRIMARY KEY
job_id	BIGINT	Идентификатор задачи	NOT NULL
recognized_text	TEXT	Полный распознанный текст	
confidence_score	DECIMAL(5,2)	Средняя уверенность распознавания (0-100)	
page_count	INTEGER	Количество обработанных страниц	
processing_time_ms	BIGINT	Длительность обработки в миллисекундах	
created_at	TIMESTAMP	Дата сохранения результата	NOT NULL

Индекс на поле job_id обеспечивает уникальность соответствия «одна задача — один результат» и быстрый поиск результата по идентификатору задачи. Поле recognized_text хранит извлечённый текстовый слой, который публикуется в Kafka для обновления индекса Elasticsearch. Поле hocr_key содержит ссылку на файл с разметкой расположения текстовых блоков, используемый для отображения структуры документа. Поле confidence_score ниже порога 60% инициирует установку флага ocr_review_required в сервисе Catalog через событие Kafka.

Elasticsearch Index.

Elasticsearch выполняет роль специализированного поискового хранилища, денормализующего данные из реляционных баз для обеспечения полнотекстового поиска, морфологического анализа и фасетной навигации. Индекс materials строится путём агрегации информации из Catalog (материалы, метаданные, OCR-текст), Identity (имена авторов) и Social (количество лайков). Синхронизация осуществляется через Kafka Connect:

изменения в PostgreSQL публикуются в топики, коннектор извлекает обновления и индексирует документы в Elasticsearch. Структура данного индекса приведена в таблице 3.8.

Таблица 3.8 – Индекс materials

Поле	Тип	Анализатор	Описание
id	keyword	—	Идентификатор материала, соответствует UUID в PostgreSQL
title	text	russian_morphology	Название материала с повышенным весом (boost 3.0)
title.keyword	keyword	—	Точное значение для сортировки и агрегаций
description	text	russian_morphology	Описание материала (boost 2.0)
ocr_text	text	russian_morphology	Распознанный текст из документов (boost 0.8)
author_id	keyword	—	Идентификатор автора
author_name	text	standard	Отображаемое имя автора для поиска
status	keyword	—	Статус материала, индексируются только PUBLISHED
categories	keyword	—	Массив категорий для фасетной фильтрации
tags	keyword	—	Массив тегов (boost 2.0)
metadata	object	—	Денормализованные метаданные (дата создания оригинала, место, источник)
metadata.year	integer	—	Год создания оригинала для диапазонной фильтрации
likes_count	integer	—	Количество лайков для функционального скоринга
view_count	integer	—	Счётчик просмотров
created_at	date	—	Дата публикации в архиве
published_at	date	—	Дата публикации материала
ocr_confidence	float	—	Уверенность распознавания для фильтрации низкокачественного OCR
suggest	completion	—	Поле для автодополнения и подсказок

Анализатор russian_morphology объединяет стандартную токенизацию с морфологической обработкой русского и белорусского языков через плагин analysis-morphology. Настроенные синонимы преобразуют исторические варианты написания: «Витебскъ» → «витебск», «Віцебск» → «витебск». Фильтр word_delimiter обрабатывает дефисные соединения и цифробуквенные обозначения, характерные для архивных описей.

Поле suggest типа completion обеспечивает мгновенные подсказки при вводе запроса пользователем, предлагая варианты из названий материалов, имён авторов и популярных тегов. Поле metadata.location типа geo_point закладывает основу для геопространственного поиска материалов по месту съёмки или хранения документа.

Настройки индекса

Индекс сконфигурирован с одним шардом и двумя репликами для обеспечения отказоустойчивости при минимальном объёме данных на старте. Пороговое значение refresh_interval установлено в 1 секунду для баланса между актуальностью поиска и нагрузкой на систему. Маппинг динамически отклоняет неизвестные поля в объекте metadata для предотвращения инъекции некорректных данных.

3.4 Выводы и оценка результатов разработки

В ходе дипломного проектирования разработана информационная система «Открытый электронный архив города Витебска» — масштабируемая краудсорсинговая платформа для сбора, хранения и интеллектуального поиска исторических материалов. Реализованное веб-приложение объединяет современные архитектурные паттерны и технологический стек, обеспечивающий устойчивость к нагрузкам, отказоустойчивость и возможность эволюции системы без нарушения работоспособности существующих компонентов.

Архитектурные решения, принятые на этапе проектирования, подтвердили свою эффективность в реализации. Применение паттерна микросервисной архитектуры с разделением на шесть доменных контекстов (Identity, Catalog, Social, Moderation, OCR, Gateway) позволило разрабатывать микросервисы автономно. Паттерн Backend for Frontend обеспечил чёткое разделение ответственности между клиентским приложением и внутренней инфраструктурой, скрыв сложность межсервисного взаимодействия за единым стабильным API. Гибридная схема коммуникации — синхронная через WebClient для критичных по времени операций и асинхронная через Apache Kafka для фоновой обработки — сбалансировала требования к производительности и надёжности системы.

Технологический стек, выбранный для реализации, полностью покрыл функциональные требования проекта. Java 21 и Spring Boot 4.x обеспечили высокую производительность при IO-интенсивных операциях, характерных для работы с файлами и сетевыми запросами. PostgreSQL с расширением JSONB успешно справился с хранением разнородных метаданных архивных

материалов без фиксированной схемы. Elasticsearch реализовал полнотекстовый поиск с морфологической обработкой русского и белорусского языков, нечётким поиском и фасетной навигацией. Tesseract OCR интегрирован как экономически эффективное решение для оцифровки документов, а MinIO обеспечил S3-совместимое объектное хранилище без лицензионных затрат.

Реализованные алгоритмы управления жизненным циклом материалов обеспечивают контроль качества контента через многоступенчатую модерацию при сохранении открытости платформы для пользователей. Конвейер обработки от загрузки файла до поисковой доступности полностью автоматизирован: OCR-распознавание, индексация в Elasticsearch и рассылка уведомлений подписчикам выполняются асинхронно без блокировки пользовательского интерфейса. Интеграция с Keycloak централизовала управление идентификацией и авторизацией, обеспечив при этом гибкость для расширения бизнес-логики профилей.

Структура базы данных отражает принцип полиглотной персистентности: реляционные базы PostgreSQL обеспечивают транзакционную целостность, Elasticsearch — специализированный поиск, MinIO — хранение неструктурированных данных, Redis — кэширование и сессии. Паттерн Database per Service гарантирует изоляцию доменов и автономность развития подсистем. Спроектированные индексы оптимизируют типовые паттерны доступа: составные индексы для сортировки списков, GIN-индексы для JSONB-полей, полнотекстовые индексы для резервного поиска.

Система готова к дальнейшему развитию и масштабированию. Микросервисная архитектура позволяет независимо масштабировать отдельные компоненты при росте нагрузки: OCR Service — при увеличении объёма сканирования, Social— при росте пользовательской активности, Elasticsearch — при расширении корпуса документов.

Таким образом, разработанная система достигает поставленных целей: обеспечивает долгосрочное сохранение исторических материалов, инструменты их коллективного наполнения, интеллектуальный поиск и доступность для широкой аудитории. Архитектурные решения и технологический стек соответствуют современным стандартам разработки распределённых систем и создают фундамент для эволюции платформы в полноценный цифровой архив культурного наследия города Витебска.

4 ТЕСТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ ОТКРЫТОГО ЭЛЕКТРОННОГО АРХИВА ИСТОРИИ Г. ВИТЕБСКА

Тестирование программного обеспечения – это процесс анализа программного средства и сопутствующей документации с целью выявления дефектов и повышения качества продукта. Конечной целью тестирования является предоставление пользователю качественного программного обеспечения, соответствующего заявленным требованиям и ожиданиям.

В рамках дипломного проектирования веб-приложения «Открытый электронный архив города Витебска» проводится комплексное тестирование, включающее два основных направления: функциональное тестирование и юзабилити-тестирование. Благодаря этому особое внимание уделяется не только корректности работы программного обеспечения, но и удобству его использования конечными пользователями.

Объект тестирования – веб-приложение «Открытый электронный архив города Витебска», представляющее собой краудсорсинговую платформу для сбора, хранения и интеллектуального поиска исторических материалов.

Предмет тестирования:

– корректность реализации функциональных требований;

– соответствие графического интерфейса пользователя эргономическим требованиям;

– удобство использования и доступность приложения для целевой аудитории;

– корректность обработки входных, промежуточных и выходных данных.

Виды тестирования, применяемые в работе:

14 Функциональное тестирование – проверка соответствия реализованной функциональности заявленным требованиям, включая тестирование модулей аутентификации, управления материалами, поиска, модерации и социального взаимодействия [30].

15 Юзабилити-тестирование – оценка степени удобства использования программного продукта, включающая экспертный подход (проверка по эвристикам Якоба Нильсена) и пользовательский подход (тестирование с участием представителей целевой аудитории) [31].

Раздел включает разработку тестовой документации, выполнение функционального тестирования с описанием найденных дефектов, проведение юзабилити-тестирования по экспертному и пользовательскому подходам, а также формирование итогового отчёта о качестве программного средства.

Функциональное тестирование

Тестирование веб-приложения «Открытый электронный архив города Витебска» проводилось в два этапа. На первом этапе выполнялось тестирование серверной части (backend), поскольку вся бизнес-логика системы реализована в микросервисах. Клиентская часть (frontend) построена

по принципу «тонкого клиента» и не содержит сложной логики, поэтому её тестирование проводилось после стабилизации backend.

Тестирование backend выполнялось через интерактивную документацию Swagger UI и инструмент Postman. Проверялись шесть микросервисов: Identity, Catalog, Discovery, Moderation, OCR и Gateway.

В ходе тестирования backend выявлены дефекты различной степени критичности. В таблице 4.1 представлен сводный перечень выявленных дефектов.

Таблица 4.1 – Обнаруженные дефекты сервера.

№	Модуль	Название дефекта	Severity	Статус
1	Identity	Отсутствие валидации формата email при регистрации	Major	Исправлен
2	Catalog	Переполнение поля title вызывает SQL-исключение	Critical	Исправлен
3	Discovery	SQL-инъекция в параметре поискового запроса	Critical	Исправлен
4	Catalog	Создание материала с отсутствующим обязательным полем title	Major	Исправлен
5	Gateway	Некорректная обработка истёкших JWT-токенов	Average	Исправлен
6	Moderation	Race condition при одновременной модерации материала	Major	Исправлен
7	OCR Service	Некорректная обработка повреждённых изображений	Average	Исправлен
8	Discovery	Некорректная пагинация при отрицательном значении page	Minor	Исправлен
9	Social	Дублирование лайка при параллельных запросах	Average	Исправлен
10	Gateway	Утечка информации в сообщениях об ошибках	Minor	Исправлен
11	Identity	Некорректная обработка дублирующего email при регистрации	Major	Исправлен
12	Catalog	Неверный статус материала после ошибки OCR	Average	Исправлен
13	Discovery	Некорректная сортировка результатов по релевантности	Minor	Исправлен
14	Moderation	Отсутствие уведомления автору при отклонении материала	Average	Исправлен
15	Gateway	Отсутствие rate limiting на эндпоинте авторизации	Minor	Исправлен

Для систематизации функционального тестирования разработана тестовая документация в виде набора тест-кейсов [32]. В таблице 4.2 представлены тест-кейсы, соответствующие выявленным дефектам уровня Major [33].

Таблица 4.2 – Тест-кейсы для требований уровня Major.

№	Требование	Модуль	Подмодуль	Описание тест-кейса	Ожидаемые результаты
1	REQ-2.1	Identity	Регистрация	Валидация формата email при регистрации 1. Отправить POST /api/v1/auth/register 2. В теле запроса указать email без символа «@»: «invalid-email» 3. Указать корректный password и username	HTTP 400 Bad Request, сообщение: «Некорректный формат email», пользователь не создан
2	REQ-3.1	Catalog	Создание материала	Проверка максимальной длины поля title 1. Авторизоваться в системе 2. Отправить POST /api/v1/materials 3. В поле title передать строку длиной 600 символов 4. Указать остальные обязательные поля корректно	HTTP 400 Bad Request, сообщение: «Поле title превышает максимальную длину 500 символов», материал не создан
3	REQ-3.1	Catalog	Создание материала	Проверка обязательности поля title 1. Авторизоваться в системе 2. Отправить POST /api/v1/materials 3. В теле запроса отсутствует поле title 4. Указать description и category	HTTP 400 Bad Request, сообщение: «Поле title обязательно для заполнения», материал не создан

Продолжение таблицы 4.2.

4	REQ-5.2	Moderation	Изменение статуса	Проверка конкурентной модерации 1. Авторизоваться двумя модераторами в разных сессиях 2. Открыть материал со статусом «На модерации» 3. Одновременно отправить: модератор 1 – запрос на публикацию, модератор 2 – запрос на отклонение	HTTP 200 OK для первого запроса, HTTP 409 Conflict для второго, статус материала соответствует первому действию, история изменений консистентна
5	REQ-2.1	Identity	Регистрация	Проверка уникальности email 1. Создать пользователя с email: user@example.com 2. Отправить повторный POST /api/v1/auth/register с тем же email 3. Указать другой username	HTTP 409 Conflict, сообщение: «Пользователь с указанным email уже существует», дубликат не создан

В таблице 4.3 представлено детальное описание выявленных дефектов уровня Major, обнаруженных в ходе функционального тестирования системы. Каждый дефект сопровождается алгоритмом воспроизведения, фактическим и ожидаемым результатами, а также ссылкой на соответствующее приложение с иллюстрацией. Данные дефекты были исправлены перед выпуском продукта, так как они критически влияют на корректность работы ключевых функциональных модулей.

Таблица 4.3 – Описание дефектов уровня Major.

№	Название дефекта	Важность	Алгоритм воспроизведения	Фактический результат	Ожидаемый результат	Приложение
1	Отсутствие валидации формата email при регистрации	Major	1. Отправить POST /api/v1/auth/register 2. В теле запроса указать: {"email": "invalid-email", "password": "pass123", "username": "testuser"}	HTTP 201 Created, пользователь создан с невалидным email в базе данных	HTTP 400 Bad Request, сообщение: «Некорректный формат email», пользователь не создан	Рисунок 4.1
2	Переполнение поля title вызывает SQL-исключение	Major	1. Авторизоваться в системе 2. Отправить POST /api/v1/materials 3. В поле title передать строку длиной 600 символов 4. Указать остальные обязательные поля корректно	HTTP 500 Internal Server Error, в теле ответа содержится stack trace с деталями исключения Hibernate	HTTP 400 Bad Request, сообщение: «Поле title превышает максимальную длину 500 символов», материал не создан	Рисунок 4.2

Продолжение таблицы 4.3.

3	Создание материала с отсутствующим обязательным полем title	Major	1.Авторизоваться в системе 2. Отправить POST /api/v1/materials 3. В теле запроса отсутствует поле title: {"description": "Тестовое описание", "category": "PHOTO"}	HTTP 201 Created, материал создан с null в поле title, отображается в списке без названия	HTTP 400 Bad Request, сообщение: «Поле title обязательно для заполнения», материал не создан	Рисунок 4.3
4	Race condition при одновременной модерации материала	Major	1.Авторизоваться двумя модераторами в разных сессиях 2. Открыть один и тот же материал со статусом «На модерации» 3. Одновременно отправить запросы: модератор 1 – «Опубликовать», модератор 2 – «Отклонить»	Статус материала определяется последним обработанным запросом, история изменений неконсистентна, возможен статус «Опубликован» с комментарием «Отклонено»	HTTP 409 Conflict для второго запроса, сохранение целостности конечного автомата статусов, корректная история изменений	Рисунок 4.4
5	Некорректная обработка дублирующего email при регистрации	Major	1. Создать пользователя с email: user@example.com 2. Отправить повторный POST /api/v1/auth/register с тем же email	HTTP 500 Internal Server Error, исключение нарушения уникальности на уровне базы данных	HTTP 409 Conflict, сообщение: «Пользователь с указанным email уже существует»	Рисунок 4.5

Server response

Code	Details
201 *Undocumented*	Response body 5 Response headers connection: keep-alive content-type: application/json date: Thu,26 Mar 2026 19:09:15 GMT keep-alive: timeout=60 transfer-encoding: chunked

Responses

Рисунок 4.1 – Фактический результат дефекта 1.

Code	Details
500	Error: response status is 500 Response body


```
{
  "timestamp": "2026-03-26T19:09:57.421984102",
  "status": 500,
  "code": "INT",
  "message": "500 INTERNAL_SERVER_ERROR \"could not execute statement; SQL [n/a]; nested exception is org.hibernate.exception.DataException:
}
```


Рисунок 4.2 – Фактический результат дефекта 2.

Code	Details
201 *Undocumented*	Response body 4

Рисунок 4.3 – Фактический результат дефекта 3.

Code	Details
200	Response body 3

Рисунок 4.4 – Фактический результат дефекта 4.

500 Error: response status is 500

Response body


```
{
  "timestamp": "2026-03-26T19:11:11.866629625",
  "status": 500,
  "code": "INT",
  "message": "500 INTERNAL_SERVER_ERROR \"could not execute statement; SQL [n/a]; constraint [uk_user_email];
execute statement\""
}
```


Рисунок 4.5 – Фактический результат дефекта 5.

Тестирование клиентской части проводилось после стабилизации backend. Проверялись модули: авторизация, главная страница с поиском, личный кабинет, страница материала, модерация.

Критических и значимых дефектов в клиентской части выявлено не было. Все функциональные сценарии отработали корректно.

Отсутствие дефектов frontend обусловлено применённой архитектурой «тонкий клиент» и паттерном BFF (Backend for Frontend):

– бизнес-логика полностью вынесена на сервер, клиент отвечает исключительно за отображение данных;

– вся валидация критичных данных выполняется на backend, исключая зависимость от корректности frontend;

– единый стабильный API Gateway изолирует клиент от изменений внутренней микросервисной структуры.

Юзабилити-тестирование

Юзабилити-тестирование проводилось с целью определения степени удобства использования веб-приложения «Открытый электронный архив города Витебска» для конечных пользователей. Тестирование выполнялось двумя методами: экспертным подходом и пользовательским подходом.

Экспертный подход реализован в виде эвристической оценки интерфейса по десяти принципам юзабилити Якоба Нильсена [34]. Результаты экспертной оцени приведены в таблице 4.4

Таблица 4.4 – Результаты экспертной оценки юзабилити-тестирования.

№	Эвристика	Описание	Результат оценки	Замечания
1	Информированность о состоянии системы	Пользователь всегда должен понимать, что происходит в системе	Выполнено	Индикаторы загрузки, статусы модерации, уведомления об успешных действиях

Продолжение таблицы 4.4.

2	Схожесть системы с реальным миром	Использование понятных пользователю терминов и концепций	Выполнено	Термины «архив», «материал», «модерация» соответствуют предметной области
3	Свобода действий	Возможность отмены действий и возврата к предыдущему состоянию	Частично выполнено	Отмена редактирования реализована, отмена публикации – нет
4	Единообразие и стандарты	Единые обозначения и действия для одинаковых элементов	Выполнено	Единый стиль кнопок, иконок, расположение навигации
5	Предотвращение ошибок	Сведение к минимуму условий для возникновения ошибок	Выполнено	Валидация полей, подтверждение удаления, примеры заполнения
6	На виду, а не в памяти	Информация должна быть видимой, не требовать запоминания	Выполнено	Подсказки, контекстная справка, хлебные крошки
7	Гибкость и эффективность	Ускорение работы опытных пользователей	Выполнено	Горячие клавиши, автозаполнение, шаблоны метаданных
8	Эстетичный и минималистичный дизайн	Отсутствие лишней информации	Выполнено	Чистый интерфейс, акцент на контенте, нейтральная цветовая схема

Продолжение таблицы 4.4.

9	Понимание проблем и их решение	Понятные сообщения об ошибках и способы их устранения	Выполнено	Информативные сообщения, ссылки на справку, чат с модератором
10	Справочные материалы и документация	Доступность помощи при необходимости	Частично выполнено	Встроенные подсказки есть, отдельная документация – нет

Но были выявлены отдельные недочёты, связанные с удобством взаимодействия пользователей с ключевыми функциональными элементами. Данные дефекты приведены в таблице 4.5

Таблица 4.5 – Дефекты найденные в результате экспертной оценки.

№	Дефект	Эвристика	Рекомендация
1	Отсутствие кнопки «Отменить публикацию» после одобрения материала	Свобода действий	Добавить возможность снятия материала с публикации в личном кабинете
2	Отсутствие раздела «Помощь» или «FAQ»	Справочные материалы	Создать раздел с инструкциями по работе с архивом

Далее было выполнено юзабилити-тестирование с приминением пользовательского подхода.

Цель заказчика: создание открытой краудсорсинговой платформы для сбора и сохранения исторических материалов города Витебска с высоким уровнем вовлечённости граждан.

Цель пользователя: найти интересующую информацию о прошлом города, поделиться собственными материалами, получить обратную связь от экспертного сообщества.

Целевая аудитория, разделенная на сегменты представлена в таблице 4.6

Таблица 4.6 – Целевая аудитория.

Сегмент	Характеристика	Потребности	Сегмент
Краеведы и историки	Возраст 40–70 лет, высшее образование, опыт работы с архивами	Достоверность источников, детальные метаданные, экспорт материалов	Краеведы и историки
Обычные горожане	Возраст 25–60 лет, разный уровень компьютерной грамотности	Простота загрузки фото, визуальная привлекательность, социальное взаимодействие	Обычные горожане
Модераторы и администраторы	Сотрудники архивов, библиотек, музеев	Эффективные инструменты проверки, статистика, управление контентом	Модераторы и администраторы

Ниже на рисунках 4.6-4.8 приведены карты эмпатии для каждого из сегментов целевой аудитории [35].

Рисунок 4.6 – Карта эмпатии для сегмента «Краеведы и историки».

Рисунок 4.7 – Карта эмпатии для сегмента «Обычные горожане».

Рисунок 4.8 – Карта эмпатии для сегмента «Модераторы и администраторы».

Возможные сценарии взаимодействия приведены в таблице 4.7

Таблица 4.7 – Сценарии взаимодействия.

Шаг	Действие	Вопросы пользователя	Пожелания	Эмоции	Комментарии
1	Регистрация в системе	«Сложно ли будет разобраться? Успею ли до вечера?»	Простая форма без лишних полей	Настороженность, надежда	Впервые на платформе, пришёл по рекомендации коллеги
2	Вход в личный кабинет	«Где здесь загружать фотографии?»	Понятная навигация, видимая кнопка	Ожидание, любопытство	Интерфейс соответствует макетам (рис. 2.7)
3	Нажатие «+» для создания публикации	«Что если я что-то неправильно заполню?»	Подсказки по каждому полю, примеры	Волнение, ответственность	Модальное окно «Создать публикацию» (рис. 2.10)
4	Загрузка фотографий	«Поддерживает ли система TIFF? Какое максимальное разрешение?»	Поддержка больших файлов, прогресс загрузки	Тревога (большие файлы), облегчение	Presigned URL для прямой загрузки в MinIO
5	Заполнение метаданных	«Как правильно описать?	Шаблоны описания, словарь терминов, автосохранение	Затруднение, желание сделать качественно	Поля: название, описание, дата создания, категория, теги
6	Отправка на модерацию	«Когда будет результат? Что если отклонят?»	Видимый статус, уведомления на email	Надежда, нетерпение	Статус «На модерации» с иконкой лупы (рис. 2.9)

Продолжение таблицы 4.7.

7	Получение обратной связи от модератора	«Что именно нужно исправить?»	Конкретные замечания, возможность уточнить в чате	Благодарность, мотивация к доработке	Чат с модератором (рис. 2.11)
8	Публикация материала	«Увидят ли это другие? Будет ли польза?»	Статистика просмотров, лайки, комментарии	Гордость, удовлетворение	Материал становится доступен в поиске

Добровольцы успешно протестировали основные сценарии — от регистрации до взаимодействия с модератором. Платформа получила положительную оценку за простоту форм и эффективный поиск, однако потребовала доработки в части информирования о статусах модерации и логики чата. По итогам тестирования внесены изменения: добавлены всплывающие подсказки к статусам и переработан интерфейс чата для повышения его наглядности.

В ходе дипломного проектирования проведено комплексное тестирование веб-приложения «Открытый электронный архив города Витебска», включающее функциональное и юзабилити-тестирование.

Функциональное тестирование серверной части выявило 15 дефектов различной степени критичности, из которых 2 критических, 5 значимых (Major) и 8 незначительных. Все выявленные дефекты были исправлены разработчиком, что подтверждено повторным тестированием. Критические ошибки, связанные с SQL-инъекцией и переполнением полей, устранены в первую очередь. Клиентская часть, построенная по архитектуре «тонкий клиент», показала стабильную работу без критических дефектов благодаря вынесению бизнес-логики на сервер.

Юзабилити-тестирование, выполненное экспертным и пользовательским методами, подтвердило высокое качество интерфейса: 8 из 10 эвристик Якоба Нильсена выполнены полностью, 2 — частично. Выявленные недочёты касались отсутствия возможности отмены публикации и раздела помощи, а также неинтуитивности статусов модерации и логики чата для добровольцев. По результатам пользовательского тестирования внесены изменения: добавлены подсказки к статусам и доработан интерфейс чата.

Разработанная тестовая документация, включающая тест-кейсы и описание дефектов, может быть использована для регрессионного тестирования при дальнейшем развитии системы. Все функциональные требования, заявленные в техническом задании, реализованы и протестированы. Приложение готово к эксплуатации и соответствует требованиям качества для краудсорсинговой платформы с высоким уровнем вовлечённости граждан.

5 ТЕХНИКО-ЭКОНОМЧЕСКОЕ ОБОСНОВАНИЕ СЕБЕСТОИМОЧСТИ ЦЕНЫ ВЕБ-ПРИЛОЖЕНИЯ «ОТКРЫТЫЙ ЭЛЕКТРОННЫЙ АРХИВ ИСТОРИИ Г. ВИТЕБСКА»

5.1 Характеристика веб-приложения

Целью дипломного проекта является разработка веб-приложения открытого электронного архива с функцией интеллектуального поиска.

Назначение данного приложения – предоставление пользователям возможности публикации и просмотр исторических заметок, различного формата.

Платформа представляет собой клиент-серверную архитектуру с использованием NoSQL и S3 хранилищ для организации и обработки данных. Основной функционал включает публикацию материалов с прохождением модерации, интеллектуальную обработку поискового запроса, и составление ленты рекомендаций.

Целевая аудитория – жители, желающие рассказать о событиях, людях или местах своего города, школьники, студенты и исследователям, которые ищут материал для научных работ по истории, а также эксперты, следящие за истинностью исторических фактов.

С экономической точки зрения проект имеет значительный потенциал. Оно помогает пользователям экономить время для поиска исторических фактов, вне зависимости от их цели (поиск материалов для научных работ или досуг). Это повышает удовлетворенность сервисом и подталкиваем пользоваться им на постоянной основе. Распределение данных между разными хранилищами и использование различных инструментов их обработки позволяет снизить операционные требования, следовательно, и расходы для поддержания работы приложения.
Персонализированные рекомендации, и дополнительные функции, например, оцифровка электронных документов, могут привлечь новую аудиторию и помогают удержать старую.

Реализация проекта «Открытый электронный архив города Витебска» отвечает потребностям рынка и позволит объединить усилия жителей, исследователей и энтузиастов для совместного сохранения культурной памяти. Платформа станет современным, удобным и доступным инструментом для систематизации исторических данных и популяризации наследия города.

5.2 Определение объема и трудоемкости веб-приложения

Объем разрабатываемого веб-приложения архива определяется исходя из количества функций, реализуемых программой, которые представлены в таблице 5.1, и рассчитываются по формуле [36]

$$V_0 = \sum_{i=1}^{n} V_i, \tag{5.1}$$

где V_0 – общий объем ПО;

V_i – объем функции ПО;

n – общее число функций.

Среда разработки веб-приложения архива: серверная составляющая – IntelliJ IDEA, клиентская составляющая – Visual Studio Code.

Таблица 5.1 – Функции, реализуемые программой

Наименование функции	Объем серверной части функции V_i	Объем клиентской части функции V_i	Общий объем функции V_i
Организация ввода информации	600	400	1000
Контроль, предварительная обработка и ввод информации	500	100	600
Формирование баз данных	1700	0	1700
Обработка наборов и записей базы данных	2500	0	2500
Манипулирование данными	8500	0	8500
Организация поиска и поиск в базе данных	2300	0	2300
Обработка файлов	800	0	800
Управление файлами	500	0	500
Обработка ошибочных и сбойных ситуаций	400	200	600
Обеспечение интерфейса между компонентами	1200	800	2000
П			
Графический вывод результатов	200	1000	1200
Итого	19200	2500	21700

По формуле (5.1) общий объем веб-приложения составит

$$V_0 = 21700 \text{ LOC}.$$

На основании общего объема ПО определяется нормативная трудоемкость ($Т_Н$) с учетом сложности ПО. Так как разрабатываемый продукт относится к 1-ой группе сложности, то нормативная трудоемкость составляет 556 человеко-дня [36]. Учитывая использование ИИ время, затраченное на разработку, сокращается в среднем на 20% [37]

$$Т_Н = 556 \cdot 0{,}8 = 445 \text{ чел.} - \text{дн.}$$

Общая трудоемкость ПО рассчитывается на основе нормативной с учетом дополнительного коэффициента сложности $К_{СЛ}$ следующим образом [36]:

$$Т_О = Т_Н \cdot (1 + К_{СЛ}), \tag{5.2}$$

где $Т_О$ – общая трудоемкость ПО;

$Т_Н$ – нормативная трудоемкость ПО;

$К_{СЛ}$ – дополнительный коэффициент сложности ПО = 0,08 + 0.07.

Следовательно, по формуле (5.2):

$$Т_О = 445 \cdot (1 + 0{,}15) = 511{,}75.$$

Таким образом, общая трудоемкость с учетом округления составила 512 человеко-дня.

На основании общей трудоемкости разработки ПО и установленного периода разработки устанавливается общая плановая численность разработчиков равна [36]

$$Ч_р = \frac{Т_О}{Т_{РД} \cdot Ф_{ЭФ}}, \tag{5.3}$$

где $Ч_р$ – плановая численность разработчиков (чел.):

$Ф_{ЭФ}$ – годовой эффективный фонд времени работы одного работника в течение года (дней в год);

$Т_{РД}$ – плановая продолжительность разработки ПО (лет). Устанавливается календарным графиком разработки, равна 0,2 года.

Эффективный фонд времени одного работника ($Ф_{ЭФ}$) рассчитывается по формуле [36]

$$Ф_{ЭФ} = Д_Г - Д_П - Д_В - Д_о, \tag{5.3}$$

где $Д_Г$ – количество дней в году (365);

$Д_П$ – количество праздничных дней в году (10 дней);
$Д_В$ – количество выходных дней в году (100 дней);
$Д_о$ – количество дней отпуска (27 дней).

Таким образом, по формуле (5.4) получаем:

$$Ф_{ЭФ} = 365 - 10 - 100 - 27 = 228 \text{ дней.}$$

Рассчитаем общую плановую численность работников по формуле (5.3)

$$Ч_р = \frac{512}{0{,}2 \cdot 228} = 12.$$

Общая плановая численность разработчиков и трудоемкость служат базой для расчета основной заработной платы.

5.3 Расчет себестоимости и цены веб-приложения

Исходные данные для расчета основной заработной платы исполнителя приведены в таблице 5.2.

Таблица 5.2 – Разработчик ПО

Разработчик	Количество человек, шт.	Эффективный фонд времени (дней)	Месячный оклад, руб. [38]
Программист Middle Frontend	1	44	2700
Программист Middle Backend	1	44	2700
Программист Junior Frontend	2	43	1200
Программист Junior Backend	2	43	1200
Программист Intern Frontend	3	42	300
Программист Intern Backend	3	42	300
Итого:	12	512	12 000

Часовая тарифная ставка рассчитывается путем деления месячного оклада ($Т_{Мi}$) на установленный месячный фонд рабочего времени [36]

$$Т_{Чi} = \frac{Т_{Мi}}{МЧФ}, \quad (5.5)$$

где $Т_{ч}{}_{i}$ – часовая тарифная ставка руб.;

МЧФ – месячный часовой фонд (168 ч).

Основная заработная плата разработчиков рассчитывается по формуле [36]

$$З_{о} = \sum_{i=1}^{n} Т_{ч}{}_{i} \cdot Т_{ч} \cdot Ф_{э} \cdot К, \tag{5.6}$$

где n – количество разработчиков, занятых разработкой ПО;

$Т_{ч}{}_{i}$ – тарифная ставка i-го исполнителя в руб.;

$Ф_{э}$ – эффективный фонд рабочего времени i-го исполнителя (дней);

$Т_{ч}$ – количество часов работы в день (8);

К – коэффициент премирования (1,3).

Рассчитаем основную заработную плату разработчиков используя формулы (5.5), (5.6)

$$З_{о} = 2 \cdot (16{,}07 \cdot 8 \cdot 44 \cdot 1{,}3) + 4 \cdot (7{,}14 \cdot 8 \cdot 44 \cdot 1{,}3) + 3 \cdot (1{,}78 \cdot 8 \cdot 42 \cdot 1{,}3) + 4 \cdot (1{,}78 \cdot 8 \cdot 42 \cdot 1{,}3) = 33\,218{,}85 \text{ руб.}$$

Дополнительная заработная плата на конкретное ПО ($З_{Д}$) включает выплаты, предусмотренные законодательством о труде (оплата отпусков, льготных часов, времени выполнения государственных обязанностей и других выплат, не связанных с основной деятельностью исполнителей), и определяется по нормативу в процентах к основной заработной плате [36]

$$З_{Д} = \frac{З_{о} \cdot Н_{Д}}{100}, \tag{5.7}$$

где $Н_{Д}$ – норматив дополнительной заработной платы в целом по организации (15%).

$$З_{Д} = 33\,218{,}85 \cdot \frac{15}{100} = 4982{,}83 \text{ руб.}$$

Следующей статьей сметы затрат являются отчисления на социальные нужды. Отчисления на социальные нужды включают в предусмотренные законодательством отчисления в фонд социальной защиты и фонд обязательного страхования (34%) в процентах от основной и дополнительной заработной платы. Отчисления в фонд социальной защиты населения и

обязательного страхования от несчастных случаев на производстве и профзаболеваний ($Р_{СОЦ}$) рассчитываются следующим образом [36] [39]:

$$Р_{СОЦ} = \frac{З_О + З_Д}{100} \cdot Н_{СОЦ} \quad (5.8)$$

где $Н_{СОЦ}$ – норматив отчислений на социальные нужды.

В итоге используя формулу (5.8) получаем

$$Р_{СОЦ} = (33\,218{,}85 + 4982{,}83) \cdot \frac{34}{100} = 12988{,}57 \text{ руб.}$$

Норма расхода материалов в суммарном выражении ($Н_М$) определяется либо в расчете на 100 строк исходного кода, либо в процентах к основной заработной плате разработчиков (3-5%). Возьмем 3% от основной зарплаты. Следовательно, расходы на материалы рассчитываем следующим образом [36]:

$$Р_М = \frac{З_О \cdot Н_М}{100}, \quad (5.9)$$

где $Н_М$ – норматив отчислений на материалы.

$$Р_М = 33\,218{,}85 \cdot \frac{3}{100} = 996{,}56 \text{ руб.}$$

Расходы на спецоборудование (дополнительное ПО) отсутствуют, т.к. применяются open source решения.

Расходы на оплату машинного времени ($Р_{Мв}$) определяются по формуле [36]

$$Р_{Мв} = Ц_{Мв} \cdot \frac{V_О}{100} \cdot Н_{Мв}, \quad (5.10)$$

где $Ц_{Мв}$ – цена одного машино-часа (0,23 руб.);

$V_О$ – общий объем ПО (строк исходного кода);

$Н_{Мв}$ – норматив расхода машинного времени на отладку 100 строк исходного кода (12 машина-часа).

Средний расход машинного времени 12 часов на 100 строк кода для ПО. Таким образом, расходы на оплату машинного времени равны:

$$Р_{Мв} = 0{,}23 \cdot 21700 \cdot \frac{12}{100} = 598{,}92 \text{ руб.}$$

Расходы по статье «Научные командировки» ($Р_{НК}$) отсутствуют, в связи с ненадобностью в рамках реализации проекта.

Расходы по статье «Прочие затраты» определяются по формуле [36]

$$П_З = \frac{З_О \cdot Н_{ПЗ}}{100}, \quad (5.11)$$

где $Н_{ПЗ}$ – норматив прочих затрат (20%).

В итоге расходы по статье «Прочие затраты» равны:

$$П_З = 33\,218{,}85 \cdot \frac{20}{100} = 6643{,}77 \text{ руб.}$$

Расчет накладных расходов ($Р_Н$) ведется по формуле [36]

$$Р_Н = \frac{З_О \cdot Н_{РН}}{100}, \quad (5.12)$$

где $Р_Н$ – накладные расходы на конкретную ПО (руб.);

$Н_{РН}$ – норматив накладных расходов в целом по организации (30%).

$$Р_Н = 33\,218{,}85 \cdot \frac{30}{100} = 9965{,}65 \text{ руб.}$$

Общая сумма расходов по всем статьям сметы $С_П$ рассчитывается по формуле [36]

$$С_П = З_О + З_Д + Р_{СОЦ} + Р_М + Р_{Мв} + П_З + Р_Н, \quad (5.13)$$

$$С_П = 33\,218{,}85 + 4982{,}83 + 12988{,}57 + 996{,}56 + 598{,}92 + 6643{,}77 + 9965{,}65 = 69395{,}15 \text{ руб.}$$

Организация-разработчик участвует в освоении ПО и несет соответствующие затраты, на которые составляется смета, оплачиваемая заказчиком по договору. Сметой предусматриваются не только затраты (основная и дополнительная зарплата, начисления на зарплату и т.д.), но и налоги, предусмотренные законодательством, и прибыль организации-разработчика ($П_О$) вычисляется по формуле [36]

$$П_О = \frac{С_П \cdot У_{РП}}{100}, \quad (5.14)$$

где $С_П$ – себестоимость;

$У_{РП}$ – уровень рентабельности (15%).

$$П_О = 69395{,}15 \cdot \frac{15}{100} = 10409{,}28 \text{ руб.}$$

Прогнозируемая цена ПО без налогов ($Ц_П$) [36]

$$Ц_П = С_П + П_О, \tag{5.15}$$

$$Ц_П = 69395{,}15 + 10409{,}28 = 79804{,}43 \text{ руб.}$$

Отпускная цена ПО включает в себя налог на добавленную стоимость [36]

$$Ц_о = Ц_п + НДС = Ц_п + \frac{Ц_п \cdot Н_{ндс}}{100}, \tag{5.16}$$

где $Н_{ндс}$ – ставка налога на добавленную стоимость (20%).

$$Ц_о = 79804{,}43 + 79804{,}43 \cdot \frac{20}{100} = 95765{,}32 \text{ руб.}$$

Прогнозируемый экономический эффект у организации-разработчика проекта составляет 80% (за вычетом налога на прибыль, равного 20% на март 2026), следовательно, получаем:

$$ПЧ = П_0 \cdot 0{,}8 = 10409{,}28 \cdot 0{,}8 = 8327{,}42 \text{ руб.}$$

Для упрощения расчетов до составления сметы затраты на освоение определяются по нормативу ($Н_О = 10\%$) от себестоимости ПО в расчет на 3 месяца и рассчитываются по формуле [36]

$$Р_О = \frac{С_П \cdot Н_О}{100}, \tag{5.17}$$

где $Н_О$ – норматив расходов на освоение (%).

$$Р_О = 69395{,}15 \cdot \frac{10}{100} = 6939{,}52 \text{ руб.}$$

Организация-разработчик осуществляет сопровождение ПО и несет затраты в соответствии с договором и сметой на сопровождение. Смета составляется по аналогии со сметой на освоение ПО. Для упрощения расчетов для составления сметы затраты на сопровождение определяются по

установленному нормативу ($Н_С = 20\%$) от себестоимости ПО (в расчете на год) и рассчитываются по формуле [36]

$$Р_С = \frac{С_П \cdot Н_С}{100}, \tag{5.18}$$

где $Н_С$ – норматив расходов на сопровождение (%).

Подставив в формулу (5.19) данные, получаем

$$Р_С = 69395{,}15 \cdot \frac{20}{100} = 13879{,}03 \text{ руб.}$$

Смета затрат на разработку веб-приложения архива сведена в таблицу 5.3.

Таблица 5.3 – Смета затрат на разработку веб-приложения архива

Наименование статей затрат	Условное обозначение	Значение, руб.
Основная заработная плата	$З_О$	33 218,85
Дополнительная заработная плата	$З_Д$	4982,83
Отчисления в фонд социальной защиты населения и обязательного страхования от несчастных случаев на производстве и профзаболеваний	$Р_{СОЦ}$	12988,57
Материалы	$Р_М$	996,56
Машинное время	$Р_{Мв}$	598,92
Прочие затраты	$П_З$	6643,77
Накладные расходы	$Р_Н$	9965,65
Полная себестоимость	$С_П$	69395,15
Прибыль чистая организации-разработчика	$П_О$	10409,28
Прогнозируемая цена ПО	$Ц_П$	89003,27
Отпускная цена ПО	$Ц_О$	95765,32
Затраты на освоение ПО	$Р_О$	6939,52
Затраты на сопровождение ПО	$Р_С$	13879,03

Так как основной целью веб-приложения архива является сохранение исторического наследия. В качестве модели распространения PWYW. Весь функционал архива будет доступен бесплатно, но пользователям будет представлена возможность произвести пожертвование в размере от 1-50 руб. в ответ на которое пользователь может получить дополнительные визуальные элементы к имени на сайте, попасть в открытый список спонсоров, получить

благодарственное письмо или приоритет в очереди на распознание документов, а также другие опции в зависимости от суммы пожертвования.

Примем средний размер пожертвования в 10 руб. и количество пользователей, совершающих пожертвование, в 1%. Получим что необходимое количество пользователей, с учетом периода в 1 год:

$$N = (\text{Ц}_\text{О} + \text{Р}_\text{О} + \text{Р}_\text{С}) \div 10 \div 0{,}01 = 1166139 \text{ чел. год.}$$

Или же 3200 человек в день. Необходимое количество посетителей будет пропорционально уменьшатся при повышении срока окупаемости.

6 ОХРАНА ТРУДА. РАСЧЕТ ЕСТЕСТВЕННОГО ОСВЕЩЕНИЯ В ПРОИЗВОДСТВЕННОМ ПОМЕЩЕНИИ

Целью дипломного проекта является разработка веб-приложения открытого электронного архива с функцией интеллектуального поиска.

Назначение данного приложения – предоставление пользователям возможности публикации и просмотр исторических заметок, различного формата. Платформа представляет собой клиент-серверную архитектуру с использованием NoSQL и S3 хранилищ для организации и обработки данных. Основной функционал включает публикацию материалов с прохождением модерации, интеллектуальную обработку поискового запроса, и составление ленты рекомендаций.

Во время работы над данным сервисом на разработчика могут оказывать неблагоприятные воздействия следующие опасные и вредные производственные факторы:

16 Недостаточное и некачественное освещение приводит к повышенному напряжению зрительного анализатора, снижению остроты зрения, утомлению, а также повышает риск производственных травм.

17 Нарушение показателей микроклимата. Недостаточная вентиляция приводит к накоплению углекислого газа, как следствие снижение умственной работоспособности.

18 Нерациональная организация рабочих. Неправильное размещение оборудования, несоответствие размеров рабочего места антропометрическим данным работника приводят к повышенному утомлению, развитию скелетно-мышечных нарушений.

В настоящем разделе рассмотрим более подробно вопросы, связанные с обеспечением производственного освещения, а именно расчету естественного освещения в производственном помещении.

Удовлетворяющее нормам производственное освещение обеспечивает возможность нормальной производственной деятельности. От него зависят сохранность зрения, состояние центральной нервной системы, безопасность трудовой деятельности. Недостаточная освещённость рабочей зоны или повышенная яркость затрудняют восприятие зрительной информации, создают предпосылки к травмированию. Рациональное освещение является одним из важнейших факторов создания благоприятных условий труда, способствующих повышению производительности труд и качеству выпускаемой продукции.

Естественное освещение является биологически наиболее ценным: глаз человека приспособлен к нему максимально, как следствие негативное влияние, оказываемое на здоровье работника минимально. В связи с этим внимание естественному освещению необходимо уделять даже больше чем искусственному и выбирать рабочее место где оно соответствует принятым нормам.

Основой для выбора нормы освещённости является разрядом выполняемых зрительных работ. Наименьший эквивалентный размер объекта различения на экране монитора = 1 мм, что соответствует Vв разряду зрительных работ, значение нормируемого показателя при естественном освещении (КЕО) для данного разряда приведены в таблице 6.1 [40], [41].

Таблица 6.1 – Норма естественного освещения при IVв разряде зрительных работ

Тип освещения	Значение КЕО $e_{н}$, %
Верхнее или комбинированное освещение	3
Бокове освещение	1

Расчётные показатели КЕО определяются по формуле [3]

$$KEO = \frac{E_{вн}}{E_{нар}} \cdot 100\ \%, \tag{6.1}$$

где $E_{вн}$ – освещённость какой-либо точки внутри помещения;
$E_{нар}$ – освещённость точки вне помещения.

Точка в которой рассчитывается КЕО зависит от характера помещения, поэтому приведём характеристики конкретного производственного помещения, котором происходила разработка веб-приложения. Офис представляет собой open space пространство в близнец центре «Академия» в г. Минске располагается на 9 этаже площадью 192 м^2 (длинна 12 м, глубина 16 м) с высотой потолков в 3,02 м. Окна располагаются с дух сторон помещения: юго-восточной стороны размерами 1,87 м^2 и общим количество в 7 штук и с северо-западной стороны панорамное остекление с общей площадью 32 м^2. Из-за того что офис находится в многоэтажном здании не на последнем этаже, то будем возможно лишь боковое естественное освещение [43]. План-схема расположения рабочих мест приведена на рисунках 6.1.

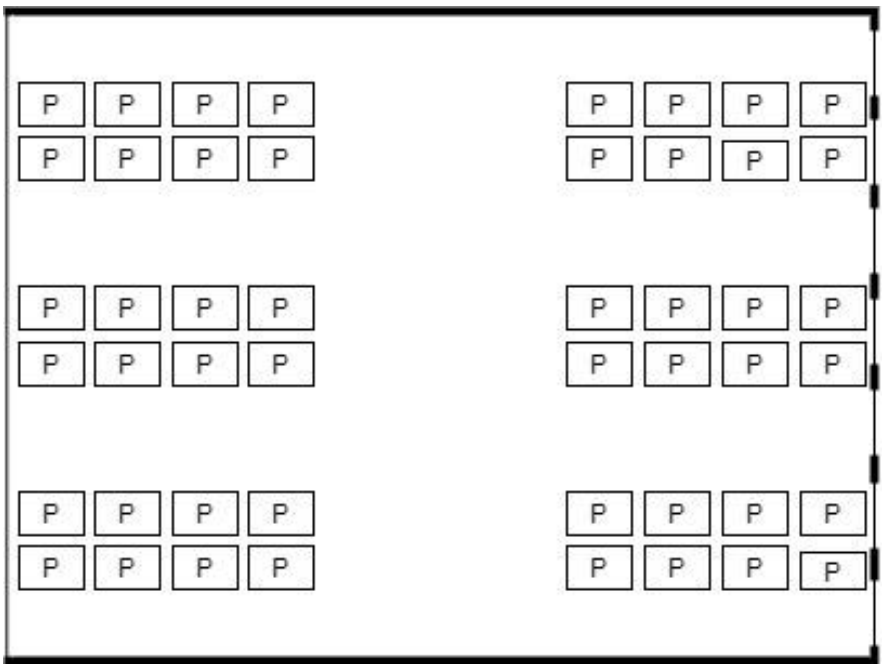

Рисунок 6.1 – План-схема расположения рабочих мест:
Р – одно рабочее место

Для помещений такой планировки, а именно глубиной более 6 м, точка для расчета показателя КЕО выбирается в зависимости от разряда зрительных работ и высоты от пола до верха световых проемов [41]. Зависимость расстояния до точки от разряда работ приведена в таблице 6.2

Таблица 6.2 – Расстоянии от световых проемов до точки нормирования КЕО

Высота от пола до верха световых проемов	Разряд зрительные работ
1,5	I –IV
2,0	V – VII
3,0	VIII

В нашем случае для IV разряда используется точка на удалении 1,5 высоты помещения что равно 4,2 м и является центром крайнего места работника в ряду. Далее для расчётов глубину помещения будем брать исходя из этой точки.

Расчетное значение КЕО при боковом освещении рассчитывается по формуле [41]

$$e_{\text{р}}^{\text{б}} = \left(\sum_{i=1}^{L} \varepsilon_{\text{б}i} \beta_i + \sum_{j=1}^{M} \varepsilon_{\text{зд}i} b_{\text{ф}j} k_{\text{зд}j} \right) \cdot r_{\text{о}} \tau_{\text{о}} \div K_{\text{з}}, \tag{6.2}$$

где L – количество участков небосвода, видимых через световые проемы из расчетной точки;

$\varepsilon_{бi}$ – геометрическое КЕО в расчетной точке при боковом освещении, учитывающий прямой свет от i-го участка неба;

M – количество участков фасадов зданий, расположенных напротив застройки, видимых через световые проемы из расчетной точки;

β_i – коэффициент, учитывающий неравномерную яркость i-го участка облачного неба (по определению Международной комиссии по освещению — МКО);

$b_{фj}$ – средняя относительная яркость j-го участка фасадов зданий, расположенных напротив застройки;

$k_{здi}$ – коэффициент, учитывающий изменения внутренней отраженной составляющей КЕО в помещении при наличии зданий, расположенных напротив застройки;

$\varepsilon_{здi}$ – геометрический КЕО в расчетной точке при боковом освещении, учитывающий свет, отраженный от j-го участка фасадов зданий, расположенных напротив застройки;

r_o – коэффициент, учитывающий повышение КЕО при боковом освещении за счет света, отраженного от поверхностей помещения и подстилающего слоя, прилегающего к зданию;

τ_o – общий коэффициент светопропускания;

$K_з$ – коэффициент запаса.

КЕО используется при расчетах величины световых проемов, поэтому для упрощения расчетов пригодность производственного помещения к установленному разряду зрительных работ произведём путем сравнения реальной и расчетной величин световых проемов.

При боковом одностороннем естественном освещении требуемая площадь световых проемов может быть определена выражением [42]

$$S_o = \frac{e_н K_з \eta_о}{\tau_{об} \rho_о} \cdot K_{зд} \cdot S_п \div 100\ \%, \tag{6.3}$$

где $S_о$ – требуемая площадь световых проёмов при боковом освещении, м^2;

$S_п$ – площадь пола помещения, м^2;

$e_н$ – нормативное значение КЕО, %;

$K_з$ – коэффициент запаса, учитывающий снижение КЕО и освещенности вследствие загрязнения и старения световых проемов (принимаем равным 1,3 в связи с отсутствием загрязнения внутри помещения);

η_o – световая характеристика окон, % (примем равным 17 [42]);

$K_{зд}$ – коэффициент, учитывающий затемнение окон противостоящими зданиями (принимаем равным 1);

$\tau_{об}$ – общий коэффициент светопропускания (принимаем равным 0,98 [42]);

ρ_o – коэффициент, учитывающий повышение КЕО благодаря свету, отраженному от поверхностей (принимаем равным 1,05).

Подставив в формулу (6.2) значения получим необходимые площади освещения

$$S_o = \frac{1 \cdot 1{,}3 \cdot 17}{0{,}98 \cdot 1{,}05} \cdot 1 \cdot 192 \div 100\ \% = 41{,}2\ \text{м}^2$$

Суммарная площадь световых проемов в выбранном производственном помещении равна 45 м^2, что больше нормативного показателя. По строительным нормам отклонение расчетного КЕО может отклоняться от нормативного в диапазоне 10% [2]. Рассчитанная площадь попадает в этот предел, следовательно, разработка системы солнцезащитных устройств не требуется.

В связи с этим, можно признать существующий вариант удовлетворяющим нормам по освещению производственных помещений. Расположение рабочих мест в диапазоне двух высот окон, гарантирует достаточную освещённость на рабочей поверхности. В качестве достоинства существующего решения можно выделить то, что световые проёмы, расположенные с двух сторон, почти полностью убирают тени с рабочих мест разработчиков.

Таким образом, проведенный расчет естественного освещения производственного помещения показывает, что существующий вариант световых проёмов удовлетворяет характеру зрительных работ при разработке веб-приложения архива.

ЗАКЛЮЧЕНИЕ

В результате дипломного проектирования разработано веб-приложение «Открытый электронный архив города Витебска» — краудсорсинговая платформа для сбора, хранения и интеллектуального поиска исторических материалов.

Актуальность разработки обусловлена необходимостью создания централизованной цифровой среды для сохранения культурного наследия города Витебска, объединяющей разрозненные исторические данные в едином пространстве с возможностью коллективного наполнения и экспертной модерации. Существующие решения либо не предполагают активного участия пользователей в создании контента, либо ориентированы на узкие задачи, что обусловливает востребованность комплексной платформы, сочетающей функции архивного хранилища и социальной сети.

Целевая аудитория разработанной системы включает три ключевых сегмента: краеведы и историки (возраст 40–70 лет), нуждающиеся в инструментах систематизации и достоверной атрибуции материалов; обычные горожане (возраст 25–60 лет), стремящиеся делиться семейными архивами и личными историями; модераторы и администраторы из числа сотрудников архивов, библиотек и музеев, обеспечивающие контроль качества публикуемого контента. Для каждого сегмента разработаны карты эмпатии и типовые сценарии взаимодействия, отражающие специфику информационных потребностей и эмоционального опыта пользователей.

Разработанная информационная система реализует следующие функциональные возможности: регистрация и аутентификация пользователей с интеграцией Keycloak; создание и управление архивными материалами с поддержкой метаданных; интеллектуальный полнотекстовый поиск с фасетной фильтрацией и автокоррекцией запросов; многоступенчатая модерация с прозрачной системой статусов и обратной связью через чат; социальное взаимодействие посредством лайков, подписок и персонализированных рекомендаций; автоматическая OCR-обработка отсканированных документов с извлечением текстового слоя. Платформа отличается от существующих аналогов сочетанием функций архива и социальной платформы, интеллектуальным поиском с учётом специфики исторических материалов и встроенной системой оцифровки документов.

Архитектурное решение построено на микросервисном подходе с применением паттерна Backend for Frontend, обеспечивающего чёткое разделение ответственности между клиентской и серверной частями. Технологический стек включает Java 21 и Spring Boot 4.x для серверной разработки, PostgreSQL для транзакционного хранения данных, Elasticsearch для полнотекстового поиска с морфологической обработкой русского и белорусского языков, Apache Kafka для асинхронной межсервисной коммуникации, Redis для кэширования и управления сессиями, Tesseract OCR для распознавания текста, MinIO для объектного хранения файлов и Docker

для контейнеризации. Гибридная схема коммуникации между микросервисами — синхронная через WebClient для критичных по времени операций и асинхронная через Kafka для фоновой обработки — обеспечивает баланс между производительностью и надёжностью системы.

В ходе эргономического проектирования разработаны алгоритмы работы пользователей, формализующие сценарии взаимодействия с системой; сформулированы эргономические требования, включающие психологические и психофизиологические характеристики; разработан сценарий информационного взаимодействия между интерфейсом и пользователем. Экспертная оценка эргономичности по методике анализа групповых эргономических показателей показала значение интегрального показателя эргономичности 0,807, что соответствует оценке «отлично». Выявлены направления для доработки: внедрение диалогов подтверждения для необратимых действий, расширение встроенных подсказок, оптимизация расположения элементов с учётом пользовательских привычек.

Проведено комплексное тестирование, включающее функциональное тестирование серверной части и юзабилити-тестирование. Функциональное тестирование выявило 15 дефектов различной степени критичности, все из которых исправлены. Юзабилити-тестирование экспертным методом по эвристикам Якоба Нильсена подтвердило высокое качество интерфейса (8 из 10 эвристик выполнены полностью), а пользовательское тестирование с участием добровольцев позволило выявить и устранить недочёты в информировании о статусах модерации и логике чата с модератором.

Технико-экономическое обоснование показало целесообразность разработки: прогнозируемая отпускная цена составляет 95 765,32 руб., чистая прибыль организации-разработчика — 8 327,42 руб. при уровне рентабельности 15%. Расчёт естественного освещения производственного помещения подтвердил соответствие существующих световых проёмов нормативным требованиям для IV разряда зрительных работ.

Разработанное веб-приложение достигает поставленной цели — создания открытой краудсорсинговой платформы для сохранения историко-культурного наследия города Витебска с высоким уровнем вовлечённости граждан. Система готова к эксплуатации и обладает потенциалом для дальнейшего развития в направлении расширения форматов исторических записей, разработки мобильного приложения и внедрения машинного обучения для углублённого анализа поисковых запросов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] УК «Витебский областной краеведческий музей» [Электронный ресурс]. – Режим доступа: https://ratusha-vit.by/branches/ratusha. – Дата доступа: 10.12.2025.

[2] Проект «Беларусь помнит, помнит каждого» [Электронный ресурс]. – Режим доступа: https://беларусьпомнит.бел – Дата доступа: 10.12.2025.

[3] Проект «Беларусь помнит, помнит каждого» [Электронный ресурс]. – Режим доступа: https://беларусьпомнит.бел – Дата доступа: 10.12.2025.

[4] FamilySearch [Электронный ресурс]. – Режим доступа: https://www.familysearch.org/ru/global– Дата доступа: 10.12.2025.

[5] Национальный архив Республики Беларусь. – [Электронный ресурс]. – Режим доступа : https://narb.by/ru

[6] Тенденции применения краудсорсинга в архивном деле. – [Электронный ресурс]. – Режим доступа : https://www.pedopyt.ru/categories/19/articles/4942

[7] Kufar.by. – [Электронный ресурс]. – Режим доступа : https://www.kufar.by

[8] Instagram. – [Электронный ресурс]. – Режим доступа : https://www.instagram.com

[9] Library.by. – [Электронный ресурс]. – Режим доступа : https://library.by

[10] Что такое Spring Framework? От внедрения зависимостей до Web MVC. – [Электронный ресурс]. – Режим доступа : https://habr.com/ru/articles/490586

[11] Современный учебник JavaScript. – Режим доступа : https://learn.javascript.ru/

[12] Elasticsearch. – [Электронный ресурс]. – Режим доступа : https://www.elastic.co/elasticsearch

[13] Apache Kafka: основы технологии. – [Электронный ресурс]. – Режим доступа : https://habr.com/ru/companies/slurm/articles/550934

[14] Разбираемся с Redis. – [Электронный ресурс]. – Режим доступа : https://habr.com/ru/companies/wunderfund/articles/685894

[15] Tesseract documentation – [Электронный ресурс]. – Режим доступа : https://tesseract-ocr.github.io

[16] Разбираемся, что такое S3 и делаем простое объектное хранилище – [Электронный ресурс]. – Режим доступа : https://habr.com/ru/companies/cdnnow/articles/840874

[17] Анализ содержания функций и их распределение в СЧКС. – Режим доступа: https://lms.bsuir.by/mod/book/view.php?id=315406. – Дата доступа: 10.12.2025.

[18] Проектирование деятельности пользователей системы [Электронный ресурс]. – Режим доступа: https://lms.bsuir.by/pluginfile.php/568588/mod_resour

ce/content/2/Проектирвоание%20деятельности%20пользовaптелей%20системы.pdf. – Дата доступа: 10.12.2025.

[19] Разработка эргономических требований [Электронный ресурс]. – Режим доступа: https://lms.bsuir.by/pluginfile.php/568593/mod_resource/content/2/Рекомендации%20по%20сост%20перечня%20ЭСТ.pdf. – Дата доступа: 10.12.2025.

[20] ISO 9241-9:2000 [Электронный ресурс]. – Режим доступа: https://www.iso.org/standard/30030.html. – Дата доступа: 10.12.2025.

[21] ГОСТ Р ИСО 14915-2–2016 [Электронный ресурс]. – Режим доступа: https://lms.bsuir.by/pluginfile.php/568595/mod_resource/content/2/4293750537.pdf. – Дата доступа: 10.12.2025.

[22] Эргономическая оценка проектируемой компьютерной системы [Электронный ресурс]. – Режим доступа: https://lms.bsuir.by/pluginfile.php/568600/mod_resource/content/2/Оценка%20эргономичности%20компьютерной%20системы.pdf. – Дата доступа: 10.12.2025.

[23] PostgreSQL Documentation — [Электронный ресурс]. – Режим доступа : https://www.postgresql.org/docs/

[24] MinIO Documentation — [Электронный ресурс]. – Режим доступа : https://min.io/docs

[25] Docker Documentation — [Электронный ресурс]. – Режим доступа : https://docs.docker.com

[26] Keycloak Documentation — [Электронный ресурс]. – Режим доступа : https://www.keycloak.org/documentation

[27] OAuth 2.0 RFC 6749 — [Электронный ресурс]. – Режим доступа : https://datatracker.ietf.org/doc/html/rfc6749

[28] Spring Cloud Gateway — [Электронный ресурс]. – Режим доступа : https://spring.io/projects/spring-cloud-gateway

[29] JWT RFC 7519 — [Электронный ресурс]. – Режим доступа : https://datatracker.ietf.org/doc/html/rfc7519

[30] Юнит тесты Vs функциональное тестирвоание – [Электронный ресурс]. – Режим доступа : https://habr.com/ru/articles/151926

[31] Юзабилити-тестирование сайта – [Электронный ресурс]. – Режим доступа : https://habr.com/ru/companies/testograf/articles/800435

[32] Серьезность и приориет дефекта в тестировании – [Электронный ресурс]. – Режим доступа : https://tquality.ru/blog/sereznost-i-prioritet-defekta-v-testirovanii

[33] Общие правила оформление тест-кейов и их атрибуты – [Электронный ресурс]. – Режим доступа : https://habr.com/ru/companies/sovcombank_technologies/articles/849318

[34] Эвристики Нильсена – [Электронный ресурс]. – Режим доступа : https://simpleone.ru/glossary/nielsen-heuristics

[35] Карта эмпатии: как накладывать эмоции на продукт – [Электронный ресурс]. – Режим доступа : https://habr.com/ru/articles/856408

[36] Палицын, В. А. Технико-экономическое обоснование дипломных проектов / В. А. Палицын. – Минск : БГУИР, 2006. – 76 с.

[37] Ловушка искусственного интеллекта. – [Электронный ресурс]. – Режим доступа : https://habr.com/ru/articles/968444/

[38] Зарплаты в IT. – [Электронный ресурс]. – Режим доступа : https://career.habr.com/salaries

[39] Закон Республики Беларусь 15 июля 2021г. № 118-З. – [Электронный ресурс]. – Режим доступа : https://ssf.gov.by/uploads/folderForLinks/zakon.pdf

[40] Постановление Совета Министров Республики Беларусь 25мянваря 2021г. №37. / Совет Министров Республики Беларусь. – Минск, 2021. – 1255 с.

[41] Строительные Нормы Республики Беларусь СН 2.04.03-2020. Естественное и искусственное освещение зданий и сооружений : строительные нормы Республики Беларусь / Министерство архитектуры и строительства Республики Беларусь. – Минск, 2020. - 86с.

[42] Безопасность жизнедеятельности человека : учебно-методическое пособие : в 3 ч. Ч. 3 : Охрана труда / И. А. Телеш [и др.]. - Минск : БГУИР, 2022. - 162 с. : ил.

[43] Постановление Министерства здравоохранения Республики Беларусь 28 июня 2012 г. №82 «Об утверждении санитарных норм и правил» / Министерство здравоохранения Республики Беларусь. – Минск, 2012. - 67 с.

ПРИЛОЖЕНИЕ А
(обязательное)
Отчет о проверке на заимствование

АНТИПЛАГИАТ
ОБНАРУЖЕНИЕ ЗАИМСТВОВАНИЙ

Отчет предоставлен сервисом «Антиплагиат» - http://cs-kby.antiplagiat.ru

Отчет о проверке

Автор: Бабич Александр Сергеевич

Название документа: ELASTICSEARCH. ПЕРЕХОД К NOSQL РЕШЕНИЯМ ДЛЯ ОРГАНИЗАЦИИ ЭФФЕКТИВНОГО ПОИСКА В БОЛЬШИХ ДАННЫХ

Проверяющий: Черных Елена Константиновна

Организация: Центральная научная библиотека Национальной академии наук Беларуси

РЕЗУЛЬТАТЫ ПРОВЕРКИ

Совпадения: 2,85%

Оригинальность: 97,15%

ИИ-контент: 0%

Цитирования: 0%

Самоцитирования: 0%

«Совпадения», «Цитирования», «Самоцитирования», «Оригинальность» являются отдельными показателями, отображаются в процентах и в сумме дают 100%, что соответствует проверенному тексту документа.

Проверено: 81,05% текста документа, исключено из проверки: 18,95% текста документа. Разделы и элементы, отключенные пользователем: Титульный лист, Библиография, Приложение, Таблицы

- **Совпадения** — фрагменты проверяемого текста, полностью или частично сходные с найденными источниками, за исключением фрагментов, которые система отнесла к цитированию или самоцитированию. Показатель «Совпадения» – это доля фрагментов проверяемого текста, отнесенных к совпадениям, в общем объеме текста.
- **Самоцитирования** — фрагменты проверяемого текста, совпадающие или почти совпадающие с фрагментом текста источника, автором или соавтором которого является автор проверяемого документа. Показатель «Самоцитирования» – это доля фрагментов текста, отнесенных к самоцитированию, в общем объеме текста.
- **Цитирования** — фрагменты проверяемого текста, которые не являются авторскими, но которые система отнесла к корректно оформленным. К цитированиям относятся также шаблонные фразы; библиография; фрагменты текста, найденные модулем поиска «СПС Гарант: нормативно-правовая документация». Показатель «Цитирования» – это доля фрагментов проверяемого текста, отнесенных к цитированию, в общем объеме текста.
- **Текстовое пересечение** — фрагмент текста проверяемого документа, совпадающий или почти совпадающий с фрагментом текста источника.
- **Источник** — документ, проиндексированный в системе и содержащийся в модуле поиска, по которому проводится проверка.
- **Оригинальный текст** — фрагменты проверяемого текста, не обнаруженные ни в одном источнике и не отмеченные ни одним из модулей поиска. Показатель «Оригинальность» – это доля фрагментов проверяемого текста, отнесенных к оригинальному тексту, в общем объеме текста.

Обращаем Ваше внимание, что система находит текстовые совпадения проверяемого документа с проиндексированными в системе источниками. При этом система является вспомогательным инструментом, определение корректности и правомерности совпадений или цитирований, а также авторства текстовых фрагментов проверяемого документа остается в компетенции проверяющего.

Рисунок А.1 – Результат проверки на заимствования в системе «Антиплагиат»

ПРИЛОЖЕНИЕ Б
(обязательное)
Листинг программы

Веб-приложение «Открытый электронный архив истории г. Витебска» и его эргономическое обеспечение

Листинг программы

ГУИР.508115

Листов XX

СОДЕРЖАНИЕ

Б.1 xx
Б.2 ……..... xx
Б.3 xx